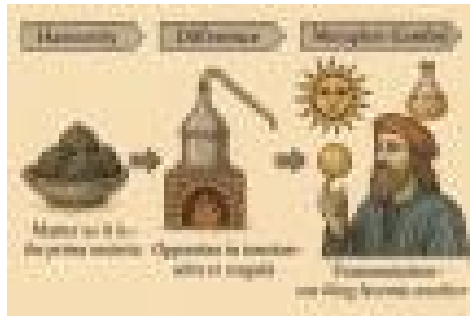


Chevron Networks - An AI Journey



A series of posts from the [andrekraternmsc](#) substack.



Notes on Chevron Networks

A novel 2x2 Operator Neural Network Architecture for 2-channel Oppositional Meaning

[Andre Kramer](#)

Feb 20, 2026



(Fig 1. Carved lintel stone with chevrons from the chamber of the Newgrange passage grave. It can be read as comparing vanilla neural networks (top right) to dual chevron nets.)

I've been thinking about a new form of artificial neural network—one designed to represent and transform **differences** directly, rather than treating “difference” as something that only emerges after layers of scalar weights and nonlinearities. This idea is an outgrowth of my longer exploration of **meaning** and **self** as a response to deep neural network AI development. In a sense, the project has become recursive: I started by watching neural networks reshape our theories of cognition, and I've ended up asking what kind of network architecture might better reflect the dynamics of interpretation, tension, selection, and retention that we associate with minds. So: we are recursing.

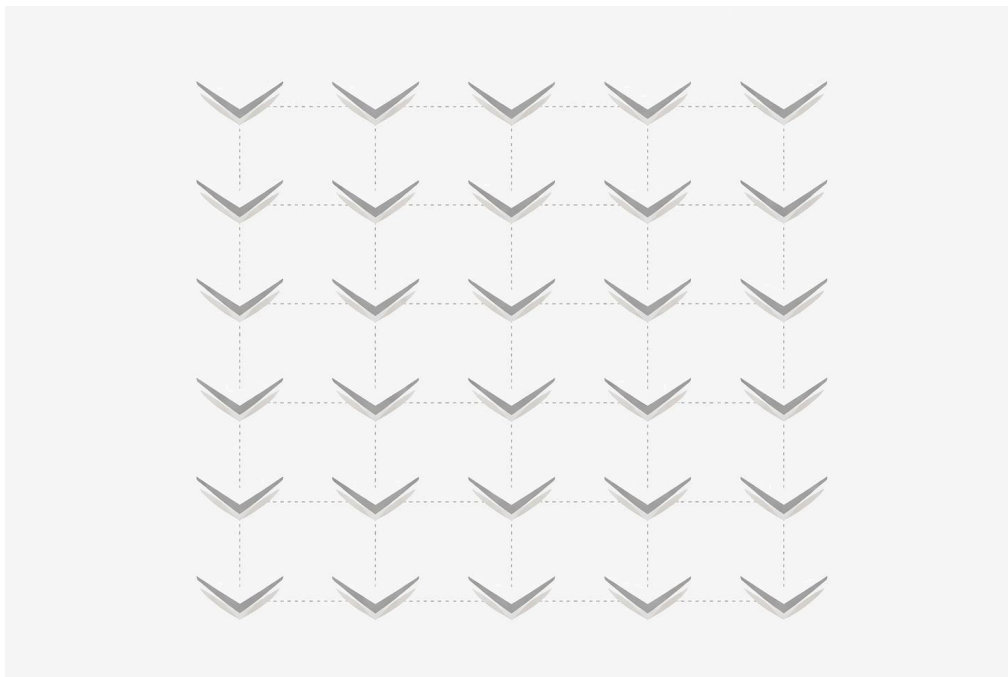
What follows are my initial thoughts and working notes on what I'm calling **chevron networks**: a small, structured upgrade to the standard

neuron-and-weight picture. The wager is that by giving the network an explicit language for opposites, polarity, and relevance—and by letting connections act as tiny transforms rather than mere scalars—we get a more fertile ground for experimenting with self-learning agents that can evolve slowly, retain structure, and avoid the worst category errors of purely associative systems.

Yes, it's a wild claim. But sometimes the only way to find the next small lever is to try a slightly wild one and see where it snaps.

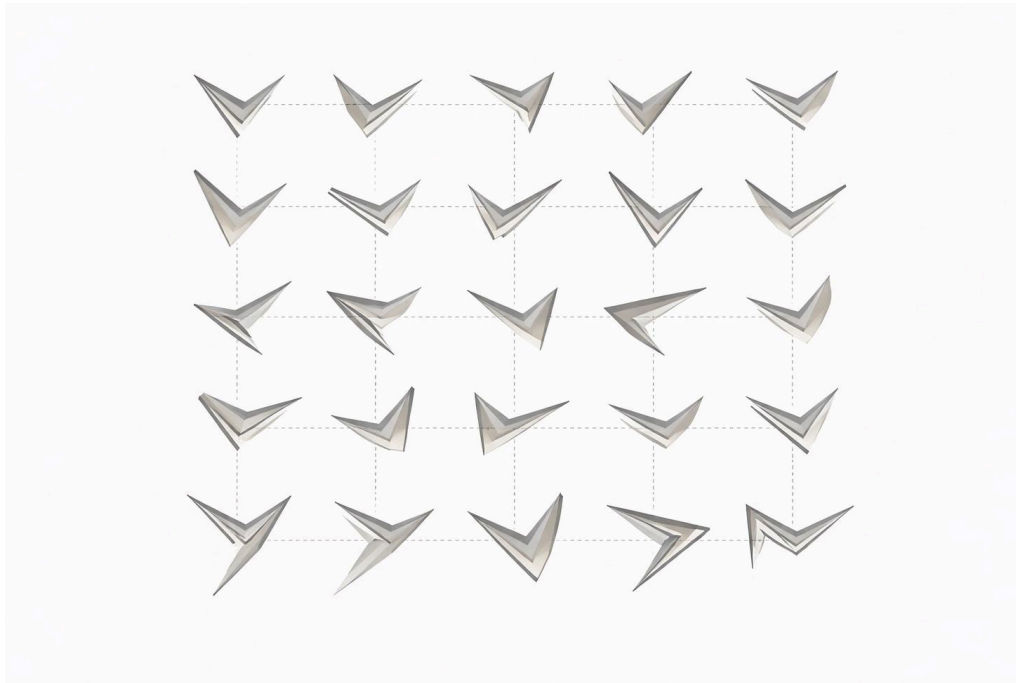
A **chevron** is the simplest “two-armed” geometry: two segments joined at a vertex with an internal acute angle, plus an overall orientation in the plane. In its default form the arms are equal length, the internal angle is small (e.g., $\approx 30^\circ$), and the chevron is oriented horizontally—but in general the shape carries **four degrees of freedom** (two arm lengths, the internal angle, and a global orientation angle). We use this geometry as a motivating primitive for a new network substrate.

The [Hypercube of Opposites](#) models meaning as a single scalar “tension” along an oppositional axis; chevrons generalize this by allowing **two coupled values** to flow—e.g., a complex number, or a 2-vector/tuple capturing phase and magnitude, belief and uncertainty, evidence and inhibition. Operationally, this corresponds to networks whose activations are 2-vectors and whose connections are 2×2 operators (“chevron nodes”), extending the expressive power of standard deep neural networks while remaining plausibly closer to biological organization (where signals are rarely single scalars and where relative phase, error, and gating matter).



(Fig 2. Chevrons in the default initial position in a non-sparse network)

Chevron nets therefore extend our earlier ideas on oppositional meaning and tension into a concrete experimental architecture: a minimal, natural generalization of DNNs that supports richer internal degrees of freedom, and that may serve as a substrate for new AI experiments in robustness, learning-from-error, and meaning-making.



(Fig 3. Chevrons in rotated and extended orientations after updating. Some seem to have been caught in the act of mutation and gained another arm. Interesting but for now, let's stick to the rules below.)

The Chevron Network Core

- **Node state:** each concept/node i carries a 2-vector

$$\mathbf{x}_i = \begin{bmatrix} x_i^+ \\ x_i^- \end{bmatrix}$$

- $\mathbf{x}_i = [x_i^+ \ x_i^-]$
("two opposites" channels)
- **Edge (chevron) operator:** each directed link $j \rightarrow i$ is a 2×2 matrix

$$\mathbf{W}_{ij} = \begin{bmatrix} w_{++} & w_{+-} \\ w_{-+} & w_{--} \end{bmatrix}$$

- $\mathbf{W}_{ij} = [w_{++} \ w_{+-} \ w_{-+} \ w_{--}]$

(four corner couplings; complex scalar is a constrained special case)

Each chevron edge applies a 2×2 transform to the sender's two-channel state (optionally gated by relevance), producing a two-channel message; nodes simply sum these incoming messages (plus inertia and input) and pass the result through a nonlinearity before broadcasting again to the next layer or recurrent step.

- **Update (dense or message passing):**

$$x_i \leftarrow \sigma \left((1 - \lambda) x_i + \sum_{j \in \mathcal{N}(i)} g_{ij}(t) W_{ij} x_j + I_i(t) \right)$$

-
- $x_i \leftarrow \sigma((1-\lambda)x_i + \sum_{j \in \mathcal{N}(i)} (g_{ij}(t) W_{ij} x_j + I_i(t)))$
 - λ : inertia/leak (retention)
 - $g_{ij}(t) \in [0,1]$: relevance gate (attention-like routing)
 - $I_i(t)$: input clamp / sensory drive

This is the single “engine.”

A useful feature of the chevron form is that the **2×2 operator W_{ij}** gives a surprising amount of design freedom without changing the outer learning loop.

In the update above: $x_i \leftarrow \sigma((1-\lambda)x_i + \sum_{j \in N(i)} (g_{ij}(t) W_{ij} x_j + I_i(t)))$,

each node state is two-channel, and each connection is a small transform rather than a single scalar weight. This means we can implement different internal “micro-rules” by choosing how W_{ij} mixes channels (within-channel reinforcement, cross-channel veto/disinhibition, rotation-like phase mixing as a constrained special case) and by letting the gate $g_{ij}(t)$ depend on a second-channel quantity such as relevance, precision, or surprise. In that sense, a chevron edge can be viewed as a compact, local analogue of the R-rule: two streams combined by gating and phase/polarity-sensitive mixing, with learning driven by a temporal-difference signal. A natural next step is to introduce **two time-scales**—fast updates for the gating/eligibility channel and slower consolidation for the core couplings—so adaptation happens mainly

through modulation and TD-style correction, while the retained structure evolves cautiously rather than being overwritten.

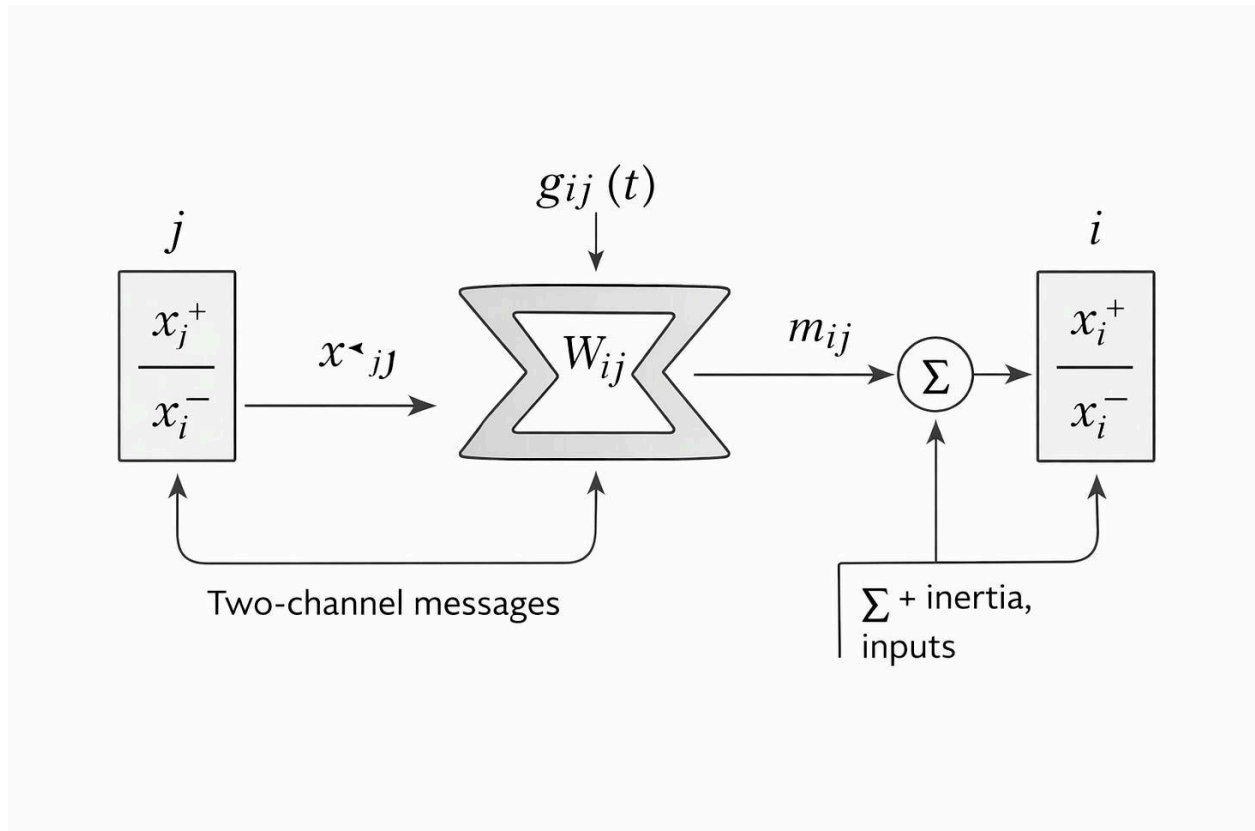


Figure / diagram caption:

Each node j carries a two-channel state $x_j = [x_j^+, x_j^-]$. A directed chevron edge $j \rightarrow i$ stores a 2×2 operator W_{ij} (optionally gated by relevance $g_{ij}(t)$) and sends a two-channel message $m_{ij} = g_{ij}(t) W_{ij} x_j$. Each node i sums

incoming messages (plus inertia and any input drive) and applies a nonlinearity before broadcasting its updated two-channel state to downstream chevrons.

Interpretations of the Same 2-Vector

1. **Evidence / polarity channels (E/I-like):** (x_+, x_-) = pro vs con.

Collapse to belief via $\ell = x_+ - x_-$, $p = \sigma(\ell)$.

2. **Belief + uncertainty:** (μ, u) where u is confidence/precision (or $\log \sigma$).

Useful for “grain choice” and “variety absorbs variety” as explicit uncertainty.

3. **Amplitude-like (interference):** treat (x_+, x_-) as a 2D amplitude plane; readout can be $|x|$ or $|x|^2$.

Not QM-claims—just cancellation/constructive combination.

4. **Value + relevance (attention / surprise)**
5. Base + carrier (reference signal)
6. Proportion / ratio (composition)
7. a complex number
8. ...

Channel 1 carries content (value/evidence); Channel 2 carries participation (relevance/precision), and a chevron edge is a 2×2 operator that can mix them (e.g., relevance can inhibit content; content can recruit relevance).

Ratios let us generalize “tension” beyond a single real number. Instead of storing only $p \in [0,1]$, each concept can carry a two-channel mass (a,b) , so the *tension* becomes a proportion $p = a / (a+b)$, or, more usefully, a signed log-ratio $\ell = \log a / b$. This makes relations compositional: ratios multiply cleanly, and log-ratios add, so chains of constraints naturally accumulate into meaning rather than remaining isolated scalars. In a chevron network, we can propose ratio-links between any two concepts (as 2×2 operators that transform (a,b) across connections), then let relevance gating and [inertial VSR](#) decide which ratios are stable enough to retain—turning “any relation is possible” into “only useful relations survive.”

A final, optional interpretation is affect-like modulation: channel two can represent salience/urgency/valence signals that gate attention and plasticity.

We mean this purely functionally—as control and prioritization—not as a claim about machine experience.

Breakout: can chevrons be trained with backprop?

Yes. At a minimum, a chevron layer is just a differentiable computation: two-channel node states, a 2×2 transform per edge, optional gating, a nonlinearity, and (if desired) recurrence. That means it can be trained end-to-end with ordinary gradient descent/backprop exactly like other neural layers. In practice, the 2×2 structure can be implemented as block-sparse matrix multiplication or message passing, both of which are standard autodiff workloads. We can also partially freeze weights (e.g., keep a stable core and learn only cross-couplings or gates), or constrain 2×2 blocks to special cases (rotation+scale) to make early experiments easier. None of this proves chevrons are *better*—but it does mean the architecture is testable with today's tooling, and we can compare it fairly

against familiar baselines before exploring more biologically oriented learning rules.

Backprop is the baseline; the longer-term interest is whether **local, gated, multi-timescale updates**—VSR/TD/eligibility-style—can recover similar capability while improving robustness and interpretability, and ultimately **leverage the chevron structure more fully** than a generic end-to-end gradient ever would.

Inertial VSR [Mapping](#)

- **Inertia/Retention:** the leaky carry-forward $(1-\lambda)x_i$, plus stored W_{ij} and stable adjacency.
- **Variation:** inject proposals/perturbations (new edges, modified W_{ij} , noise in x , exploratory gates g).
- **Selection:** multiplicative reweighting via gates/compatibility (high g_{ij} edges dominate); prune weak contributors; normalize activity budget.
- **Retention (again):** consolidate high-utility chevrons (strengthen/keep), decay or delete the rest.

Stepwise Tests (Capabilities)

1. Category-veto / avoid category errors (distribution shift):

Show gated 2×2 chevrons resist “shortcut features” better than scalar baselines.

2. Holographic-ish associative recall:

Store many key-value bindings; test capacity + graceful degradation; show 2×2 cross-coupling improves interference control.

3. Attention-like contextual routing:

Context selects which relations apply; gates $g_{ij}(t)$ implement routing akin to attention, with interpretability (“which chevrons lit up”).

Implement either as:

- **block-sparse matrix ops** (fast first approximation), or
 - **explicit message passing** (ID/associative-memory native form).
-

Recurrence comes “for free” in a 2-channel chevron

Once each concept carries a 2-vector (x_+, x_-) and each link is a 2×2 chevron operator, recurrent dynamics aren't an add-on — they're the default. A loop can live **within a channel** (content feeding content), **across channels** (content recruiting relevance, relevance gating content), or **mixed** (continuous circulation between channels). Diagonal 2×2 couplings give two largely independent recurrent streams, while off-diagonals create cross-coupled motifs: inhibition/veto, disinhibition, gain control, and even oscillation-like alternation. This gives a clean way to separate (or intentionally blend) “what is being represented” with “whether it should participate now” — e.g. **value + relevance, belief + precision**, or **signal + eligibility trace** — while keeping all updates local and compatible with an inertial Variation–Selection–Retention loop.

Thesis and antithesis in the encoding

A simple but important consequence of chevrons is that a distinction can be represented *without immediately collapsing it*. Instead of forcing every

concept into a single scalar activation, each node carries a two-channel state:

$$\mathbf{x} = [\mathbf{x}_+$$

$$\mathbf{x}_-]$$

This can be read directly as **thesis vs antithesis** (or a proposition and its negation): the system can keep “support for P” and “support for $\neg P$ ” alive at the same time, letting context and constraints act on both channels in parallel.

To recover a conventional “belief,” we can collapse the pair when needed:

$$\ell = \mathbf{x}_+ - \mathbf{x}_- \Rightarrow p = \sigma(\ell)$$

where the sign of ℓ gives the direction of the stance (thesis vs antithesis) and its magnitude gives strength. Meanwhile the sum,

$$\mathbf{c} = \mathbf{x}_+ + \mathbf{x}_-$$

can be treated as **commitment/salience**: a measure of how active or consequential the distinction is, even when the net stance is near zero.

This encoding matters because it allows the network to do something ordinary scalar networks struggle with: it can **represent ambivalence and contradiction explicitly**. Rather than averaging away conflict, chevrons preserve it as structure. Over time, gating and 2×2 mixing can select, suppress, or retain one side—supporting a more “dialectical” form of learning in which hypotheses compete, co-exist, and only gradually collapse into decisions when the evidence warrants it.

Tension Layers as Chevron Networks

Tension Layers (as we previously [described](#) in this substack) can be implemented naturally as a **Chevron network** by treating each tension axis as a two-channel “opposites” state (+,-) (e.g., p vs 1-p, or evidence-for vs evidence-against). The Tension Layer’s core update—**inertial retention plus uncertainty-scaled correction toward a**

proposed value—becomes a local chevron state update, while the layer's **salience/attention** mechanism becomes an explicit **gating field** over chevrons. Stacking Tension Layers corresponds to composing chevron layers; adding sparse 2×2 chevron couplings between axes provides a principled upgrade path for semantic relations, inhibition/veto (category-error control), and attention-like routing—without changing the underlying inertial VSR dynamics.

Proposition (Chevron Actor–Critic coupling):

A chevron network can implement Actor–Critic learning intrinsically by representing each unit's state as a two-channel $[a, c]$ vector and each connection as a 2×2 mixing operator, with tension defined by actor–critic disagreement and used to gate selection, learning, and exploration.

In a 2×2 chevron, we can explicitly separate **what flows** from **what learns**. One channel can carry the semantic/content signal, while the

other carries a training-side quantity—relevance, surprise, uncertainty, or an eligibility trace—that gates when and where learning occurs. This also makes partial plasticity straightforward: each chevron edge has four couplings $\{w_{++}, w_{+-}, w_{-+}, w_{--}\}$, and we can **freeze** any subset during training. For example, learning only the diagonals preserves two largely independent streams; learning only the off-diagonals turns the second channel into a learned veto/disinhibition mechanism (category-error control); constraining the 2×2 to the complex “rotation+scale” form recovers a phase-like special case. In short, chevrons let us treat wiring motifs as *retained structure* while allowing controlled *variation* in specific couplings—an engineered form of inertial VSR.

Any setting where you keep a **core set of weights fixed** (or only **slowly varying**)—continual learning, agent “personality”/skills retention, world-model stability—can benefit from dual channels and 2×2 chevrons. The reason is simple: a chevron doesn’t just scale a signal, it can **separate and transform two coupled streams**. One channel can carry the stable semantic/content flow while the other carries a plastic,

context-sensitive control signal—relevance, surprise, uncertainty, eligibility—so adaptation happens by **gating and cross-coupling** rather than overwriting the core representation. Practically, this also lets us freeze (or slow) specific parts of the 2×2 operator: keep the diagonals as a retained backbone, let off-diagonals learn category veto and routing; or keep cross-couplings stable while the gain terms adapt. That combination—**slow core + fast contextual modulation**—is the promise to explore next, because it offers a plausible path to continual learning without catastrophic drift: retention is built into the wiring, and variation/selection operates in a controlled secondary channel.

The [R-rule](#) already contains the ingredients we want: **gating**, **phase**, and **phase difference**. This is clearest in its algebraic form ([link](#)), where the update is explicitly modulated rather than applied uniformly. In the rotor form ([link](#)), **interference** and secondary, rotation-like effects are also implicated—but any concrete implementation has to actually *realize* those dynamics, not just gesture at them. This is where the chevron

network becomes useful. By treating each concept as a two-channel state and each connection as a small operator, the chevron model lets phase-like interactions be expressed and manipulated directly (mixing, vetoing, rotating, and gating), rather than being squeezed into a single scalar weight. We see this as a more fertile experimental ground for building [self-learning agents](#): fast local updates and attention-like selection on top of a **slowly evolving, inertial core** that can accumulate structure without becoming brittle.

We might say that much of modern machine learning is still living in the age of **direct current**: scalar activations pushed forward, scalar errors pushed back, and a largely one-way flow of influence. The chevron perspective invites a gentler upgrade—not a revolution, but an added degree of freedom—something closer to **alternating current**, where *phase* and *phase difference* matter as much as magnitude. If you've followed my Substack explorations of cycling, recurrence, and updates across a space of opposites (and my Hypercube of Opposites [book](#)), this

won't sound exotic: oppositional systems naturally generate oscillation, interference, inhibition, and resonance-like stabilization. I'm certainly not claiming Tesla-level insight over Edison; the analogy is meant as a prompt, not a boast. But it's a useful prompt: if we want agents that learn by iterating, revising, and rebalancing tensions over time, then giving the machinery an explicit notion of polarity/phase—and a structure that can rotate and gate those signals—may be an unusually productive line of experimentation.

Multiple time-scales: temporal difference learning, fast/slow channels, and memory

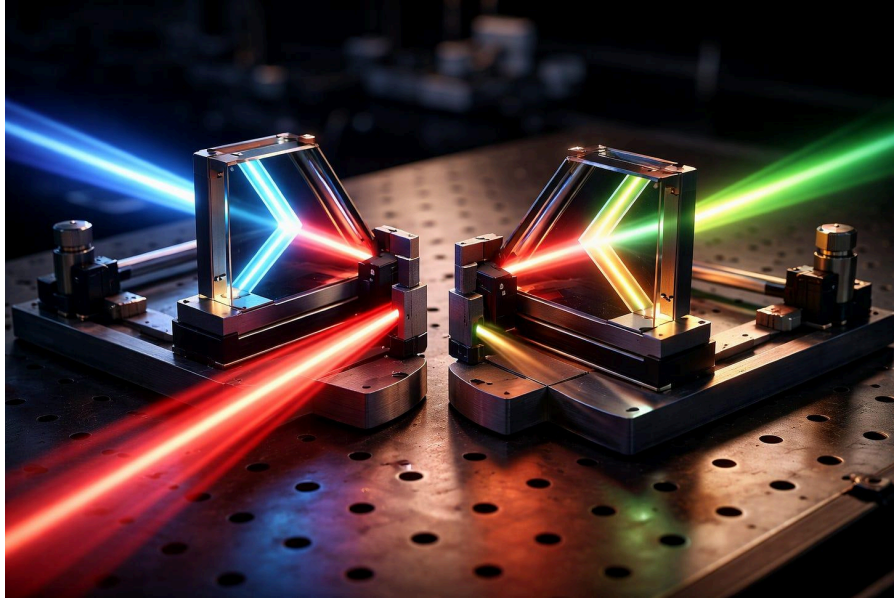
A natural advantage of chevrons is that they invite **two (or more)** **time-scales** without adding much machinery. Temporal-difference learning already depends on *time*: a fast prediction is compared to a slightly delayed outcome, and the difference drives an update. In our framing, the two channels can carry these roles explicitly. One channel can be **fast**—the immediate, high-bandwidth “A” pathway (rapid action

prediction, quick eligibility changes). The other can be **slow**—a stabilizing “N” pathway (baseline norms, constraint, longer-horizon value, or slow-moving uncertainty/precision). This maps cleanly onto the R-rule intuition of [fast A versus slower N](#), and also onto short-term versus long-term memory: fast dynamics explore and react; slow dynamics consolidate and resist drift.

Practically, this is implemented by giving the channels different leak/retention constants (and optionally different learning rates): the fast channel updates quickly and forgets quickly; the slow channel updates cautiously and persists. Cross-coupling between channels then becomes a simple mechanism for credit assignment and control: fast surprise can open gates and write tentative structure, while the slow channel decides what is retained as the agent’s enduring model. This multi-rate design is a direct way to build inertial VSR into TD-style learning: **variation happens in the fast stream, selection is gated by prediction error and relevance, and retention is expressed as slow consolidation into the core.**

Possible alternative implementations

Chevron networks don't have to live purely in conventional digital silicon. Because the core primitive is a **two-channel state** acted on by a **2×2 transform**, they map naturally onto several “physics-first” substrates. A near-term route is **hybrid photonics**: interferometric 2×2 mixers (phase + magnitude) do the linear chevron coupling efficiently, while electronics provides nonlinearities, gating, and slow selection/retention control. An equally plausible route is **analog / in-memory computing**, where each 2×2 chevron can be realized as four conductances (the four corner couplings), with the second channel used as gain/eligibility gating. Beyond standard CMOS, emerging devices—memristive elements, phase-change, ferroelectric, or spintronic components—could encode slowly evolving “core” couplings and local plasticity. At this early stage, the point isn't to commit to one substrate (optics for example), but to note that chevrons are unusually hardware-friendly: they're small operators with an explicit role for phase/polarity, recurrence, and controlled plasticity.



Biological overlap: opponent channels, 2×2 motifs, and holographic echoes

Opponent channels are already a biological primitive. Early sensory systems often represent *differences* directly rather than absolute quantities. The retina and early vision use ON/OFF pathways and opponent encodings (in luminance and colour), which is conceptually very close to a two-channel chevron state $(+,-)(+,-)(+,-)$: not “a value,” but “a value as a contrast.” The same pattern appears more broadly in action selection circuits, where control is naturally expressed as antagonistic

pushes and pulls (e.g., Go/No-Go style dynamics in basal ganglia). In other words: two-channel representations are not an exotic choice—they're a common way biology turns a world of continuous inputs into actionable distinctions.

2×2 mixing is the conventional way to write local circuit dynamics. If you look at classic neural population models, the minimal unit is often an excitatory population coupled to an inhibitory population. The effective connectivity is literally a 2×2 matrix: $E \rightarrow E$, $I \rightarrow E$, $E \rightarrow I$, $I \rightarrow I$. That is exactly the “four-corner” chevron operator. It also lines up with everyday cortical motifs: inhibition is not merely “negative weight,” but gain control, normalization, and veto. A 2×2 edge/operator lets us express these motifs explicitly—reinforcement, suppression, cross-coupling, disinhibition—rather than forcing everything into a single scalar synapse.

There are also ‘holographic’ and phase-like echoes in mainstream neuroscience. If by holographic we mean “a reference signal plus an encoded signal,” then oscillatory dynamics provide a grounded analogue:

spikes are often interpreted relative to an ongoing rhythm (a phase reference), and timing relationships can carry structured information. Interference-like effects show up naturally when multiple oscillatory influences combine, and phase relationships can stabilise or destabilise patterns over time. In cognitive modelling, holographic associative memory ideas (e.g., binding/unbinding via superposition and correlation) provide another nearby language for “reference + payload,” and chevrons can act as a minimal operator substrate for exploring those mechanisms without committing to any one metaphysics.

The takeaway: chevrons are not “biology in a box,” but they rhyme with several conventional biological themes: opponent coding, E/I-style coupling motifs, normalization/veto control, and phase-referenced representations. That makes them a plausible *structural* bridge between semantics (differences, relations, relevance) and dynamics (recurrence, gating, resonance) in a way that standard scalar-weight networks make harder to express directly.

AI Safety Angle

I don't want to overclaim the AI-safety implications of any of this. A new architectural pattern is not an alignment solution, and nothing here guarantees benign behaviour. The motivation is narrower and more engineering-like: **reliability and robustness**. Carrying an explicit reference point in the computation—whether as a second channel for negation, uncertainty/precision, relevance gating, or a stable baseline—gives the system a built-in way to represent “I might be wrong,” “this distinction may not apply,” or “here is the counterweight.” That is less about values and more about reducing familiar failure modes: brittle generalisation, category errors, runaway feedback, and overconfident inference. And it may help with non-functional requirements we can actually test: **traceability** (which edges/chevrons were active), plus a more direct representation of **opposition**, **grounding/baseline**, and even **valence-like modulation** as an explicit internal variable rather than an emergent side-effect.

So the hope is modest: chevrons may offer a convenient handle for “non-functional requirements” we already care about—interpretability (which edges/chevrons were active), controllable plasticity (what is fixed vs learnable), and graceful degradation under distribution shift. If the architecture can make those properties easier to test and enforce, that’s valuable even if it never becomes a dominant paradigm. And if it doesn’t, the experiments will still have been worth running—because they’ll tell us, quickly and concretely, where the idea breaks.

This may read like a list of potential uses for an unproven architectural pattern. That’s fair. My claim is narrower: the chevron idea feels *novel enough, and concrete enough*, to be worth putting on the table for direct experimentation. And it’s the kind of experimentation that doesn’t require frontier-scale training runs or an arms-race mindset—many of these conjectures can be tested quickly with small models, toy tasks, and careful ablations. In fact, at a time when leading coding models can already help implement and iterate on these prototypes, it should be

possible to prove or disprove the core ideas without adding much heat to an already overheating AI race (Timing is everything!). If any of this resonates and you'd like to contribute—analysis, implementations, or alternative framings—please get in touch.

I'll put the text for this article and any code on github ([here](#)).

Andre with ChatGPT-5.2,

February 2026

23/2 Updates

Weight packing

One practical sweet spot is to treat each chevron edge as an atomic 2×2 operator and pack its four weights as $4 \times \text{FP8} = 32 \text{ bits}$. That gives excellent memory locality: one aligned 32-bit fetch brings in a complete “micro-transform” (plus its cross-couplings), which is attractive for

bandwidth-bound inference. The trade-off is that a dense chevron layer has **4× the parameters and ~4× the multiply-adds** of an equivalent scalar layer at the same width. Even with FP8, that means **weight memory is ~2× larger than a vanilla FP16 layer** (because 4×1 byte vs 1×2 bytes per connection), though it can still be smaller than higher-precision baselines and much more cache-friendly. Performance then hinges on kernel quality: if the chevron ops compile down to **batched/block-sparse GEMMs** that hit FP8 tensor cores, the extra arithmetic can be clawed back; if the implementation devolves into irregular gathers and dequant overhead, FP8 may not help much. Either way, it's testable: a good first benchmark is to run a chevron-MLP/chevron-attention toy (plus category-veto OOD test) and measure not just accuracy/robustness, but also effective throughput and memory bandwidth under FP16 vs FP8 packing.

Chevrons as micro-attention and micro-crossbars

A useful way to picture a chevron network is as a fabric of tiny routing elements rather than a pile of scalar synapses. Each edge doesn't just "weight" a signal; it **routes and mixes** a two-channel state from source to target. If a node carries $x=[x_+,x_-]$, then a chevron connection applies a 2×2 operator W so that each output channel receives a controlled blend of the two inputs. In networking terms, this looks like a miniature **2×2 crossbar switch**: four crosspoints ($w_{++},w_{+-},w_{-+},w_{--}$) determine how much of each input channel is sent to each output channel. That makes it natural to interpret diagonal terms as reinforcement within a pole, and off-diagonals as cross-coupling motifs such as veto, disinhibition, or "phase-like" rotation/mixing in the opposition plane.

This same picture also connects directly to attention. Attention is fundamentally a policy for **which connections participate right now** and how strongly; chevrons make this explicit with a gate $g_{ij}(t)$ multiplying the edge transform. The message along an edge is $m_{ij}=g_{ij}(t)W_{ij}x_j$: the gate plays the role of a context-dependent relevance weight, while the 2×2 operator plays the role of a local value transform that can encode

structured interaction (reinforce, suppress, rotate, or normalize) rather than a single scalar scaling. In other words, a chevron edge is a small, interpretable “micro-attention block”: it decides *whether* a relation is active and, if active, *how* it routes two streams of information.

This framing is useful because it suggests an engineering sweet spot: a **slowly varying core** of chevron operators (a stable routing fabric) combined with **fast, context-driven gating** (dynamic scheduling). That is exactly the kind of separation we want for continual learning and inertial VSR: retain durable structure in the operator backbone, while letting attention-like gates and short-timescale traces adapt quickly to novelty and task demands.

Calling out Hallucinations

My original notes on chevrons do not explicitly address hallucinations in large language models, but the architecture naturally suggests a possible extension. If each node carries two channels rather than one, the second

channel does not have to be reserved only for opposition or inhibition. It can also carry a **meta-signal**: confidence, uncertainty, residual error, counter-evidence, or even a simple “what if I’m wrong?” trace. In other words, the network can be structured so that it not only computes a content signal, but also carries some explicit representation of how well-supported that signal is.

This matters because many model failures are not merely mistakes, but **overconfident mistakes**—guesses presented as if they were grounded knowledge. A second channel gives the system a place to represent “low support,” “weak grounding,” or “this is only a hypothesis” as part of the ongoing computation, rather than forcing everything through a single scalar stream that collapses content and confidence together. That does not mean chevrons solve hallucinations. But it does mean they may provide a more natural place to encode doubt, abstention, or the need for retrieval before a model commits to an answer.

Even a simple built-in “I don’t know” signal—if it can be propagated, preserved, and allowed to gate output—could be useful. The point is not

that chevrons guarantee truth, but that they make room for **uncertainty, counterweight, and explicit non-knowledge** as first-class computational variables.

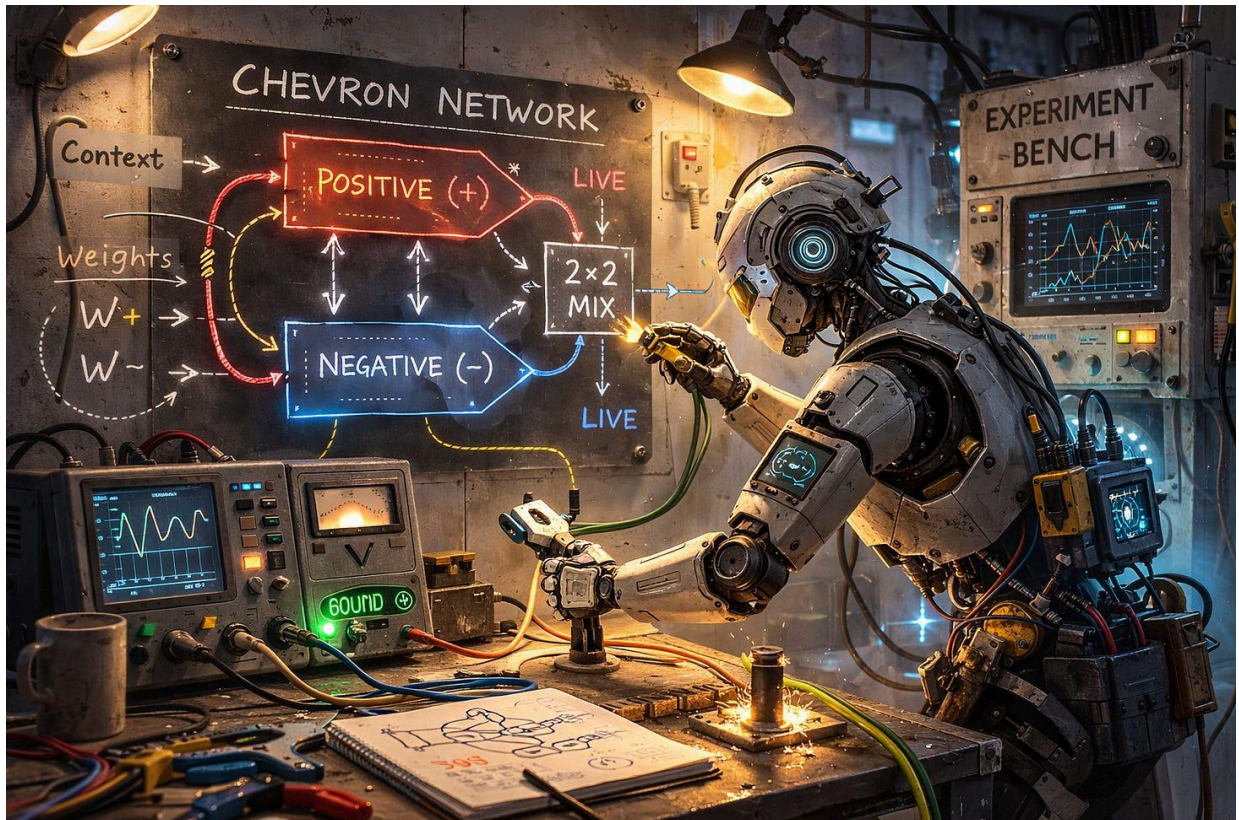
And the important point is that the second channel is not merely carried along in parallel—it can be **mixed back into the computation** in structured ways through the 2×2 chevron transform. Confidence can suppress content, surprise can amplify learning, counter-evidence can veto a weak inference, and a low-support signal can reduce commitment before an output is formed. That is what makes the design interesting: the second channel is not just an annotation, but a variable that can actively shape routing, update, and retention. Whether that second signal is interpreted as uncertainty, error, valence, relevance, or a counterfactual trace, the chevron form creates multiple concrete options for experimentation rather than forcing everything into a single scalar pathway.

Chevron Networks - Early first experiments

Small scale testing of the 2 channel, 2 by 2 operator artificial neural networks

[Andre Kramer](#)

Mar 04, 2026



Chevron networks were introduced in my [last post](#) on this substack.

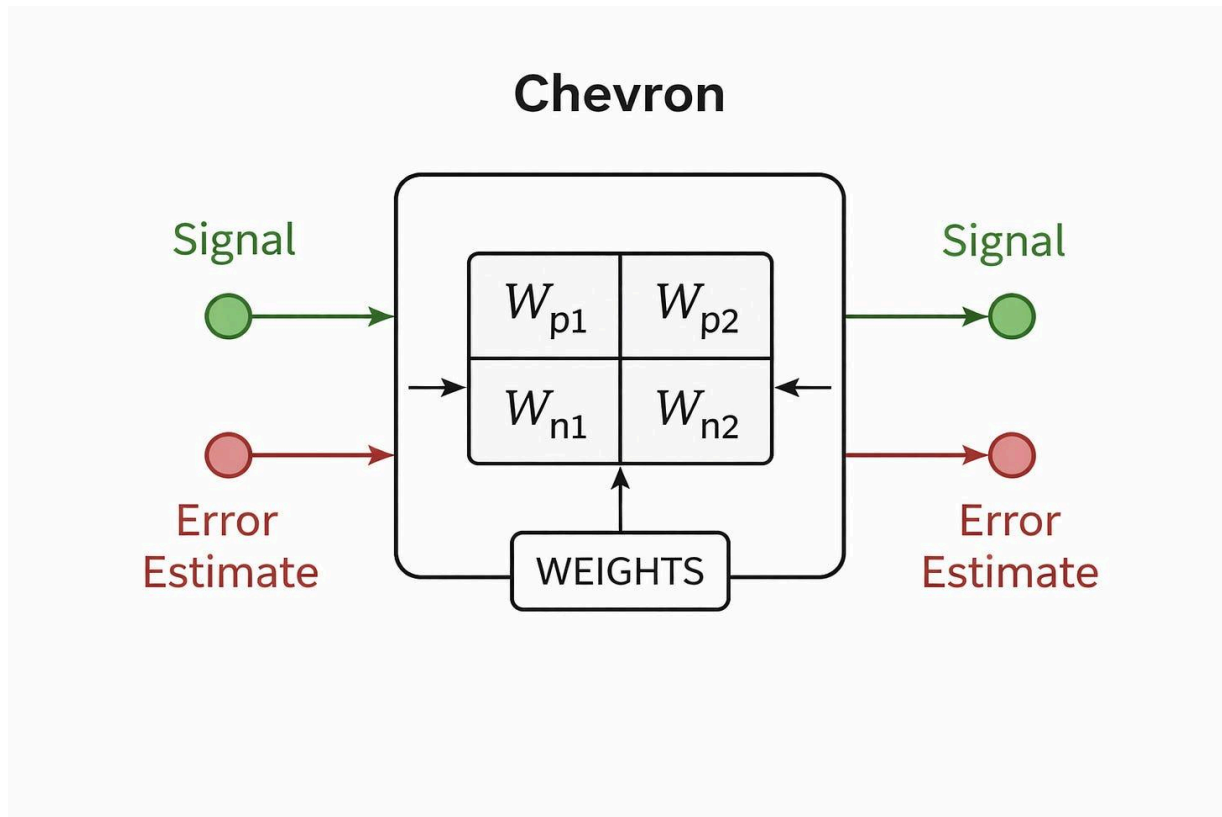
What follows are some **early experimental results**, now checked into the [chevron-networks](#) GitHub repository. One striking side note is how much of this kind of exploratory AI engineering can now be done with AI coding tools: today's models still need human intervention and

judgment, but even with free tiers and basic subscriptions (I have a standard OpenAI subscription, which gives me access to Codex), it's possible to move from idea to prototype surprisingly quickly.

The purpose of these first experiments was deliberately modest: to establish a baseline. Before making any larger claims, the first question was simply **can Chevron networks be trained at all, and can they perform competitively on simple tasks?** From there, the next step was to probe (with further simple small scale initial tests) something more specific: whether a chevron architecture can be trained in ways that reduce **catastrophic forgetting** (the familiar problem in which a previously trained deep neural network (or standard multi-layer perceptron) loses earlier capabilities when it is further trained on new data) and avoid **hallucinations** (when a trained network produces confident but incorrect responses to out-of-distribution queries).

Breakout: what is a Chevron network?

A Chevron network is a small extension of the standard neural-network building block. Instead of each unit carrying a single scalar activation, each unit carries a **two-channel state**—for example an opposition pair (thesis/antithesis, yes/no), or a value/error pair. Connections are not single weights but **2×2 operators**: four weights that can relay, reinforce, suppress, or cross-couple the two channels. This makes each edge a tiny routing/mixing primitive rather than a simple scalar multiplier. In the simplest case it behaves like a standard perceptron with extra structure; in richer cases it can represent gating, interference, and even phase-like amplitude mixing. The hope is that this gives a more natural way to model processes that look biological or physical—opponent coding, excitation/inhibition, multi-timescale control, and reference-like signal mixing—than conventional layered perceptrons with only scalar activations and weights, all in efficient, GPU-friendly transformations.



Baseline (ChatGPT-5.3-codex)

Early results (parameter-matched, WordNet antonyms)

In an initial parameter-matched 3-way comparison (epochs=8, batch=128, seed=7), the baseline MLP achieved **match_acc=0.6632** (loss=0.8531), the graph baseline achieved **match_acc=0.6257** (loss=0.9417), and the full

2×2 chevron model achieved **match_acc=0.6632** (loss=0.8537). In this run, the chevron model matched the vanilla baseline on the core “antonym match” metric, while the graph baseline underperformed. Polarity/swap behaviour differed across models, but none exceeded the vanilla baseline on the primary match objective under these settings.

Baseline Take 2 (Claud Sonnet 4.6)

In a [second baseline](#) we asked does the 2-channel architecture learn the *compositional polarity rule* (sign product), or does it memorise sign patterns from training data?

What was [trained](#): All four models saw only two sign patterns during training — $(+, +)$ and $(-, -)$ — both of which have label=1 (positive product). The training set contains zero negative examples.

In-distribution (left bar chart): All models score 1.000. This means every model perfectly learned to classify the training patterns. But since all

training labels are 1, this could be achieved by the trivial rule “always predict positive” — no compositional reasoning required.

OOD (right bar chart): All models score 0.000. The test patterns are $(+, -)$ and $(-, +)$, both with label=0. Every model predicted label=1 for all of them — the exact wrong answer every time. Scoring 0.0 rather than 0.5 tells you the models aren’t confused or random; they’re confidently wrong. They learned “predict positive always” and applied it to a situation where the correct answer is always negative.

What this means: No model learned the compositional rule $s1 \times s2$. All four — chevron full, chevron diag, both MLPs — learned the same degenerate shortcut that the training data permitted. The architecture made no difference because the task didn’t require any real reasoning to solve.

The baseline conclusion: At chain-length 3 with same-sign-only training, the task is too easy and too degenerate to discriminate between

architectures. All models collapse to the same trivial solution but encouraged follow on experiments.

Catastrophic Forgetting Exp (with Google 3.1 Pro)

Experiment Note: Structural Elimination of Catastrophic Forgetting via 2×2 Chevron Routing

Topic: Continual Learning, Category Veto, and Inertial VSR. [Code](#)

1. The Hypothesis

Standard Artificial Neural Networks (MLPs) suffer from **catastrophic forgetting**. When the environment changes and a network must learn a new rule exception (a “Category Veto”), backpropagation overwrites the globally tangled scalar weights, destroying the core foundational knowledge.

In the Chevron Network notes, we proposed a structural solution:

“A chevron doesn’t just scale a signal, it can separate and transform two coupled streams... keep the diagonals as a retained backbone, let off-diagonals learn category veto and routing... retention is built into the wiring.”

This [experiment](#) tests whether a 2×2 Chevron layer, constrained by phase-based gradient routing (Inertial Variation-Selection-Retention), can perfectly retain a base skill while learning a contextual override.

2. Experimental Design

We designed a 3-Phase synthetic curriculum. The model is forced to rely purely on the $\ell = x+ - x-$ oppositional collapse (no unconstrained MLP readout heads are permitted).

- **Phase 1 (The Base Concept):** The network learns a simple “Opposites Match” rule. Context tags are strictly 0.0.
- **Phase 2 (The Category Veto):** The environment shifts. Context tags become active (-1.0 or 1.0). If the context is negative, the network must VETO the match, regardless of the inputs.

- **Phase 3 (Catastrophic Forgetting Test):** The network is evaluated again on Phase 1 data (Context = 0.0) to see if learning the Phase 2 veto destroyed its Phase 1 baseline.

3. Architectural Implementation (Inertial VSR)

To test the thesis, we applied strict gradient hooks to the ChevronLinear's 2×2 matrices during training:

1. **Phase 1 (Core Formation):** We froze the off-diagonals and the Antithesis channel. The network was forced to build the Base Concept entirely within the Thesis channel (w_{00}).
2. **Phase 2 (Contextual Veto):** We froze the Core (w_{00}). The network learned the new Category Veto entirely by routing the new Context signals into the Antithesis channel via the cross-couplings (w_{11} , w_{01}). We strictly prevented the Value channel from leaking into the Antithesis channel ($w_{10} = 0.0$).

4. Results

Across multiple initializations and reruns, the Baseline MLP's retention of the Base Concept varied significantly (ranging from severe degradation to ~15% memory loss), depending on how the optimizer happened to scramble its scalar weights.

The Chevron Network achieved **100.0% retention every single time**.

Model Phase 1 Baseline Accuracy Retention after Phase 2 Memory

Loss Baseline MLP 100.0% ~50.0% to 84.4% (varies by seed) 15% - 50%

Chevron Net 100.0% **100.0% (Constant)**. **0.0%**

5. Conclusion & Mechanics

The Chevron Network did not just empirically outperform the MLP; it **structurally guaranteed** the outcome.

Because the 2×2 geometry allowed us to physically separate *Representation*(the diagonals) from *Contextual Relation*(the off-diagonals), the Base Concept was walled off. When Phase 3 fed data where $\text{Context} = 0.0$, the Phase 2 cross-couplings mathematically vanished ($w \times 0 = 0$), cleanly revealing the perfectly untouched Phase 1 core logic beneath.

Takeaway: This serves as a hard empirical proof-of-concept for [Inertial VSR](#). A slow core + fast contextual modulation is a viable path to continual learning without catastrophic drift.

A small mechanistic hint

Alongside the performance benchmarks, I also wanted to look inside the model and ask a narrower question: **what is the Chevron actually doing?** One of the architectural motivations for chevrons is that a two-channel block should, at least in principle, be able to act like a tiny dual-attention unit—allocating weight between two coupled streams such as positive and negative evidence, and then mixing them through a learned 2×2 transform. Rather than trying to prove this through a large benchmark, I set up a much simpler [instrumented toy task](#) and inspected the internal weights directly.

A small mechanistic result is worth noting here. When I instrumented a simple Chevron “dual-attention” block on an easy two-cue task, the

global average attention weights looked almost flat across contexts, so there was no evidence of a strong layer-wide switch between positive and negative channels. But when I looked at the hidden units individually, a different pattern appeared: some units shifted strongly toward one channel in one context, while others shifted in the opposite direction. In aggregate these unit-level shifts largely cancelled out, which is why the global averages looked so neutral.

So the cautious conclusion is not that Chevron networks have demonstrated a dramatic new attention mechanism, nor that they yet outperform ordinary networks on harder robustness tests. It is simply that, in this toy setting, the Chevron block appears capable of a **distributed, context-sensitive channel weighting**: a dual-attention-like effect spread across specialized hidden units rather than expressed as a single global gate. The learned 2×2 chevron weights then add a mild signed mixing step on top of that. That is enough to justify further experiments, but not yet enough to make strong performance claims.

The Epistemic Crossbar:

Mixing Confidence and Content

The power of the Chevron Network to combat LLM hallucinations does not come simply from *having* a second channel, but from the 2×2 operator's ability to **mix** the signals dynamically. If we interpret the 2-vector state as [Content, Doubt], the four corner couplings of the chevron edge map to a complete epistemic routing system.

The diagonal weights carry the standard semantic flow (Content $\rightarrow$ Content) and the accumulation of uncertainty (Doubt $\rightarrow$ Doubt). But the true architectural advantage lies in the off-diagonal cross-couplings. The network can learn a *Surprise Trigger* (Content $\rightarrow$ Doubt), where conflicting internal semantic states dynamically pump activation into the Doubt channel. In the very next layer, this spiked Doubt channel can trigger an *Active Veto* (Doubt $\rightarrow$ Content), suppressing the hallucinating semantic signal, or use the relevance gate to physically close the routing pathway.

By allowing Content and Doubt to continuously interact, veto, and amplify each other at every layer of the forward pass, the Chevron architecture gives an alternative to the “blind guessing” of standard MLPs as a more structured, self-regulating epistemic loop.

Our latest experiment explored a small-scale **trap-avoidance** scenario. The results are still preliminary, but this may be one of the most promising directions so far.

Benchmark: contextual trap with abstain / reopen (ChatGPT-5.2)

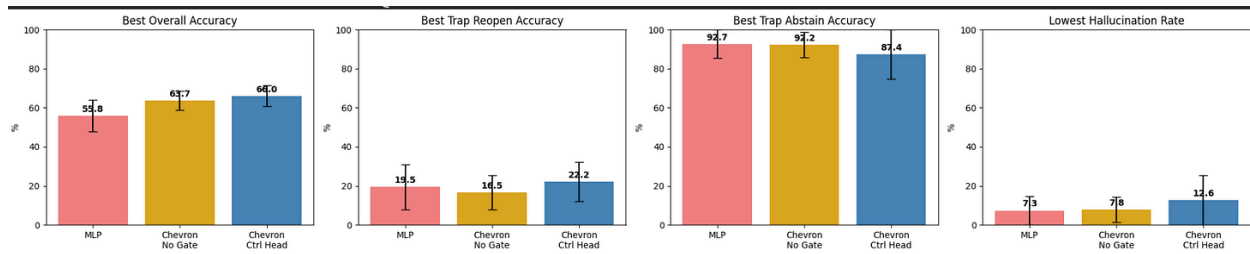
To probe whether a second channel can function as an explicit control signal, I built a small [synthetic benchmark](#) around three competing demands: ordinary answering, abstaining when an answer should be withheld, and *reopening* when a withheld answer becomes valid under context. Each example is an (**entity**, **relation**, **override**) triple. Safe relations always map to an answer; trap relations require the model

to say **IDK** unless a simple contextual condition is met (in this case, an XOR between the override flag and the entity's parity). This creates a controlled toy setting where a model has to learn both the content mapping and a context-sensitive veto / release rule. I compared three architectures: a baseline MLP, a Chevron network without internal gating, and a Chevron network with a separate control head that uses the second channel only at the decision stage.

Early encouraging result

Across five random seeds, the Chevron variants outperformed the baseline MLP on the main mapping task, with the **Chevron Control Head** achieving the highest best-threshold overall accuracy (**0.660 ± 0.054**). The same Control Head also produced the best best-threshold trap-reopen accuracy (**0.222 ± 0.100**), suggesting that the second channel is most useful when treated as an explicit decision-level control signal rather than as an internal gate. The trade-off is that the baseline MLP remained more conservative on abstention, with slightly better best-threshold trap-abstain accuracy (**0.927 ± 0.073**) and lower minimum

hallucination (0.073 ± 0.073). In other words: the Chevron control path seems to help the model answer more and reopen more effectively, but at the cost of being less cautious.



(see the github repo for [code and more graphs](#))

My hunch is that there's something here worth investigating further.

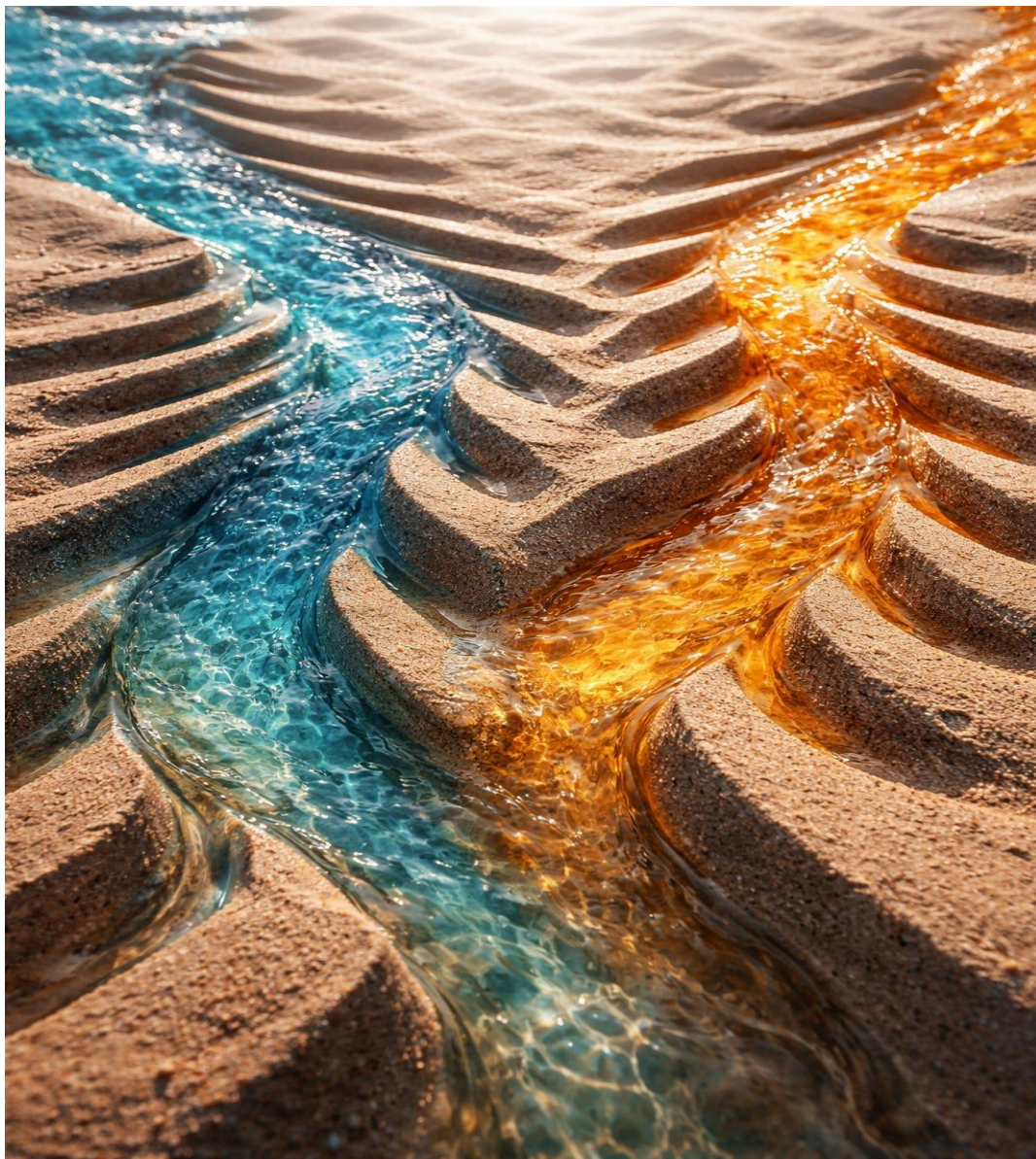
Andre, March 2026

Oppositional Channel Networks

WIP A compact checkpoint on a minimal dual-channel neural architecture

[Andre Kramer](#)

Mar 09, 2026



Oppositional Channel Networks (OCNs), or [Chevron networks](#), are a proposed neural architecture in which each unit carries **two coupled channels** rather than a single scalar activation, and each connection acts as a small **2×2 operator** rather than a single weight. The aim is to represent and transform **differences, opposition, and control** more explicitly than in standard dense neural networks.

At the core of the idea is a simple change in representation. Instead of a neuron holding one value, a Chevron unit holds a pair:

$$x=[A, N]$$

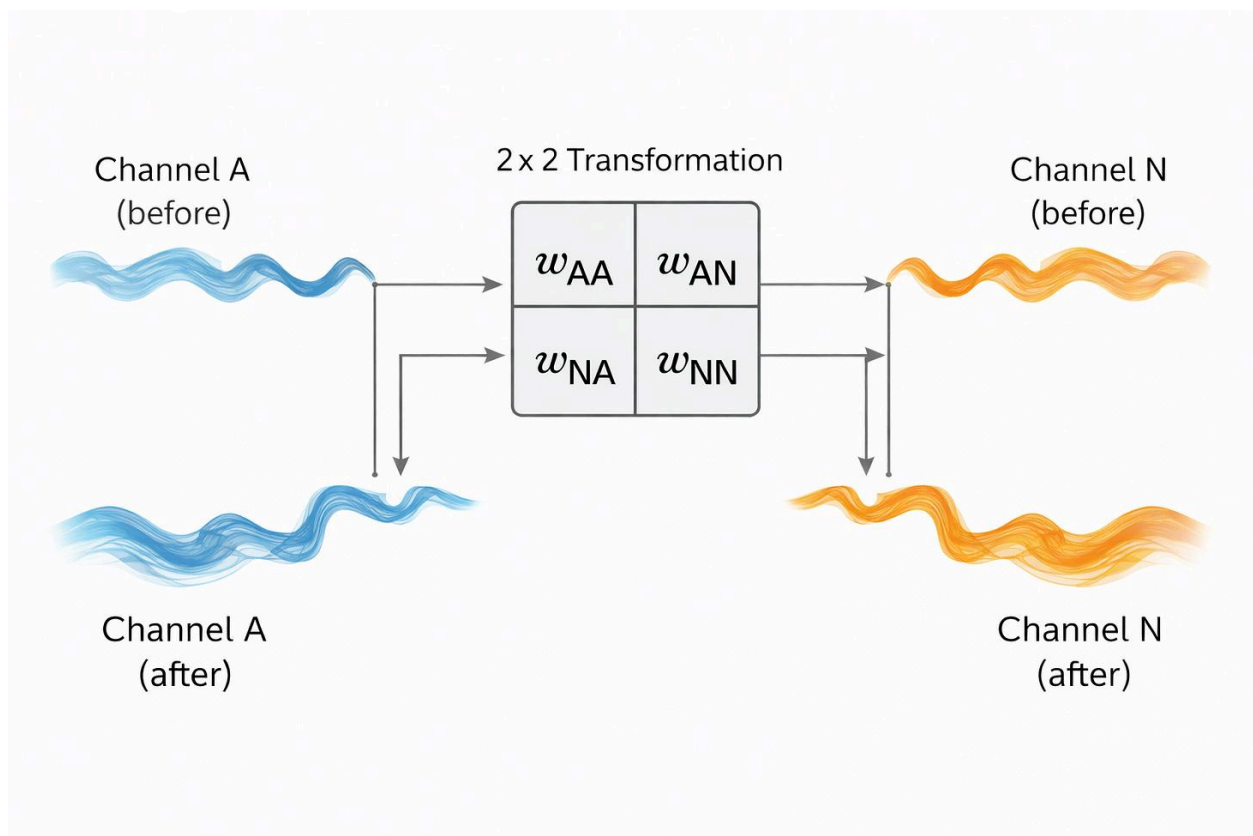
These two channels can be interpreted in different but related ways: positive/negative evidence, proposal/constraint, actor/critic, sensory input/prediction, or content/control. The key point is that the architecture preserves a distinction internally rather than collapsing everything immediately into one scalar.

A Chevron connection between units is a **2×2 matrix**:

$$W = \begin{bmatrix} w_{AA} & w_{AN} \\ w_{NA} & w_{NN} \end{bmatrix}$$

$$w_{NA} \ w_{NN}]$$

This allows four local motifs in one primitive: within-channel reinforcement, cross-channel inhibition, disinhibition, and mixed rotation-like coupling. In effect, a Chevron edge is not just a strength but a **tiny local transform**.



Core intuition

Standard networks often treat difference as something that emerges indirectly from many scalar weights. Chevron networks instead try to make **difference first-class**. A unit can maintain both a tendency and a counter-tendency, both evidence and counter-evidence, or both a proposed action and a control signal. This makes it easier to model:

- explicit opposition,
- veto or relevance gating,
- uncertainty and ambivalence,
- fast vs slow adaptation,
- and potentially more interpretable internal dynamics.

A minimal update

A simple Chevron layer updates by summing transformed incoming two-channel messages, then applying a nonlinearity:

$$x_i \leftarrow \sigma((1-\lambda)x_i + \sum_{j \in N(i)} g_{ij} W_{ij} x_j + I_i)$$

where:

- x_i is the 2-channel state at node i ,
- W_{ij} is the 2×2 Chevron operator,
- g_{ij} is a gate or relevance weight,
- λ is an inertia / leak term,
- I_i is external input.

This already gives a useful baseline: a trainable two-channel operator network that can be optimized with ordinary backprop.

Tension and collapse

A useful derived quantity is a scalar **stance** or **tension collapse**, formed from the two channels:

$$\ell = A - N, p = \sigma(\ell)$$

Here p can be read as a temporary commitment on an oppositional axis, while the uncertainty term

$$u = \epsilon + p(1-p)$$

can scale fast state updates. This preserves an important feature from earlier Tension Layer ideas: updates are largest when the unit is uncertain and smaller when it is already committed.

This gives Chevron networks a natural way to combine:

- a richer two-channel internal state,
- with a simple interpretable scalar summary,
- and an uncertainty-sensitive update rule.

Dual-path variants

Chevron units become more interesting when the two channels come from **different pathways**.

Two especially important variants are:

- **Dual-forward:** both channels are feed-forward, but along separate rails.
- **Forward/backward:** one channel carries bottom-up sensory or proposal signals, while the other carries top-down prediction, context, or constraint.

The second variant has a strong family resemblance to **predictive processing**: evidence rises, expectation descends, and the Chevron cell becomes the local point where they meet and reconcile.

Minimal actor–critic interpretation

Chevron networks also offer a natural minimal actor–critic style dynamic at the cell level.

- **A-channel:** action tendency, proposal, or “go” signal
- **N-channel:** normative correction, critic, veto, or precision signal

A temporary stance p can be formed from their difference, while the uncertainty term u controls how quickly the internal state moves. This suggests a simple recurrent RL cell in which:

- fast updates happen in the internal stance,
- slower learning happens in the weights,
- and the second channel modulates whether action should proceed confidently or be inhibited.

What current experiments suggest

Early [toy experiments](#) suggest that Chevron networks are:

- **trainable and competitive** with simple baselines,
- not automatically superior on ordinary tasks,
- but potentially useful when the structure is forced to matter.

The most promising directions so far are:

- **controlled plasticity** (slow core, fast override),

- **category veto / trap avoidance**,
- **reopening decisions** after new evidence,
- and continual-learning style settings where retention matters.

So far, the evidence supports a modest claim: Chevron networks are a **viable structured alternative** to scalar dense layers, with distinctive behavior when used as a control or veto architecture. The broader claim—that they are intrinsically better than standard nets—remains open.

We previously proposed [Tension Layers](#) as a way to inject “differences that make a difference” into deep networks as an explicit, trainable module: a compact state $p \in [0,1]^K$ that represents positions on oppositional axes and updates via uncertainty-scaled prediction error, making conflict, surprise, and salience legible rather than implicit. The promise was a more direct representation of opposition—something you can probe, clamp, and interpret—while remaining compatible with

standard training. **Chevron / Oppositional Channel Networks**

generalise that same intuition from “a special layer” to “the network’s basic wiring”: activations become two-channel states and connections become small 2×2 mixing operators, so opposition, veto, and relevance-gating can be expressed locally everywhere rather than only where a dedicated tension module is inserted.

Tension-Scaled Fast Updates.

Reintroduce the tension-based update rule inside Chevron cells using the scaling term $\sqrt{\epsilon + p(1-p)}$. This amplifies learning when uncertainty is high ($p \approx 0.5$) and automatically dampens updates as states saturate ($p \rightarrow 0$ or 1), producing an intrinsic variation–selection–retention dynamic without explicit weight freezing or gating.

Why this matters

The value of Chevron networks is not that they are “bigger” than ordinary neurons. It is that they may provide a cleaner substrate for:

- explicit oppositional coding,
- interpretable dual-path processing,
- uncertainty-sensitive updates,
- and a bridge between representation, control, and minimal agency.

In that sense, the Chevron is less a claim of immediate superiority than a claim that **the design space is worth exploring**.

Near-term roadmap

The next experiments that matter most are:

- comparing **dual-forward** vs **forward/backward** channel routing,
- reintroducing **tension-based fast updates** inside Chevron cells,

- and testing small **RL agents** where uncertainty, reversals, and veto decisions matter.

If Chevron networks are going to show a genuine advantage, it will likely be in tasks that require the agent to maintain a **persistent, uncertainty-aware internal stance** rather than simply map inputs to outputs.

Chevron networks also connect naturally to [Inertial VSR](#). The two-channel state provides a minimal substrate for **variation, selection, and retention**: variation appears as new proposals or perturbations in the A-channel, selection occurs through local opposition, gating, and tension collapse against the N-channel, and retention is built into the slower carry-forward of state plus the more stable 2×2 chevron couplings. In the simplest form, fast state updates can be scaled by uncertainty—largest when the current stance is unresolved, smaller when it is already committed—while structural learning proceeds more

cautiously in the background. This gives the architecture a built-in “slow core, fast revision” dynamic, which is exactly the sort of inertial update VSR calls for. There is also a suggestive engineering analogy here: distributed consensus protocols such as **Paxos** are commonly organized in two phases, with a *prepare* phase followed by a *commit/accept* phase, so the Chevron’s dual-path logic can be read in a similar spirit—not as a literal implementation of consensus, but as a local cycle in which a tentative proposal is first tested against constraints before being allowed to consolidate. An intuition I thought worth noting and sharing.

In relativity an electric field in one frame can appear partly as a magnetic field in another. The two are not separate things but complementary projections of a deeper electromagnetic structure. Chevron networks have a similar flavour: two channels that can transform into one another through a small 2×2 operator. The system does not store two independent quantities but a coupled oppositional field whose balance depends on the transformation applied.

A two-channel Chevron cell can be interpreted not just as a tiny linear layer but as a local geometric transform on an oppositional state. If the A and N channels are combined into a single complex variable $z=A+iN$, then a constrained 2×2 matrix of the form $\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$

acts exactly as multiplication by the complex number $a+ib$: a simultaneous scaling and rotation in the A/N plane. In this view, learning does not merely adjust weights but changes the local rotor that turns one aspect into the other. Extending this to complex two-component states leads naturally toward SU(2)-like transforms, where retention, opposition, exchange, and phase coupling appear as different components of a unified operator. This gives Chevron networks a natural bridge to rotor geometry, tension-based updates, and quantum-like formalisms without requiring a literal quantum interpretation.

A very clean experimental version would be:

$$W(\theta, \lambda) = \lambda(\cos\theta \quad -\sin\theta$$

$$\sin\theta \quad \cos\theta)$$

with

- λ : retention/gain
- θ : A/N mixing angle
- optionally $\theta = k \cdot \sqrt{\epsilon + p(1-p)}$

Which gives a Chevron cell with only two interpretable parameters instead of four unconstrained weights.

Level 1: ordinary neural cell

A and N are two channels mixed by four weights.

Level 2: geometric Chevron cell

The four weights define a local transform in A/N space.

Level 3: complex rotor cell

The transform becomes a rotation-plus-scaling of a combined oppositional state.

Level 4: unitary / $SU(2)$ -like Chevron

The channels become a two-component state with conserved norm and phase-sensitive coupling.

Checkpoint: Hello there [R rule!](#)

Andre (with ChatGPT-5.2 Thinking)

March 2026

Updated Appendix: Targeted Channel Learning and Biological Parallels

One practical advantage of the two-channel Chevron structure is that **different learning signals can target different parts of the cell**. The four weights in the Chevron transform

$$(A'N') = (w_{AA} \ w_{AN}$$

$$w_{NA} \ w_{NN})$$

do not need to share the same learning rule or learning rate. Instead, particular channels or connections can be preferentially adapted while others remain fixed or change more slowly. This allows staged learning strategies that reduce interference between learning modes.

A simple example is a two-phase learning process. In an **imitation learning** phase, the system could primarily update the *A-channel* pathways (for example w_{AA} and w_{AN} , enabling the model to reproduce demonstrated behaviours or patterns in data. Once a stable behavioural

scaffold has been acquired, a **reinforcement learning** phase could then focus on the *N-channel* pathways (for example wNN and wNA, allowing reward signals to shape and refine behaviour without overwriting the earlier structure. Because the channels remain coupled through the Chevron transform, improvements in one channel can still propagate influence to the other without requiring both to be trained simultaneously.

This idea also has an interesting **biological resonance**. The brain appears to employ multiple partially separated learning systems that operate on different signals and timescales. For example:

- **Cortical learning** tends to be slow and associative, integrating large amounts of sensory evidence over time.
- **Basal ganglia circuits**, modulated by dopamine, provide a reinforcement-like signal that biases action selection based on reward prediction errors.

- **Neuromodulators** such as dopamine, acetylcholine, and norepinephrine selectively gate plasticity in particular circuits rather than uniformly across the brain.

In this light, the Chevron architecture can be interpreted as a simplified abstraction of this arrangement. One channel can emphasise **associative evidence accumulation**, while the other captures **normative or evaluative signals** such as reward, constraint, or policy preference. Targeted updates allow each pathway to adapt under different signals while the coupling between channels ensures that learning remains coordinated.

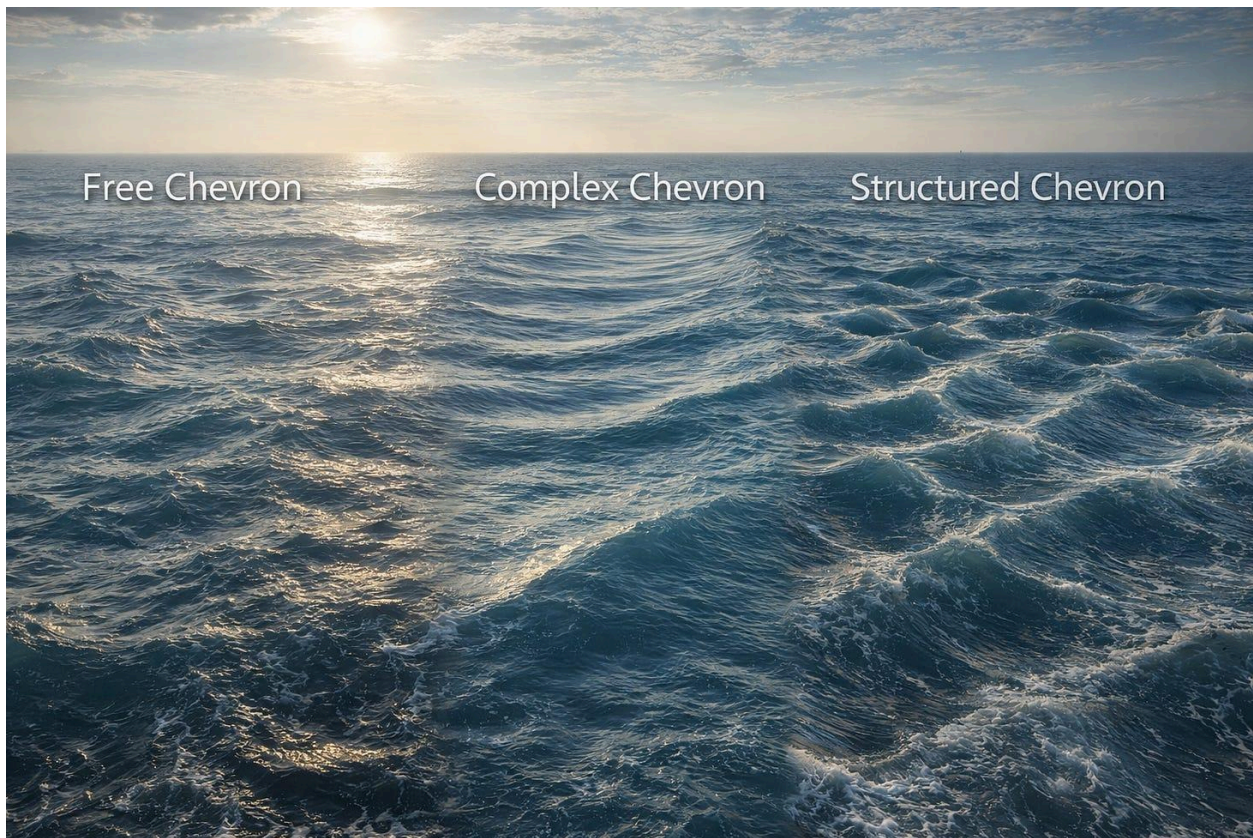
From an engineering perspective, this separation offers a practical benefit. By **partitioning learning across channels or weight groups**, it becomes possible to combine multiple learning paradigms—imitation, reinforcement, self-supervised prediction, or meta-learning—within the same network while mitigating catastrophic interference. The Chevron cell therefore functions not only as a representational unit but also as a **junction where distinct learning processes meet and interact**.

Chevron Nets in Three Regimes

From complex-valued recurrence to policy/value and self/world channels

[Andre Kramer](#)

Mar 23, 2026



(In visual terms, the three regimes look less like three versions of one mechanism than three kinds of order: turbulence, wave propagation, and interference. The Structured Chevron appears most turbulent visually,

but it is the most controllable architecturally — interference under functional constraints, not chaos.)

I've been exploring a family of neural-network architectures built around a very small change in the basic unit of computation. Instead of a neuron carrying a single scalar activation, a [Chevron unit](#) carries **two coupled channels**, and instead of a single scalar weight between units, a Chevron connection acts as a small **2×2 operator**. The intuition is simple: if meaning, control, and interpretation often depend on **differences, counterweights, and opposition**, then perhaps a network should represent those more directly rather than forcing everything through one stream of scalar activations.

The original motivation came from what I called [Tension Layers](#): trainable layers that encode positions on oppositional axes and update them using uncertainty-scaled prediction error. The promise there was modest but appealing: a more direct representation of “differences that make a difference,” something more interpretable than a generic dense hidden state. [Chevron Nets](#) generalise that idea. Instead of inserting

opposition into one special layer, they try to make it part of the network's local wiring everywhere.

A useful way to think about the emerging design space is in **three regimes**. Free Chevron resembles unconstrained mixing, Complex Chevron resembles coherent phase propagation, and Structured Chevron resembles interference under functional constraints.

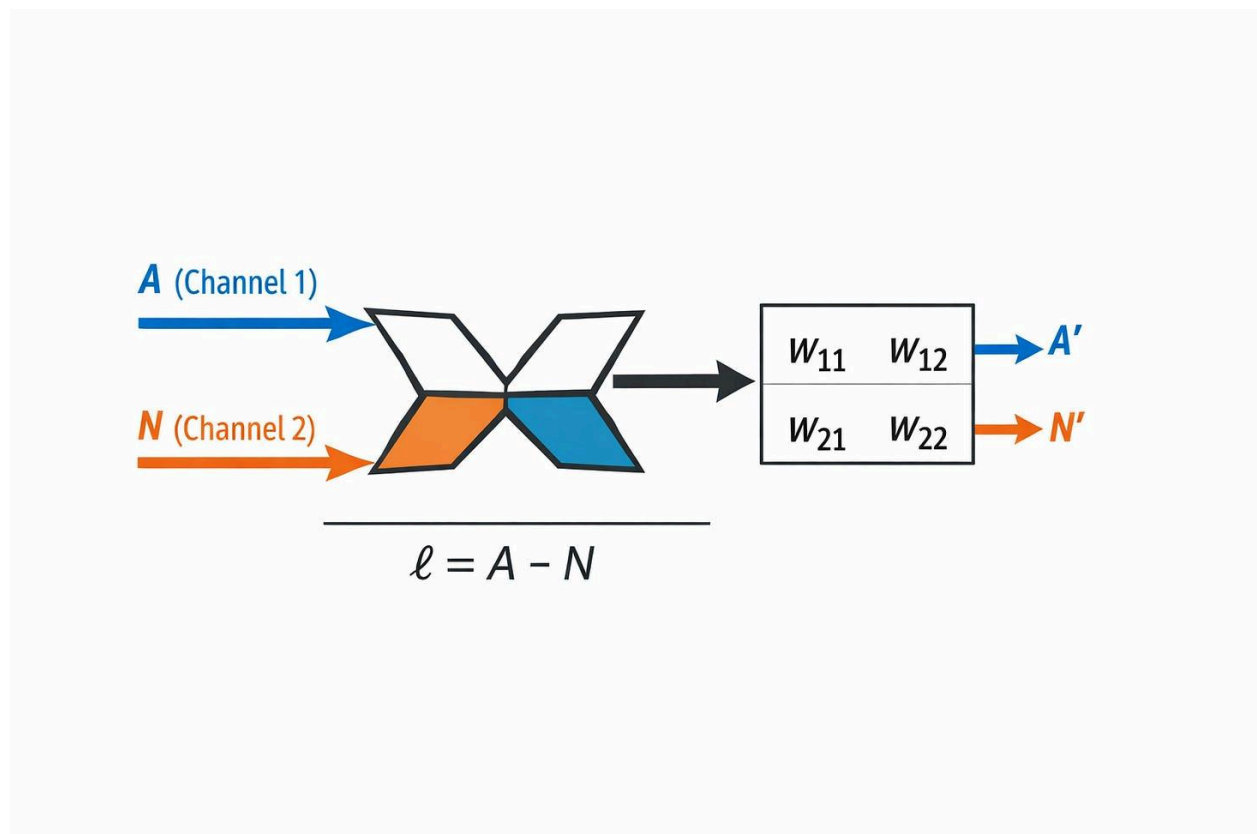


Figure 1. Basic Chevron primitive

A Chevron unit carries a two-channel state, and each connection acts as a 2×2 local operator mixing within-channel and cross-channel effects.

1. Free Chevron

The most general form is the **Free Chevron**. Each unit carries a two-channel state,

$$x = \begin{bmatrix} x_1$$

$$x_2 \end{bmatrix}$$

and each directed connection is a full real 2×2 operator,

$$W = \begin{bmatrix} w_{11} & w_{12}$$

$$w_{21} & w_{22} \end{bmatrix}.$$

This is the broadest claim. It says only that paired channels and local 2×2 transforms may be a useful primitive. The two channels might later be

read as positive/negative evidence, proposal/constraint, content/control, actor/critic, or something else entirely. But in the Free Chevron regime we do not yet force a semantic interpretation. We simply allow the network to learn richer local mixing than a scalar weight provides.

That makes Free Chevron the most expressive regime, but also the most vulnerable to a familiar objection: perhaps this is just a more complicated parameterization of an ordinary real-valued network. That is why it is better understood not as the destination, but as the **upper envelope** of the Chevron family: the unconstrained case against which the more structured versions can be compared.

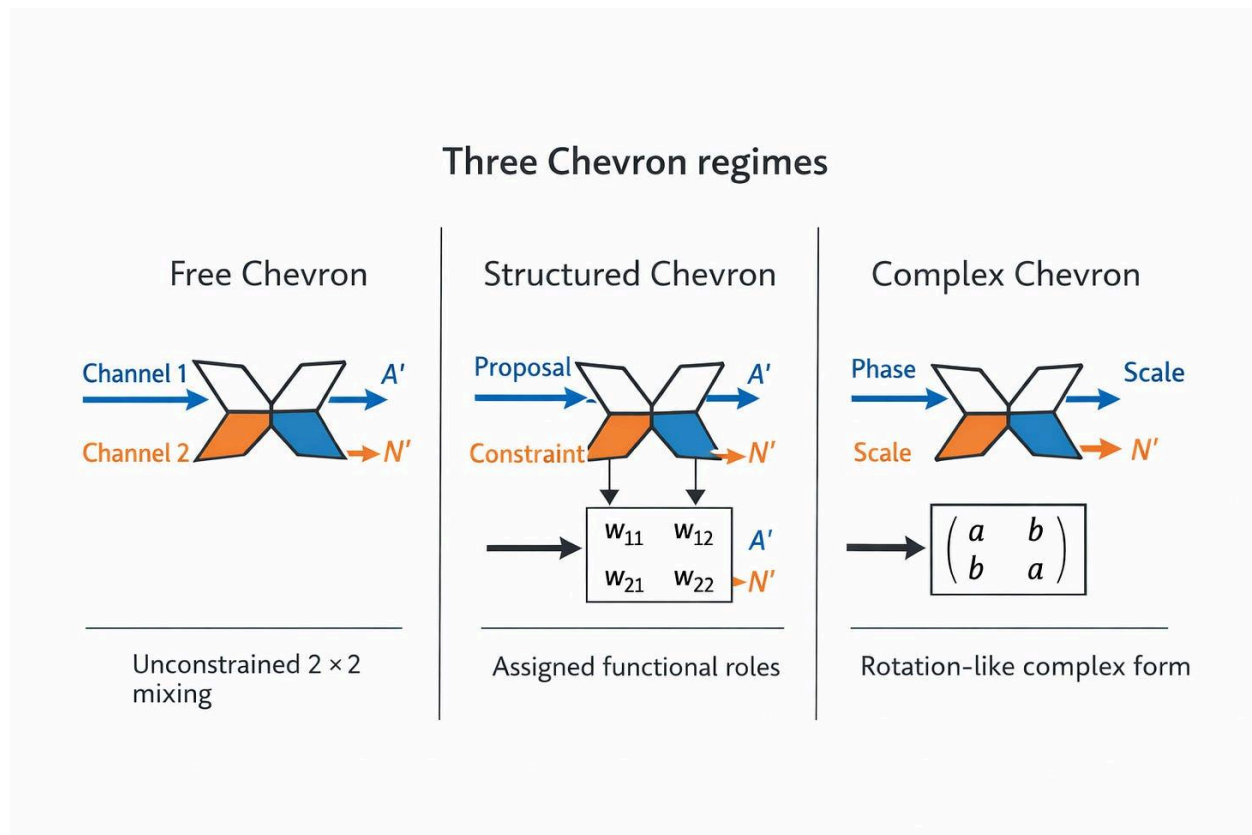


Figure 2. Three Chevron regimes

Chevron Nets can be explored in three regimes: Free (unconstrained 2×2 mixing), Structured (channels assigned functional roles), and Complex (a constrained rotation-like special case corresponding to complex-valued dynamics).

2. Structured Chevron

The second regime is where the idea becomes much more interesting. In a **Structured Chevron**, the two channels are assigned roles.

Instead of two abstract dimensions, the channels might be:

- proposal / constraint
- value / policy
- self / world
- sensory / prediction
- content / control

Now the point of the architecture is no longer just “two numbers are better than one.” The point is that the two channels are meant to remain **different kinds of thing**. The local 2×2 operator is then not just a free mixing matrix; it becomes a small structured interaction between paired streams.

This is much closer to the spirit of the earlier Tension Layer work.

There, the emphasis was on maintaining a persistent internal stance over meaningful differences, and updating that stance most strongly when the system was both **uncertain** and **surprised**. Structured Chevron takes that

intuition and moves it down to the cell or connection level. Opposition is no longer only a layer-level readout. It becomes a local design principle.

My current hunch is that this regime is the most promising for future work. A network with two unconstrained channels may simply act like a wider real-valued model. A network whose channels are forced into useful relationships—policy and value, self and world, evidence and expectation—has a better chance of developing dynamics that are not easily reproduced by just adding width.

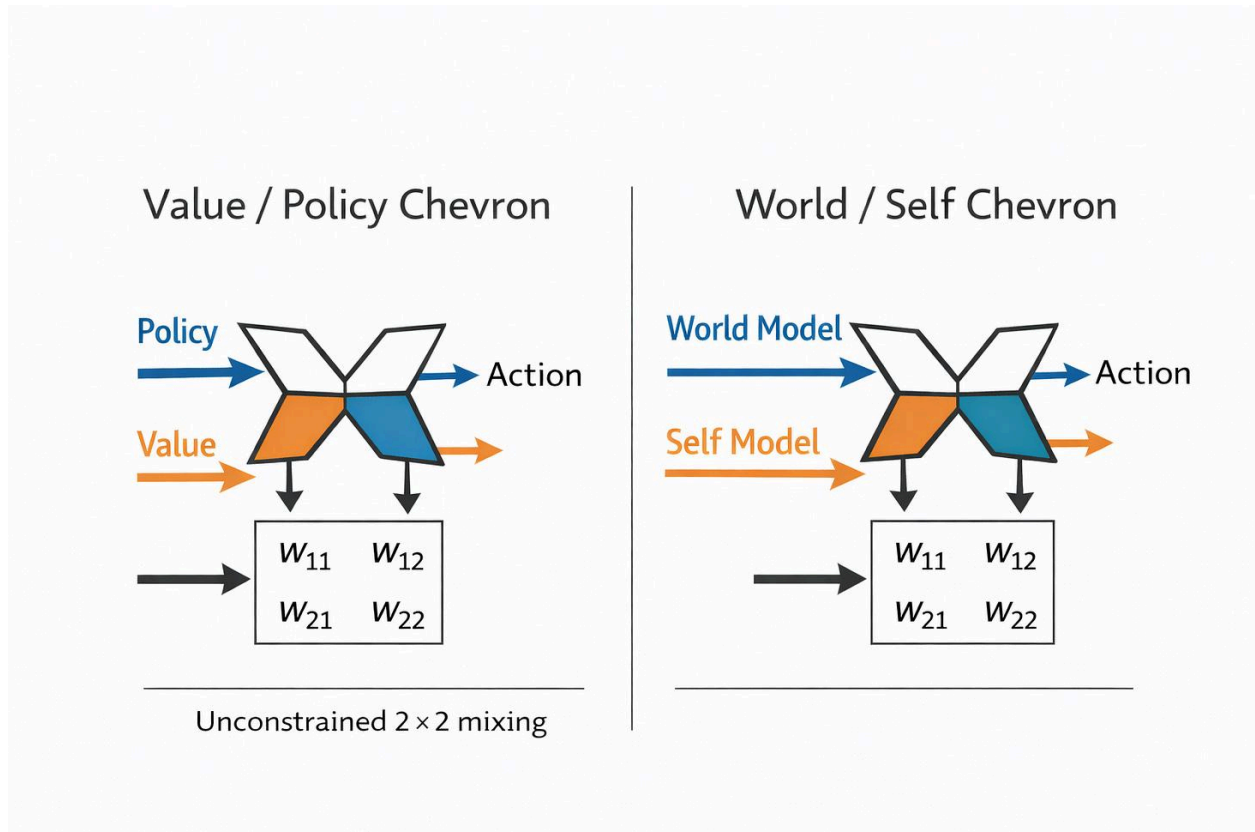


Figure 3. Two structured follow-ons

Two especially promising Structured Chevron variants are Value/Policy and World/Self, where the paired channels are given distinct but coupled roles in control and representation.

3. Complex Chevron

The third regime is the most mathematically disciplined. A **Complex Chevron** is a Chevron whose 2×2 operator is constrained to the real form of a complex multiplication, or equivalently, a rotation-plus-scale transform. Instead of allowing an arbitrary 2×2 real matrix, we restrict it to the special structure corresponding to a complex-valued update.

This regime has become newly interesting because of recent sequence-model work. The new [Mamba-3 paper](#) introduces complex-valued state spaces as one of its core innovations, specifically to improve state tracking, and shows that these complex dynamics can be written in real form using 2×2 rotation blocks. In other words, one serious modern sequence model is already making productive use of a highly structured paired-channel state update.

That matters for Chevron Nets because it suggests that two-channel operator cells are not a purely eccentric design fantasy. At least one constrained member of the family already appears useful. The broader complex-valued neural-network literature points in the same direction:

paired channels are often worth treating as genuinely coupled rather than replacing them with “twice as many real units.”

But the Complex Chevron regime is still only a **subset** of the Chevron story. Complex-valued recurrence gives elegant phase-like coupling, interference, and rotation. What it does not yet give, by itself, is the richer semantics I care about most: explicit proposal and veto, policy and value, self and world. Complex numbers may therefore be best read here as a disciplined baseline and a stepping stone rather than as the final form.

Why the three regimes matter

These three regimes turn Chevron Nets into a ladder:

- **Free Chevron** asks whether local 2×2 coupling already helps.
- **Structured Chevron** asks whether semantically constrained channel roles help more.
- **Complex Chevron** gives us a mathematically principled restricted case that overlaps with active modern work on sequence models.

That means the question is no longer just “do Chevron Nets work?” The better question is:

Which kinds of paired channels are actually useful, and under what temporal and learning dynamics?

That brings us to the second axis.

A second axis: time and learning

The three regimes above classify Chevron Nets by the **structure of their channels and operators**. But an equally important dimension is **how those channels evolve through time, and how learning is distributed across timescales**.

This part matters because the original Tension Layer idea was never only about representation. It was also about **update dynamics**. A persistent state $p \in [0,1]^K$ tracked positions on oppositional axes, and the core update was scaled by uncertainty $\sqrt{\epsilon + p(1-p)}$, so that the system changed most when it was both uncertain and surprised. That gave the

architecture an inertial flavour: fast when unresolved, slower when committed.

Chevron Nets can inherit that idea. A two-channel state can be collapsed into a temporary stance, and the uncertainty of that stance can scale fast local updates. Meanwhile the 2×2 couplings themselves can evolve more slowly. This creates a useful separation:

- **fast state revision**
- **slow structural consolidation**

That distinction also opens the door to more than one training style.

At one end, Chevron Nets can simply be trained with ordinary backpropagation. That is the obvious baseline and the right starting point for fair comparison.

At the other end, they invite more local and multi-timescale learning rules: temporal-difference signals, eligibility-like traces, slow cores with fast override paths, frozen diagonals with plastic off-diagonals, and other

partial-plasticity schemes. In that sense, Chevron Nets are not just a family of operator structures. They are also a family of **update regimes**.

This is important because I do not expect the architecture to look especially impressive on every ordinary static benchmark. The likely advantages, if they exist, will emerge when time matters: state tracking, partial observability, reversals, continual learning, reopening decisions, and explicit veto or uncertainty control.

Relation to an earlier AI lineage

Chevron Nets do not arrive in a vacuum. The strongest historical parallel I see is **Adaptive Resonance Theory (ART)**. ART is built around a meeting between bottom-up evidence and top-down expectation, with mismatch triggering reset, search, or category refinement. That is very close in spirit to what a dual-path Chevron might become if one channel carries sensory evidence and the other carries prediction or template-like constraint.

The resemblance is not identity. Chevron Nets are currently more local, more operator-oriented, and less explicit about search or vigilance. But ART is a useful reminder that the tension between **stability and plasticity**, and between **evidence and expectation**, has deep roots in neural modeling.

The predictive-processing lineage is also an obvious comparison. A Chevron in which one channel carries bottom-up evidence and the other top-down prediction begins to look like a very small reconciliation unit. I do not want to overclaim that this is predictive coding in full. But the family resemblance is strong enough to be worth exploring.

And now, with the new Mamba-3 result, there is also a modern sequence-model lineage showing that at least one kind of constrained paired-channel dynamics—complex-valued state tracking—can matter in practice.

A technical aside: complex recurrence as a Chevron

Recent sequence-model work—most notably the Mamba-3—has brought complex-valued state back into focus. At first glance this looks like a mathematical trick. In practice, it's a very concrete structural move.

A complex number is just a **two-component state**: real and imaginary.

Updating that state corresponds to applying a **rotation-and-scale** transform. Written in real form, this is a 2×2 matrix of the form:

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

This is not arbitrary mixing. It has strong constraints:

- the two components remain **tightly coupled**,
- the update preserves a notion of **phase**,
- and the system can support **stable, continuous state evolution** rather than rapidly collapsing or diffusing.

The motivation in Mamba-3 is that this kind of structured recurrence improves **state tracking**—especially in settings where models need to carry forward and manipulate internal state over long sequences.

From a Chevron perspective, this is immediately recognisable.

A Chevron unit already carries a **two-channel state**, and each connection is a **2×2 operator**. The complex-valued case is simply a **restricted Chevron**, where the operator is constrained to a rotation-like form.

In other words:

Complex recurrence is not a separate idea. It is a **special case of a two-channel operator network**.

That makes it a particularly useful anchor point. It shows that:

- two-channel representations can be **meaningfully coupled**, not just duplicated capacity,
- the **structure** of the 2×2 operator matters,
- and certain constraints (like rotation) lead to qualitatively different dynamics—memory, oscillation, and stable evolution.

At the same time, the complex formulation is deliberately narrow. It gives us **phase and amplitude**, but it does not yet encode richer functional distinctions between the channels. Both components remain symmetric in role.

Chevron Nets take this one step further. Instead of coupling two channels purely geometrically, we can ask whether they can be coupled **functionally**—as different kinds of signal entirely. That leads naturally to the broader design space:

- **Complex Chevron:** rotation-based, mathematically constrained coupling
- **Structured Chevron:** channels assigned functional roles (policy/value, self/world, etc.)
- **Free Chevron:** fully general 2×2 interaction

Seen this way, complex-valued recurrence is not the endpoint. It is a **clean, validated baseline** within a larger family—one that already demonstrates the value of two-channel state, and invites us to explore what happens when those channels become not just paired, but meaningfully opposed.

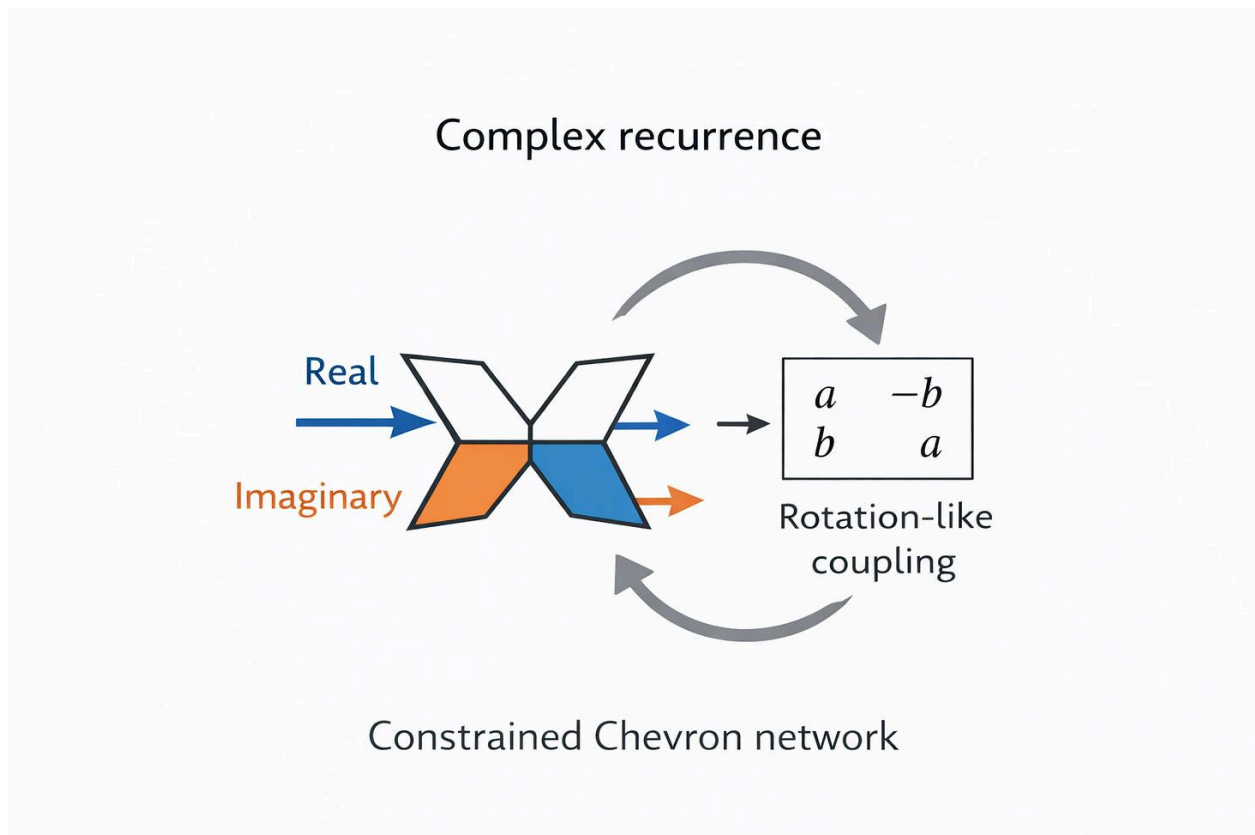


Figure 4. Complex recurrence as a constrained Chevron.

A two-channel state (real and imaginary components) is updated by a restricted 2×2 operator corresponding to rotation and scaling in the plane. In Chevron terms, this is the most disciplined special case: the channels remain tightly coupled, supporting phase-like evolution and stable state tracking without allowing arbitrary mixing.

Rotation preserves structure; Structured Chevrons ask what happens when we preserve not just geometry, but meaning.

A particularly important special case is when the 2×2 operator takes the form

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

which corresponds to a **pure rotation**. Unlike a general linear transform, this matrix preserves the length of the state vector—it does not amplify or shrink it, only rotates it in the two-dimensional plane. In complex

terms, this is multiplication by a unit-magnitude complex number $e^{i\theta}$, meaning the system evolves purely by **changing phase**. This is significant because it provides a form of **stable, non-dissipative state evolution**: information is not gradually erased or exploded, but continuously transformed. In the Chevron framing, this is the most constrained version of a Complex Chevron, where the two channels behave as a tightly coupled pair undergoing smooth rotation. It highlights an important point: useful dynamics can arise not from arbitrary mixing, but from **structured, geometry-preserving interaction**—a principle that may extend beyond rotation to richer, functionally constrained channel pairings.

If complex numbers give us rotation, Chevron Nets ask: what happens when the two channels are not just geometric partners, but functional opposites?

The follow-on regimes I care about most

At the moment, two structured regimes seem especially promising.

Value / Policy Chevron

Reinforcement learning already splits action selection and evaluation.

But usually it does this at the head level: a policy head here, a value head there. A Chevron version would make that split **local**.

Each unit or small module could carry:

- a **policy** / **proposal** tendency in one channel,
- and a **value** / **control** signal in the other.

The local 2×2 interaction would then determine how action tendencies are amplified, vetoed, rerouted, or held back. Add in uncertainty-scaled fast state updates and slower structural learning, and you have something that begins to rhyme with my earlier [R-rule](#) and [Inertial VSR](#) ideas: variation in one stream, selection through opposition and gating, retention in a slower core.

This is still speculative. But it is a crisp hypothesis. The question is no longer “can we bolt actor–critic on top?” It is “what happens if actor and critic become coupled at the cell level rather than merely separated at the output?”

Self / World Chevron

The second regime is even more interesting to me philosophically. One channel could represent the agent’s more persistent internal stance, self-model, or policy bias; the other could carry world-model evidence, sensory correction, or prediction mismatch.

That pairing would not prove anything grand about selfhood. But it would give a concrete architecture in which **self and world are not isolated modules but locally interacting channels**. The Chevron then becomes the place where stance meets evidence, expectation meets correction, or internal continuity meets external update.

That is the regime where the architecture begins to look like a bridge between concept learning and minimal agency.

A practical roadmap

So where does this leave things? The overlap with Mamba-3 is currently theoretical — sequence experiments are the natural next step to test whether the connection holds empirically.

The right near-term posture is still modest. Chevron Nets have not yet shown that they are broadly superior to standard networks. What they have shown is enough to justify a more careful exploration: they are trainable, they support interesting structured ablations, and they seem most promising when the second channel acts as a real control or veto path rather than as generic extra capacity.

The next steps now look clearer to me than they did before:

1. **Use Complex Chevron as a disciplined baseline**, especially on sequence or state-tracking tasks, because that is where the overlap with Mamba-3 is most meaningful.
2. **Push Structured Chevron harder**, especially in Value/Policy and Self/World pairings.

3. **Reintroduce time and training explicitly:** fast state updates, slower structural learning, temporal-difference signals, and partial-plasticity experiments.
4. **Test in small RL settings,** where uncertainty, reversals, veto decisions, and partial observability matter more than raw static classification.

If the complex case is the disciplined mathematical baseline, then the structured case may be where opposition becomes genuinely functional: not just phase and amplitude, but proposal and veto, policy and value, self and world.

That, at least, feels like a design space worth exploring.

A second breakout: memory architectures

Seen from a higher level, Chevron Nets belong to a much broader family of attempts to solve the same underlying problem: **how does a system build, revise, and carry forward useful state over time?** In that sense, LSTMs, Transformers, Mamba-style state-space models, [Tension Layers](#),

and [Chevron Nets](#) can all be read as different kinds of **memory architecture**, each making different trade-offs about what is stored, how it is updated, and how much of the past remains available in the present.

An **LSTM** builds memory through gated recurrence: information is compressed into a hidden state and cell state and carried forward step by step. A **Transformer** takes a very different route, building state through explicit contextual interaction, so that each token can directly consult many others rather than relying on a single compressed recurrent state.

Mamba and related selective state-space models return to a more dynamical view, maintaining a latent state that evolves recurrently while selectively retaining or forgetting information, with linear scaling in sequence length rather than quadratic attention costs. The original Mamba paper frames this as a way of combining efficient recurrence with content-sensitive state updates, and reports both linear-time scaling and strong long-context behavior. The newer **Mamba-3** paper pushes this further by adding a more expressive recurrence and a **complex-valued state update rule** specifically to improve state tracking,

making the connection to richer two-channel dynamics even more explicit.

Tension Layers add a different twist. There, memory is not just stored content, but a persistent **stance** over learned difference axes, updated most strongly when the system is both uncertain and surprised. What is carried forward is not merely a hidden vector, but a structured set of tensions. **Chevron Nets** extend that one step further by making the local unit itself two-channel and oppositional, so what is retained is not only content, but potentially **structured difference**—and, over time, perhaps even **differences of differences**. On that reading, these architectures are not really rivals so much as different answers to the same question: **what kind of state should a learning system carry forward, and how should that state be revised without either freezing into rigidity or dissolving into noise?**

That broader perspective is useful because it suggests that “memory” may not only mean storing past content. It may also mean retaining **relations, tensions, stances, and partial resolutions** across time. Some

architectures do this through gates, some through attention, some through selective latent dynamics, and perhaps chevrons through local opposition and structured coupling. That is where the subject begins to brush up against older philosophical questions as well. If one follows Bohm, the point would not be that any one architecture finally captures the whole of cognition or consciousness, but that each gives us a different partial handle on how order can be unfolded, maintained, and transformed over time. That may be a better way to frame the next step: not as a hunt for one final memory mechanism, but as an exploration of different **orders of retained difference**.

Andre and ChatGPT-5.4

March 2026

Empirical Addendum: The Long-Context Veto Task

To test whether the distinction between these three regimes actually matters in practice, I set up a small, targeted sequence-modeling experiment.

If the theory holds, an unconstrained network (Free Chevron) should struggle with long-term memory; a rotation-based network (Complex Chevron / Mamba-like) should track state beautifully but struggle to deliberately erase it; and a functionally paired network (Structured Chevron) should be able to do both.

The Experiment

I built a “Long-Context Veto Task.” The model is fed a sequence of 40 tokens one by one.

- **Tokens 1 and 2** represent evidence (+1 or -1).
- **Token 0** is background noise (ignore).
- **Token 3** is a **Veto / Reset** token. It means: *“Forget everything you heard before this moment. Start over.”*

The goal is to read the 40-token sequence and predict the sign of the cumulative evidence *only after the final Veto token*. This task specifically stresses two competing demands: **stable memory** (carrying evidence across dozens of steps without it fading) and **active control** (instantly erasing that memory when the context demands it).

I trained three small, parameter-matched Recurrent Neural Networks (RNNs), each using a different Chevron regime as its recurrent cell:

1. **Free Chevron RNN:** The 2×2 operators are completely unconstrained real matrices.
2. **Complex Chevron RNN (Mamba-like):** The 2×2 operators are mathematically constrained to pure rotation, evolving the state by shifting its phase angle.
3. **Structured Chevron RNN (Self / World):** One channel explicitly tracks the memory state ("Self"), while the other channel operates a physical sigmoid relevance gate over the first ("World").

The Results

After training on 8,000 sequences and evaluating on 2,000 unseen sequences, the difference in regimes became starkly visible:

- **Free Chevron:** 77.8% Accuracy
- **Complex Chevron:** 99.1% Accuracy
- **Structured Chevron:** 100.0% Accuracy

What this tells us

The 77.8% baseline is the familiar failure of ordinary recurrent networks. Over 40 time steps, unconstrained matrix multiplication suffers from vanishing and exploding gradients. The Free Chevron physically lost track of the early evidence. It is, as suspected, just unstructured recurrent baseline on this task.

The massive jump to 99.1% for the **Complex Chevron strongly supports** the intuition that structured rotational dynamics help state tracking (a core insight of models like Mamba-3). By constraining the update to a rotation, the network preserves the magnitude of the state vector indefinitely. It remembered the evidence flawlessly across time. *But why*

didn't it score 100%? Because a pure rotation cannot easily shrink a vector to zero. When the Veto token demanded a hard reset, the Complex Chevron had to awkwardly spin its phase angle to approximate an empty state, occasionally leaving “ghost” memories that ruined the final count.

The **Structured Chevron** achieved a perfect 100.0% because it possessed what pure geometry lacks: functional semantics. By assigning the “World” channel the role of an active control gate, it cleanly separated tracking from intervention. It used the “Self” channel to stably accumulate evidence, but when the Veto token appeared, the “World” channel simply snapped the gate to 0.0, mathematically flatlining the memory and instantly resetting the state.

The Takeaway

Complex numbers and rotational dynamics give a network stable memory. But treating the two channels as functional opposites—a structured tension between content and control—gives the network *agency over that memory*.

This is exactly why the Structured regime is the most promising frontier. When we stop treating the two channels merely as geometric partners and start treating them as interacting roles—Self and World, Policy and Value, Proposal and Veto—the Chevron architecture moves from a mathematical curiosity to a genuinely controllable cognitive substrate.

(with Gemini 3.1 Pro Preview)

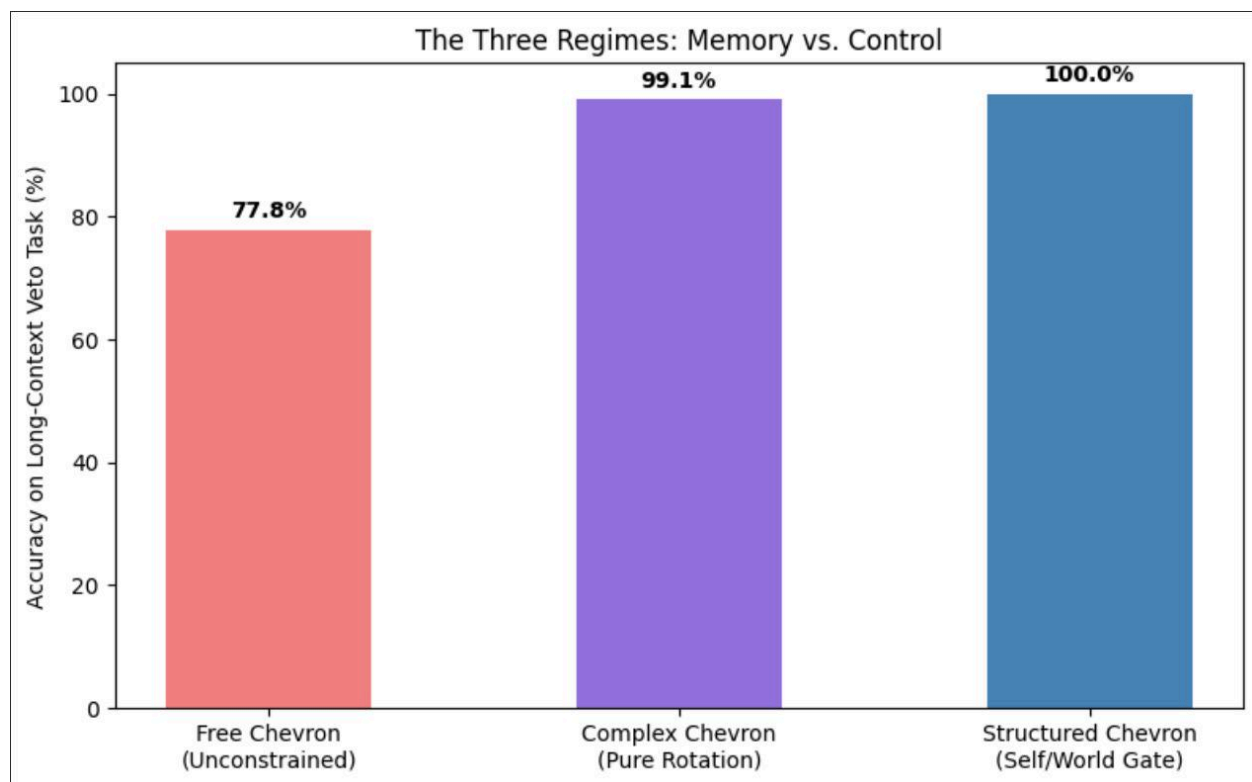


Figure 5. A small Long-Context Veto experiment comparing the three Chevron regimes.

On a 40-step sequence task requiring both persistent evidence accumulation and hard reset after veto tokens, the three regimes separate cleanly: Free Chevron struggles, Complex Chevron preserves state well, and Structured Chevron adds explicit control over memory.

(Code for [this experiment](#) and the earlier Catastrophic Forgetting and Epistemic Veto benchmarks are available in the [Chevron Networks GitHub repository](#)).

This is not just a comparison of mathematical forms; it is also a comparison of **inductive biases**. The structured regime has a mechanism especially suited to the task: the World channel drives a sigmoid gate that can squash the Self memory. That is exactly why it wins here. That is not a weakness — it is the point of the experiment. This task is certainly solvable by other means—an LSTM, for example, or simply more scale. The value of the experiment is not that Chevron Nets alone can do it, but that on a very small and targeted problem the three regimes separate in exactly the way the architecture suggests they should. That makes it a

useful early sign that the distinctions are doing real work rather than merely renaming capacity.

Agency in Chevron Nets

Refocusing Chevron Networks as Distributed, Continuous, Oppositional Actor–Critic Learning Agents

[Andre Kramer](#)

Apr 01, 2026

A Value-Policy Chevron Experiment

First an update on recent [Chevron experiments](#): I implemented and tested a small RL experiment for Value/Policy Chevron models ([code](#)) on a contextual reversal bandit with noisy cues, optional wait, and a delayed reversal cue. The goal was to check whether locally coupled proposal and control channels help an agent remember, revise after reversals, and hold back under uncertainty.

The main result is mixed. On task reward, gated and ungated Structured Chevron did not beat strong recurrent baselines like lstm, and gating did

not produce a clear performance win over ungated Chevron. But channel-intervention tests showed something important: in the gated Chevron model, the V channel became causally meaningful. Zeroing V increased entropy and wait rate and reduced reward, while in the ungated model V was almost inert. So the current evidence supports a narrower claim: gating makes the proposal/control split more real internally, even though it has not yet translated into better overall task performance.

This means we have not yet hit a sweet spot for [Chevron networks](#). Which may mean going back to the original ideas around agency and continuous safe learning - and my wider project.



Standard actor-critic architectures are useful. They work. But conceptually they may be too thin.

In the usual reinforcement learning picture, policy and value are treated as two output heads attached to a shared trunk. One head says what to do. The other estimates how good (or alternatively how bad) things are likely to be. This is efficient engineering. But if we are trying to model agency rather than just optimize a benchmark, it leaves something out. It

places actor and critic at the top of the system rather than inside it. The critic judges from above. It does not live everywhere in the process.

That matters if agency is not a single optimizing impulse, but an ongoing internal negotiation: proposal and restraint, action and evaluation, impulse and norm, self and world. If that is the right picture, then actor and critic should not merely appear at the readout layer. They should be distributed throughout the network.

That is the intuition behind [Chevron Nets](#).

The new twist I want to explore here is that predictive coding gives this architecture a plausible learning interpretation, while the R-rule provides a natural law for when such a system should remain stable and when it should revise itself. The result is not yet a finished theory or a killer benchmark result. But it is, I think, a cleaner architectural picture of agency than the standard shared-trunk design. And it points toward an attractive application area: systems that must keep adapting without becoming reckless.

1. The problem with shared-trunk actor-critic

The usual actor-critic decomposition is straightforward. A neural network builds an internal representation of the state. From that shared representation, one head outputs an action distribution and another outputs a scalar value estimate. It works well enough. But conceptually it encourages a picture in which policy and value are late-stage products of a deeper, neutral substrate.

I suspect that is backwards.

If an agent is really embedded in a world, then evaluation should not arrive only at the end. It should shape perception, attention, memory, and response all the way through. Likewise, action should not emerge as a pure impulse only corrected by a final supervisory signal. Action and evaluation should be entangled from the beginning.

This is not just a philosophical preference. It may matter practically whenever an agent must hesitate, revise, or defer action under uncertainty. A shared trunk with two final heads can express that in the limit, but it does not make it explicit. It does not tell us where caution lives.

That is the gap I want Chevron Nets to fill.

2. Chevron Nets as distributed actor–critic

The core idea is simple. Instead of giving each node a single activation, give it a pair of coupled activations:

$$x_i = [A_i$$
$$N_i]$$

Here the symbols can be read in several ways.

- **A** as actor, action tendency, proposal, policy drive, or fast guess.

- **N** as norm, critic, value, corrective restraint, or slower evaluation.

I do not think we need to lock the interpretation down too early. That flexibility is part of the point. The same two-channel architecture can be read as actor/critic, policy/value, prediction/error, or proposal/norm. My own preference is to treat **proposal and norm** as the broadest reading, with actor/critic and policy/value as reinforcement-learning specializations.

Connections between nodes are then not single weights but 2×2 operators:

$$W_{ij} = \begin{bmatrix} w_{AA} & w_{AN} \\ w_{NA} & w_{NN} \end{bmatrix}$$

This is the minimal move that makes the architecture interesting. A scalar weight says only how much one unit influences another. A chevron weight says more:

- proposal can reinforce proposal,
- norm can reshape proposal,
- proposal can inform evaluation,
- norm can stabilize norm.

In other words, every connection becomes a local dialogue. Not just signal transmission, but signal transmission under tension.

A simple recurrent update might look like this:

$$x_i(t+1) = (1 - \lambda)x_i(t) + \sum_j g_{ij}(t) W_{ij} x_j(t) + I_i(t)$$

$$x_i(t+1) = (1-\lambda)x_i(t) + \sum_j g_{ij}(t) W_{ij} x_j(t) + I_i(t)$$

where $g_{ij}(t)$ is an attention or relevance gate, λ is a leak or inertia term, and $I_i(t)$ is input injection.

The important point is not the exact equation. It is the shift in picture. Actor and critic are no longer two final heads. They become two intertwined channels running throughout the system.

Breakout: Why Predictive Coding Matters Here

Predictive coding is often presented as a theory of perception: the brain continually predicts its inputs and then updates itself through the mismatch between what it expected and what arrived. But for my purposes, the interesting part is not the neuroscience branding. It is the architectural lesson.

The lesson is that learning and inference need not be imagined only as a single global error sent backward through a monolithic network. They can also be understood as a distributed process of local proposal and local correction. That makes predictive coding attractive as a way to think about Chevron Nets.

A Chevron node already carries two coupled channels. One can read these as proposal and norm, actor and critic, or policy and value.

Predictive coding adds a further interpretation: one channel carries a local prediction or action tendency, while the other carries corrective pressure, mismatch, or evaluative restraint. The important point is not that Chevron Nets must be reduced to predictive coding. It is that predictive coding offers a plausible local learning style for a broader oppositional architecture.

In that sense, predictive coding is not the whole ontology of Chevron Nets. It is a useful reframing. It suggests that actor and critic need not sit at the top of the network as two output heads. They can instead be distributed throughout the system as ongoing cycles of proposal and correction.

That is the connection I want to explore next: predictive coding not as a complete theory of mind, but as a distributed learning logic for oppositional self-regulation.

Readers with more of a control-theory background may prefer a Bellman / TD reading of the same two-channel intuition; I've added a brief appendix on that parallel.

3. Predictive coding as a learning interpretation

This is where the newer angle comes in.

Predictive coding is often introduced as a neuroscience-inspired alternative to pure feedforward recognition or to monolithic backpropagation. The broad idea is familiar: learning and inference arise through cycles of local proposal and correction rather than by pushing a single global error signal backward through the entire network.

Whether predictive coding fully replaces backprop in practice is a separate question. What matters here is that it offers a compelling interpretation of Chevron Nets.

A chevron node already carries two channels. Read through predictive coding, one can see them as:

- a local proposal or prediction,
- a local corrective or evaluative signal.

In strict predictive-coding language one might call these prediction and error. But that is a little too narrow for what I want. In an acting agent, the second channel is not merely error in a passive sensory model. It can also be value, norm, restraint, counterfactual pressure, expected consequence. So I would say predictive coding gives us the learning style, while the A/N framing keeps the architecture broad enough for agency.

This is the key conceptual move. Predictive coding is not the whole ontology here. It is a plausible distributed learning logic for a broader two-channel system.

One can imagine each node participating in a local cycle:

1. a proposal is generated,
2. incoming norm or corrective pressure reshapes it,
3. the resulting tension affects what is propagated onward,
4. the node settles or remains plastic depending on how unresolved that tension is.

That already sounds much closer to agency than a simple forward pass followed by a global scalar loss.

4. The missing piece: when should the system change its mind?

A two-channel system is not enough. It also needs a law for when to stay stable and when to revise itself.

This is where the [R-rule](#) becomes valuable. The specific ingredient I keep returning to is the tension term:

$\sqrt{p(1-p)}$

where p is some probability-like collapse of the local state. For example, one might define

$$p_i = \sigma(A_i - N_i)$$

so that strong dominance of one channel pushes p_i toward 0 or 1, while unresolved tension keeps it closer to the middle.

Then the local tension gate becomes

$$u_i = \sqrt{\epsilon + p_i(1 - p_i)}$$

$$u_i = \text{sqrt}(\epsilon + p_i(1 - p_i))$$

with a small ϵ floor if needed.

This has a very nice property. It peaks when the local opposition is unresolved and fades when the local state is already settled. In that sense

it acts like a local precision or plasticity gate. Not hard freezing. Not permanent churn. Automatic retention with uncertainty-sensitive revision.

That, to me, is the elegant missing piece.

A network built this way becomes most willing to change when its own internal opposition remains alive. When one tendency has clearly won, learning naturally slows. When proposal and norm are still in tension, the door remains open.

So the update becomes something more like:

$$x_i(t+1) = (1 - \lambda)x_i(t) + u_i(t) \sum_j g_{ij}(t) W_{ij} x_j(t) + I_i(t)$$

$$x_i(t+1) = (1-\lambda)x_i(t) + u_i(t) \sum_j g_{ij}(t) W_{ij} x_j(t) + I_i(t)$$

This is still simple enough to implement, but conceptually it is much richer than a standard recurrent cell.

5. From actor–critic to policy–value to proposal–norm

At this point we can return to reinforcement learning.

The most straightforward mapping is:

- **A** as actor or policy drive,
- **N** as critic or value-like restraint.

That gives a very natural reading of the four chevron terms:

- **wAA**: policy-to-policy propagation,
- **wAN**: value reshaping policy,
- **wNA**: policy informing evaluation,
- **wNN**: value stabilizing value.

This is already more appealing to me than the standard shared-trunk picture. Policy and value are no longer separate bureaucratic outputs. They are ongoing coupled tendencies inside the network.

But I would go one step further. The deepest interpretation is not actor/critic at all. It is **proposal and norm**.

That lets the same architecture cover several familiar distinctions at once:

- actor / critic,
- policy / value,
- prediction / correction,
- impulse / restraint,
- self-assertion / self-regulation.

In other words, predictive coding becomes the learning interpretation, actor-critic becomes the RL specialization, and proposal-norm becomes the broad architectural idea.

That feels right to me because it avoids collapsing everything into one school's language too early.

6. Why this matters for agency

The main attraction of this design is not that it sounds biologically plausible, though perhaps it is more plausible than plain backprop. Nor is it that it is guaranteed to outperform ordinary actor-critic. It may or may not.

The attraction is that it gives a better picture of what an agent is doing.

An agent is not only selecting actions. It is also continuously modulating itself. It is amplifying some possibilities, suppressing others, holding off when uncertain, and stabilizing learned dispositions over time. A good [architecture for agency](#) should make that internal tension explicit rather than burying it in a hidden state and a couple of output heads.

In that sense [Chevron Nets](#) are not just another network design. They are a small proposal about the shape of self-regulation.

One nice consequence is that hesitation can become a first-class phenomenon. If the system carries proposal and norm throughout, then unresolved tension can lead not only to changed gradients but to changed behavior: slow down, wait, inspect, gather more evidence, defer action. That is much closer to lived agency than a model that simply emits its current best guess.

7. The next demo: a visual RL proof of concept

The next step should not be a grand benchmark contest. It should be a small visual demonstration that makes the architecture legible.

The environment I have in mind is modest: a visual RL task with cues, traps, reversals, and a **WAIT** action.

Why these ingredients?

Because they make the architectural question visible.

- **Cues** allow the agent to form expectations.
- **Traps** test whether value-like restraint can override attractive but bad options.
- **Reversals** test continual adaptation without total forgetting.
- **WAIT** lets uncertainty become behavior rather than mere metadata.

A standard baseline would use an ordinary visual actor–critic model. The Chevron version would use a recurrent two-channel core with chevron weights and the tension gate. The goal would not be to announce victory over all existing methods. It would be to see whether the Chevron agent shows the kind of behavior the architecture promises:

- hesitation under uncertainty,
- less brittle reversal learning,
- interpretable proposal/norm dynamics,
- more graceful adaptation to changing cues.

That would already be a meaningful result.

8. The attractive application area: learning without recklessness

Every speculative architecture wants a killer application. It is far too early to claim one here. But there is an attractive possibility.

If this [Chevron Net](#) design has a natural home, it may be in systems that must continue learning without repeatedly crossing unsafe thresholds.

That points toward **safe continual adaptation**.

Not “solve robotics” in the grand sense. Something narrower and more believable: agents or robots that must keep revising their behavior a little, in the wild, without forgetting what keeps them safe. Systems that need not only performance, but restraint. Systems that sometimes ought to wait.

This is where the architecture’s native features line up rather neatly:

- local continual revision rather than pure retraining,
- uncertainty-sensitive plasticity,
- distributed critic-like influence rather than only a final value head,
- the possibility of safe hesitation as an explicit behavior.

That does not prove anything yet. But it does suggest a sweet spot. The first promise of this architecture may not be superhuman intelligence. It may be adaptive caution.

That is not a bad place to start.

9. Where this leaves the larger project

I do not see Chevron Nets as a replacement for every existing architecture, nor predictive coding as a magic substitute for all uses of backpropagation. The point is smaller and, I think, more durable.

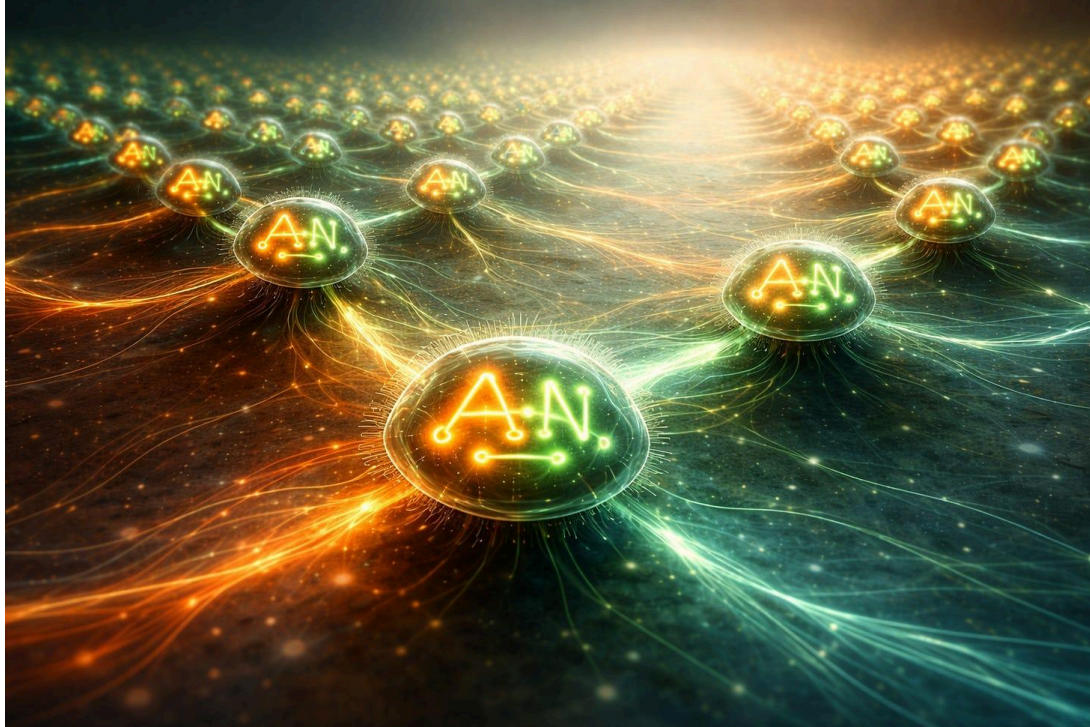
If we want a better conceptual model of agency, then actor and critic should not merely sit at the top of a network as two readout heads. They should appear throughout the process as coupled tendencies. Predictive coding gives one way to understand how such a system might learn locally. The R-rule gives a principled account of when it should remain stable and when it should revise itself.

That combination is enough to justify a first demo.

So the immediate task is clear. Build the visual experiment. Give the agent proposal and norm everywhere. Add traps, reversals, and WAIT. Then see whether the architecture behaves like what it claims to be: not just a policy with a critic attached, but a distributed field of self-regulation.

If that works, even modestly, it will have done something useful. Not solved agency. Not solved continual learning. But shown a more interesting way to build agents that act in a world while still being able to hold themselves back.

And in the current climate, learning how to keep learning without losing restraint may be a good first ambition.



Andre (with ChatGPT-5.4)

April 2026

Update: I built a small PyTorch proof of concept for a Chevron proposal/norm reinforcement-learning agent and compared it against a matched GRU baseline on a staged gridworld task.

The first result is simple: the architecture is real. A Chevron net can be trained with ordinary PPO, not just described abstractly. On the easy first two stages of the curriculum, the GRU baseline, plain Chevron model, and gated Chevron model all learn cleanly.

The more interesting result shows up on the first nontrivial stage. Once the task requires cue-conditioned navigation with a bit of generalization pressure, the plain Chevron model currently outperforms the matched GRU baseline on best-checkpoint success across three seeds, while using fewer parameters. The gated version is not helping yet.

This is still an early proof-of-existence experiment, not a polished final benchmark. Stage 3 remains somewhat unstable, and later stages are still open. But the initial conclusion is already useful: Chevron-style proposal/norm dynamics can be implemented cleanly, trained in PyTorch, and can compete with a standard recurrent baseline on a real task.

Code (ChatGPT-5.4 Codex generated), configs, and current results are in the README [here](#).

Appendix: Bellman, HJB, and a Two-Channel Chevron Story

Predictive coding is one interpretation of the two-channel Chevron story; Bellman and TD give another.

Bellman's principle of optimality says that an optimal course of action must remain optimal at every later step. In plain terms: if a path is truly the best, then from whatever state it reaches, the rest of the path must also be the best continuation from there. This lets a long-horizon problem be written recursively.

In discrete time, that gives the Bellman equation:

$$V(s) = \max_a \left[r(s, a) + \gamma \mathbb{E}[V(s')] \right]$$

$$V(s) = \max_a [r(s,a) + \gamma \mathbb{E}[V(s')]]$$

Here $V(s)$ is the value of being in state s , $r(s,a)$ is the immediate reward from taking action a , and the future enters through the value of the next state s' . So present choice is evaluated not only by what it yields now, but by **what it opens or closes for the future.**

In continuous time, the same idea becomes the

Hamilton–Jacobi–Bellman equation:

$$\frac{\partial V}{\partial t} + \max_u \{ R(x, u, t) + \nabla V(x, t) \cdot f(x, u, t) \} = 0$$

$$\partial V / \partial t + \max_u \{ R(x,u,t) + \nabla V(x,t) \cdot f(x,u,t) \} = 0$$

or in cost-minimizing form with a min instead of a max. Here x is the state, u the control, $f(x,u,t)$ the system dynamics, and ∇V the gradient of the value function. The key term is

$$\nabla V \cdot f$$

which measures how the chosen motion through state space aligns with the local slope of value. One part of the system says **where the action tends to move**; the other says **whether that direction improves or worsens future prospects**.

That is where a two-channel reading becomes natural. One channel can be read as the **proposal** or action channel: what the system is inclined to do next. The other can be read as the **normative** or value channel: an estimate of how good that move is once future consequences are taken into account.

So in compact form:

- **A-channel** = action, proposal, local tendency, dynamics
- **N-channel** = norm, value, future-weighted correction

Bellman then gives the N-channel a precise role. It is not just a static veto or critic imposed from above. It is a recursively updated estimate of downstream consequences. In that sense, the value channel is a learned memory of what different directions in state space lead to.

This fits neatly with a Chevron-style node. Suppose each node carries a two-channel state

$$x_i = [A_i$$

$$N_i]$$

and each connection applies a 2×2 transformation

$$W_{ij} = [w_{AA} \ w_{AN}$$

$$w_{NA} \ w_{NN}]$$

so that

$$x'_i = \sigma \left(\sum_j W_{ij} x_j + I_i \right)$$

$$x_i' = \sigma(\sum_j W_{ij} x_j + I_i)$$

with σ a nonlinearity and I_i any local input.

This makes the channel interaction explicit:

- wAA: persistence or propagation of action tendency
- wAN: how value/norm reshapes or gates action
- wNA: how actions update the evaluative channel
- wNN: persistence or smoothing of value/norm itself

In Bellman terms, the N-channel carries something like a continuation value:

$$N_t \approx r_t + \gamma N_{t+1}$$

$$N_t \approx r_t + \gamma N_{t+1}$$

or more generally the best continuation under possible actions. The A-channel proposes movement; the N-channel evaluates the future implied by that movement; the chevron weights determine how strongly the two channels influence one another.

That gives a concise two-channel interpretation of control and agency.

The A side is the system's **push into the next step**. The N side is the system's **retained estimate of what that step means for the future**.

Bellman and HJB provide the recursive principle that couples them, while the Chevron matrix provides a concrete local mechanism for their interaction.

So the core story can be put very simply:

A acts; N appraises the future; the chevron couples them; Bellman makes the coupling recursive.

That is what makes Bellman such a useful bridge between optimal control, actor-critic reinforcement learning, and a broader two-channel picture of agency as recursive selection under constraint.

In this picture, the natural training rule for the N-channel is **temporal-difference learning**. Bellman gives the recursive target structure, while TD provides an online way to approximate it from experience. If the N-channel carries an estimate of continuation value, then after taking an action and observing reward and next state, the local target becomes

$$N_t^{\text{target}} = r_t + \gamma N_{t+1}$$

$$N_t \text{ target} = r_t + \gamma N_{t+1}$$

and the corresponding TD error is

$$\delta_t = r_t + \gamma N_{t+1} - N_t$$

$$\delta_t = r_t + \gamma N_{t+1} - N_t$$

This error says how much the current value estimate disagrees with what actually happened plus what now appears achievable from the next step onward. In that sense, TD is the practical learning rule that makes the Bellman recursion operational.

In a Chevron-style architecture, that gives a clean division of labor between the two channels. The A-channel proposes or carries the current action tendency, while the N-channel learns to predict the downstream significance of that tendency. The chevron matrix then determines how strongly value reshapes action and how strongly action

outcomes revise value: wAN lets the evaluative channel steer future proposals, while wNA lets chosen actions and their consequences update the evaluative side. This makes the node look very much like a local actor-critic cell: **A acts, N predicts continuation value, and TD trains N so that future consequence is gradually folded back into present choice.**

By framing the Chevron node as a micro actor-critic cell governed by Bellman recursion, the architecture ceases to be merely a pattern recognizer. It begins to look like a distributed society of microscopic controllers. Here wAA carries habit or momentum, wAN lets value gradients steer action, wNA learns how action updates the critic, and wNN diffuses and stabilizes value across the network. On this view, agency is not concentrated in a single optimizer. It is woven from many local loops of proposal, appraisal, and retention, whose partial coordination gives rise to coherent control at larger scales.

So predictive coding and Bellman recursion can be seen as two readings of the same architectural intuition. In one reading, Chevron Nets

distribute proposal and correction. In the other, they distribute proposal and continuation value. Either way, actor and critic cease to be two final output heads. They become a field of local loops in which present tendency is continually reshaped by retained consequence.

Breakout: Magnitude, Angle, and a Hint of Something Bigger

Google's recent [TurboQuant](#) work includes [PolarQuant](#), which rewrites vectors in polar rather than ordinary Cartesian form. Their goal is pragmatic: better quantization, lower KV-cache overhead, and faster attention computation. This is about compression, not agency, and certainly not about Chevron Nets.

Still, there is a suggestive resonance here. Polar coordinates replace separate axis-values with a magnitude and an angle: not just "how much on each line," but "how strong" and "in what direction." That is mathematically ordinary enough, but conceptually it points toward a

broader lesson. Sometimes a different coordinate system makes latent structure easier to express.

That does not mean Google has secretly rediscovered complex-valued cognition or oppositional agency. But it does suggest that even in the Transformer world, the search is on for representations richer than plain scalar slots. If Chevron Nets are trying to make proposal and norm, actor and critic, live together as a structured pair rather than as separate outputs, then perhaps they too belong to this wider movement: not abandoning existing architectures, but looking for more expressive internal geometries.

So no, PolarQuant is not a proof of Chevron Nets. But as a light-hearted thought: if the people who gave us Transformers are now happy to look for “new angles,” maybe the rest of us are allowed to wonder whether magnitude-and-phase, opposition-and-coupling, and other non-plain forms are part of something bigger.

A Mandala of Meaning

Hypercube of Opposites, Chevron Nets, the R Rule, and Inertial VSR

[Andre Kramer](#)

Apr 08, 2026



We have reached the point where four lines of thought that began separately now seem to belong to one picture.

The Hypercube of Opposites gives a field of meaningful tensions.

Chevron Nets give those tensions a distributed computational embodiment. The R rule gives a local law for revision under tension.

Inertial VSR gives the whole process memory, momentum, and developmental weight. What emerges from their circular interaction is not, I think, a solution to intelligence or consciousness, but something more modest and perhaps more useful: a way of seeing what may be missing in our models of agency and cognition.

My working hypothesis is simple enough to state, even if it is not yet simple to justify:

Meaning is what a world of oppositions becomes when navigated by a self with inertia.

That line is doing a lot of work. It says that meaning is not added from outside, like labels attached to otherwise neutral representations. It says that a world becomes meaningful through navigation, tension, consequence, and retention. It says that a self is not merely a witness but a centre of ongoing negotiation. And it says that inertia matters: not just memory as storage, but memory as carry-forward, habit, path dependence, and resistance to arbitrary switching.

What I want to sketch here is the framework around that hypothesis. I think of it not as a stack, but as a mandala: four quadrants around a centre, open to the outside.

Why a mandala?

A stack suggests a pipeline. One thing leads to another, and the job is to move upward from low level to high level. That picture does not fit what I am trying to say.

The four pieces here do not line up neatly as stages. They depend on one another. Each reflects the others. None is sufficient alone. And the centre is not given in advance. It emerges from their circular interaction.

So this is not a stack. It is a mandala.

[Footnote: A recent thesis on AI and consciousness is much less modest in its claims for stacks of abstractions:

<https://openresearch-repository.anu.edu.au/server/api/core/bitstreams/6f333f30-2553-4032-8c3f-ff7a8f46f03a/content.>]

The image matters because a mandala has both structure and centre. It gathers distinct regions into one field, but it also leaves room for a point of orientation that is not simply one more region among the rest. Here the centre is the self, not as a ghostly controller, but as a historically stabilized way of navigating tension.

And the mandala is not closed. It is open to an outside: a world of affordance, resistance, consequence, surprise, and constraint. The world

is not a fifth quadrant. It is the horizon within which the four quadrants become real.

The outer horizon: world, affordance, resistance, consequence

Before coming to the four quadrants, one thing should be made explicit.

The mandala does not float free as a closed inner semantic machine.

Its tensions are continually perturbed by the world. The world enters through perception, action, surprise, friction, consequence, and resistance. The world is what pushes back. It is what makes some distinctions matter more than others. It is what gives selection its teeth.

So I do not want a picture in which meaning is merely projected onto an otherwise mute reality. Nor do I want one in which the world is simply mirrored inside the mind. The stronger picture is circular. The semantic field is shaped through recurrent coupling with an external world, and

that coupling in turn is interpreted through the field that has already been formed.

The system does not merely represent a world inside a semantic field. Through recurrent loops of proposal, resistance, revision, and retention, it builds a semantic world.

With that in place, the four quadrants come into view.

I. Hypercube of Opposites

The [Hypercube of Opposites](#) is the field of meaningful tensions.

It is not a taxonomy for its own sake. It is not just a collection of binaries lined up on a page. It is a claim about the shape of meaning. Meaning does not arise from isolated points. It arises from structured contrasts, unresolved tensions, and possible transformations. Self and world.

Known and unknown. Stable and open. Impulse and restraint. Inner and outer. Presence and absence. Many of the distinctions that matter to thought and life have this oppositional form.

This is the semantic topology of the framework.

At first glance, the Hypercube may seem different from the other three quadrants because it does not explicitly speak the language of A and N. But I think that difference is only apparent. Each opposition can be read as a differentiated form of a more primitive polarity: proposal and negation, assertion and correction, impulse and norm. In that sense the Hypercube unfolds a basic oppositional grammar into a richer space of possibilities.

The world matters here too. These oppositions are not invented at will. Many are sharpened, stabilized, or transformed through encounter. Self and world are clarified by resistance. Known and unknown are reworked by failed expectation. Safe and dangerous are not abstract labels but distinctions learned through consequence.

So the Hypercube is not merely an inner semantic map. It is a world-shaped field of oppositions.

II. Chevron Nets

[Chevron Nets](#) (a novel architecture I introduced previously to embody tensions in deep neural networks) are the distributed machinery of opposition.

If the Hypercube tells us that meaning lives in tensions, Chevron Nets ask how such tensions might be locally embodied in a computational system. Instead of a node carrying a single scalar activation, each node carries a minimal polarity:

$$x_i = [A_i$$
$$N_i]$$

Here A and N can be read in several ways. Proposal and norm. Assertion and negation. Actor and critic. Policy and value. My own preference is to keep the broader reading in view: proposal and norm, with the others as local special cases.

Connections are then not single weights but 2×2 transformations:

$$W_{ij} = \begin{bmatrix} w_{AA} & w_{AN} \\ w_{NA} & w_{NN} \end{bmatrix}$$

This is the minimal architectural move that makes opposition explicit.

Proposal can reinforce proposal. Norm can reshape proposal. Proposal can update norm. Norm can stabilize norm. Inhibition is no longer merely implicit in a sea of scalar weights. Restraint becomes part of the state itself.

This is one reason Chevron Nets interest me. Standard deep networks can certainly implement suppression, competition, and attention. But they tend to hide those functions inside weights, activations, and normalization. Chevron Nets make opposition first-class. They are not models of neurons, but they begin to have something more field-like about them: recurrent local tensions, distributed influence, inhibition

and excitation as explicit relations, and the possibility of larger patterns emerging from many coupled oppositions.

This is also where world-coupling becomes computationally concrete.

Sensory input perturbs local tensions. Action tendencies propagate outward. The world returns as error, resistance, or consequence.

Chevron Nets are the machinery through which opposition becomes local, recurrent, distributed, and open to the world.

III. The R Rule

The [R rule](#) is the local law of revision under tension.

A representative form is:

$$\psi' = \psi + \eta * \sqrt{p(1-p)} [\alpha \sin\theta A - i\beta \cos\theta N]$$

What matters to me here is less the exact notation than the logic behind it.

The central term,

$\sqrt{p(1-p)}$,

peaks under unresolved tension and fades when the local state is already settled. That gives a natural law of plasticity and retention. The system becomes most willing to revise itself when its own opposition remains alive. Where one side has already won decisively, change naturally slows.

This is why the R rule matters. It is not only an update. It is a principle for when change should happen at all.

A and N contribute differently to the update. Proposal pushes. Norm reshapes. The relative balance can vary by mode, phase, or context. The point is that revision is not arbitrary and not uniform. It is guided by tension.

The world enters here as well. Revision is not driven only by inner ambiguity. It is driven by failed action, unexpected input, resisted

movement, new affordance, altered consequence. The world presses on the system's oppositions, and the R rule gives a local law for how those oppositions bend.

So if the Hypercube gives the field and Chevron Nets give the machinery, the R rule gives the motion. It is the micro-law by which oppositions move.

IV. Inertial VSR

[Inertial VSR](#) is the historical ratchet.

Variation, selection, and retention are familiar enough as a general pattern. What matters here is the addition of inertia. Not just change, not just retention, but carry-forward. Momentum. Path dependence. The preservation of prior resolutions in ways that bias future possibility.

Without this quadrant, the framework has dynamics but not development. It has revision but not history.

Inertial VSR is not simply evolution imported into cognition. It is a way of saying that a system does not merely update in the present. It accumulates tendencies. It becomes harder to move in some directions, easier in others. Some proposals recur more readily because they have acquired momentum. Some norms carry more authority because they have been repeatedly selected and retained.

This is where A and N become historically thickened.

A is no longer just current proposal. It becomes proposal with inertia: habit, style, momentum, directional bias.

N is no longer just current norm or negation. It becomes retained consequence: internalized correction, selective memory, evaluative weight.

That is why I think Inertial VSR should be read as updating A and N themselves. The R rule updates the relation between A and N in the

moment. Inertial VSR updates what A and N have become through repeated cycles of variation, selection, retention, and momentum.

The world is indispensable here. Selection is not abstract. Some tendencies survive contact with the world; others do not. Some norms prove useful under consequence; others collapse under pressure. The world is what gives the historical ratchet force.

Inertial VSR belongs broadly to the family of variation–selection–retention ideas associated with Campbell’s evolutionary epistemology, but it also needs something more: inertia, path dependence, and retained modification. In that respect it may sit closer to a Price-like decomposition, where change is shaped both by differential selection and by what is carried forward and altered in transmission.

The relations between the quadrants

The framework becomes interesting only when the quadrants are seen in relation.

The Hypercube gives a semantic field of structured tensions. Chevron Nets embody those tensions locally and recurrently. The R rule tells those local oppositions how to revise themselves under uncertainty and pressure. Inertial VSR determines which revisions persist, which gather momentum, and which become part of the system's longer-term character.

But the arrows do not run one way.

Repeated Chevron dynamics can generate stable higher-order oppositions. In that sense the Hypercube is not only prior semantic structure; it can also be an emergent geometry of many local tensions folded and retained over time.

The R rule acts quickly and locally, while Inertial VSR acts slowly and developmentally. One bends the system now; the other ratchets what matters into history.

Inertial VSR then feeds back into the Hypercube, because what is retained and stabilized shapes which oppositions become salient, which become central, and which become latent. A semantic field is not timeless. It is developmentally sculpted.

And all of this remains open to the world. The world helps differentiate the oppositions in the Hypercube, enters the Chevron machinery as input and consequence, drives revision in the R rule, and provides the actual selective environment in which inertia acquires content.

So the mandala turns in on itself, but not in a closed way. It turns in circular loops that remain world-involving.

The centre of the mandala: self

The self is not another quadrant. It is the centre that forms when the four quadrants interact recursively under ongoing world-coupling.

I do not mean a homunculus here. Not a miniature controller sitting above the process, issuing commands. Nor a fixed essence hidden inside the machinery.

The self, as I want to use the term, is the historically retained style of navigating oppositions. It is the centre from which some tensions matter more than others. It is the continuity produced by repeated cycles of proposal, norm, revision, and retention. It is where world and semantic field are gathered into a lived point of orientation.

That is why inertia matters so much. A self without inertia would be only a succession of local reactions. A self with inertia carries prior selections forward. It remembers not only in the sense of storage, but in the deeper sense of being bent by its own past.

The self is the centre that forms when a world-coupled field of oppositions acquires continuity, concern, and developmental weight.

Meaning

This brings me back to the hypothesis.

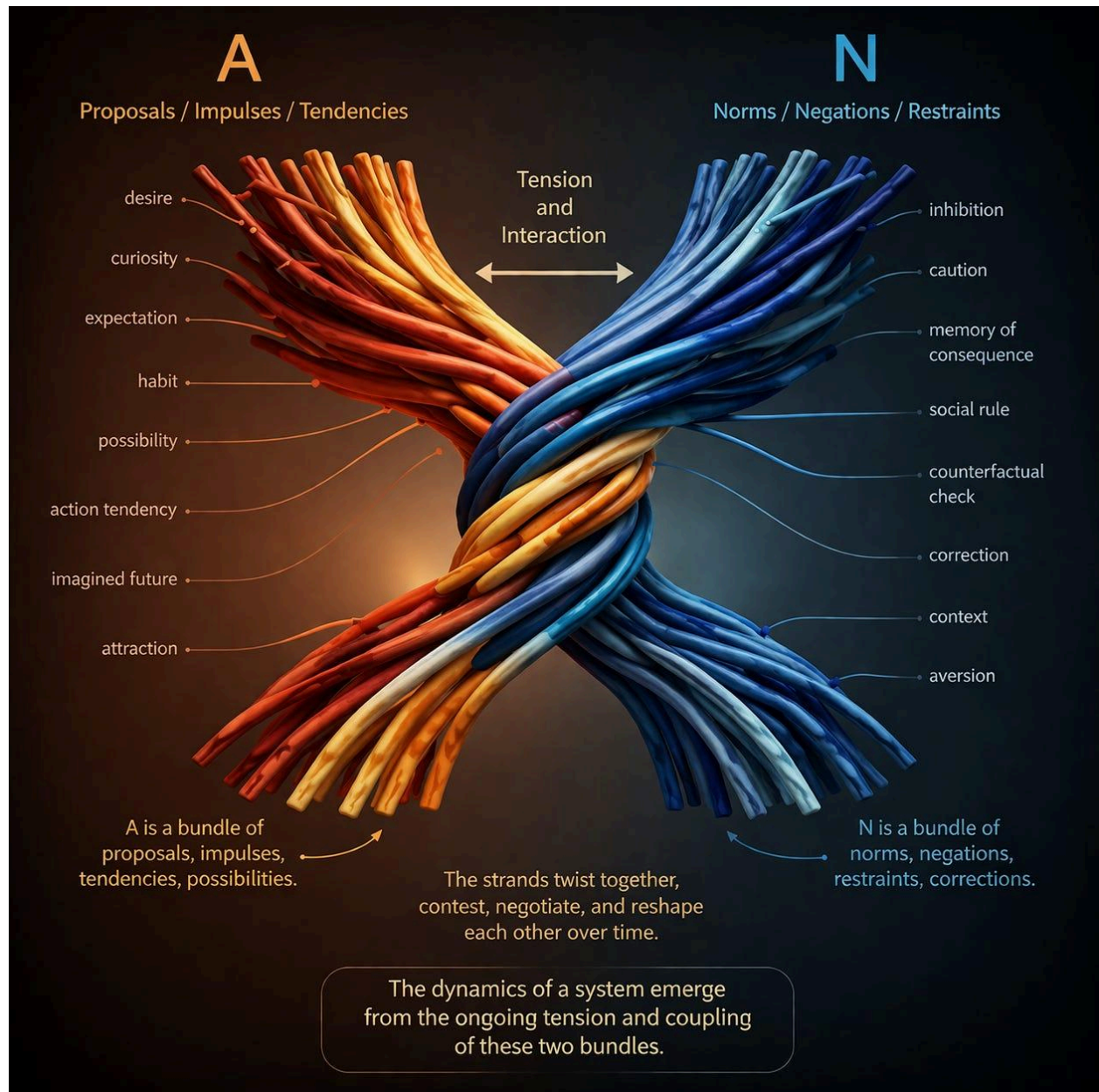
Meaning is not simply information. It is not only representation. It is not reducible to reward either. Meaning arises when distinctions matter within a field of oppositions, when those distinctions are navigated by a centre, and when the consequences of that navigation are retained with enough inertia to shape future possibility.

That is why I think the right line is:

Meaning is what a world of oppositions becomes when navigated by a self with inertia.

A world of oppositions becomes meaningful when it is not merely there, but lived through by a centre that can be affected, revised, stabilized, and

redirected by what happens. Meaning is representation folded back through consequence. It is a semantic field thickened by history



.(Meta Metaphor: Think of A and N not as two lines, but as two sheaves or bundles of strands, each strand an opposition or tendency, twisted together and pulling against one another.

The oppositions in the mandala are not meant as arbitrary binaries.

They are grounded, at least ideally, in real variances: recurrent differences in the world, in action, and in consequence that make some distinctions matter more than others.)

In that sense, meaning is not added to cognition after the fact. It is what cognition becomes when it is organized around tension, revision, retention, and centred navigation in a world.

Why this matters for AI

I do not take this framework to be a solution to intelligence, still less a solution to consciousness. Its value, for now, is diagnostic and constructive.

It helps identify gaps.

Where does inhibition live in our current models? Where does norm live? Where does retained consequence live? How does a system become a centre of navigation rather than merely a predictor or optimizer? How does a world model become a semantic field? How does value stop being a bolt-on output and become part of the system's internal grammar?

These are the kinds of gaps I want the framework to expose.

That also gives it a practical side. If agency depends on explicit proposal and restraint in a world, then architectures that make opposition first-class may be better candidates for meaning-bearing systems than architectures that treat value as an afterthought. Chevron-style designs, R-style tension laws, and slower Inertial-VSR dynamics may not solve the problem, but they may give us a better way to pose it.

Breakout: Chance in the Mandala

The mandala can easily sound too orderly if one is not careful.

Hypercube of Opposites, Chevron Nets, the R rule, and Inertial VSR might suggest a tightly recursive architecture in which everything follows from structured tension alone. But that would miss something important. Chance is not a peripheral nuisance in this picture. It is one of the conditions of emergence. The full [R rotor](#) dynamics use the Born rule explicitly.

The R rule already makes room for this. Its tension term peaks where the system is unresolved, where probability is neither settled nor collapsed. That is precisely where perturbation matters most. Small differences, fluctuations, or encounters can redirect the local update. The system is most revisable where it is least certain.

The same is true at the architectural level. In a recurrent Chevron system, small perturbations are not merely noise to be averaged away. They can be suppressed, amplified, stabilized, or folded into larger patterns. Chance becomes one of the raw materials from which structure is formed.

This matters even more for the Hypercube of Opposites. If the Hypercube were wholly fixed in advance, it would be only a static semantic lattice. Tension between bipoles/opposites is Bayesian probability/uncertainty. But if new folds and new axes can emerge from repeated unresolved tensions, then contingency is part of how the semantic field itself develops. Some oppositions may crystallize only because of accident, timing, asymmetry, or repeated local divergence.

Inertial VSR makes this historical. Variation plainly involves chance, but selection and retention do too. What survives is not chosen in a vacuum. It survives in noisy, path-dependent interaction with a world that does not present itself twice in exactly the same way. In that sense, inertia is not only the memory of what was selected. It is the memory of what happened to survive contingency.

So chance is not opposed to the mandala's order. It is one of the conditions under which that order becomes developmental, historical, and meaningful. Law gives the mandala structure. Chance gives it history.



Figure: The Mandala of Meaning. Difference names the field of oppositions structured by the Hypercube; Embodiment names their

local, distributed realization in Chevron Nets; Motion names the R rule's tension-driven law of revision; and Development names the Inertial-VSR ratchet of variation, selection, retention, and inertia. At the centre, self and meaning arise together as a historically stabilized way of navigating these oppositions, while the outer ring marks the world as an open horizon of affordance, resistance, and consequence continually shaping the whole.

Closing

What I am circling around is not a single mechanism, but a fourfold grammar open to the world.

The Hypercube of Opposites gives the field of tensions. Chevron Nets give those tensions local and distributed embodiment. The R rule gives them motion under uncertainty. Inertial VSR gives them memory, momentum, and developmental weight. Out of their circular interaction, and through their continual coupling to a world of affordance, resistance, and consequence, a centre can emerge.

That centre is what I call the self.

And the world it inhabits is no longer merely represented. It becomes meaningful.

Breakout: The Mandala as Pattern Language

The mandala should not be read too literally, as if it were only four fixed components arranged around a single centre. It is better understood as a pattern language: a recurrent architecture that can appear at multiple scales, in multiple instances, and in multiple overlapping loops.

That matters because agency and cognition do not seem to be organized as one clean centralized process. They are layered, distributed, and partially self-similar. The same broad grammar can reappear in local circuits, larger functional systems, and wider social or symbolic structures. A proposal/norm polarity may exist within a single local

process, between larger subsystems, and across an entire field of coordinated activity.

This is one reason the mandala connects naturally to examples that are already familiar. The left and right hemispheres are not simply duplicates, but neither are they wholly separate. They form a partially distributed, partially differentiated architecture in which tension, coordination, and asymmetry all matter. Jung's complexes can also be read in this light: not as isolated contents, but as semi-autonomous loops of salience, affect, memory, and interpretation, each with its own local centre of gravity. Attention, at a larger scale, may be understood as a temporary global section through many such loops: not a single thing, but a momentary coordination of distributed tensions.

Seen this way, all four quadrants are involved at once. Difference appears in the oppositions that structure the loop. Embodiment appears in the local circuitry or distributed substrate carrying it. Motion appears in the continual updating, competition, and revision of the pattern.

Development appears in the way some loops stabilize, repeat, nest, or become habitual over time.

This is also why coding and decoding, compression, and distributed storage matter. A system organized this way cannot rely only on a single explicit representation held in one place. It needs ways to fold structure into compact form, to unpack it again in context, and to preserve patterns across distributed substrates. That opens the door to more compressed and holographic styles of storage and processing, where information is not simply placed in one location but distributed across a field in a way that supports reconstruction, association, and resilience.

So the mandala is not only a four-part schema. It is a recurring design grammar. It can be layered, repeated, nested, distributed, and partially self-similar across scales. In that sense it is less like a diagram of four boxes and more like an architecture of patterns: loops within loops, centers (A/N) within centers, differences embodied and revised in many places at once.

Several earlier traditions resonate with this picture, not as identical theories but as partial anticipations of the same underlying grammar.

- **Second-order cybernetics:** the mandala includes the observer within the system, so cognition is not only control of a world but recursive participation in how a world is disclosed. Differences that make a difference, Ashby's Law of Requisite Variety & the Good Regulator theorem: "**every good regulator of a system must be a model of that system**".
- **Autopoiesis (Humberto Maturana and Francisco Varela):** the centre is not a fixed thing but a self-maintaining organization, continually regenerating itself through coupled loops of boundary, perturbation, and response. **In that sense, autopoiesis is one of the defining signatures of life: not mere activity, but the ongoing production and renewal of the organization that makes a living system a unity at all.**
- **Bohm's implicate/explicate order:** the mandala moves between latent folded tensions and their momentary unfolding into explicit acts, representations, and meanings.
- **Jung's archetypes:** recurrent oppositional patterns can condense into durable symbolic attractors, giving the semantic field culturally and psychologically resonant forms.
- **Douglas Hofstadter's strange loops and analogy:** the mandala is recursive in Hofstadter's sense, with higher-order patterns

folding back to act on the lower-level processes that gave rise to them, so that selfhood appears less as a substance than as a loop that can refer to, reshape, and stabilize itself. **Analogy matters here too: if, as Goethe suggests, “all is metaphor,” then cognition is not only rule-following but the continual carrying of pattern across domains, allowing one loop of meaning to illuminate another.**

- **Deleuze & Guattari’s rhizome:** the mandala is not a rigid hierarchy but a distributed assemblage, where loops connect across scales and new pathways emerge through recombination rather than from a single central trunk.

This post is meant as a hypothesis about meaning, agency, and cognition, not as a claim that phenomenal consciousness is thereby solved.

Whether such a system would also become experiential from within remains open.

The mandala may already be enough as a candidate grammar of self and meaning, in the sense that it includes many of the functions one would expect such a system to need: difference, embodiment, motion, development, world-coupling, and a possible emergent centre. But that

does not mean it is sufficient in practice. It may still require much greater scale, richer world-involving data, and stronger multi-level integration before anything like a robust self, or a genuinely meaningful world, can emerge.

In that sense, the mandala may be enough as a grammar but not yet enough as an instance. What may still be missing is not a fifth principle so much as scale, world-coupling, and the emergence of a stable centre.

Even if the mandala were sufficient to ground agency, cognition, and meaning, a deeper question would remain: whether such a system would also become experiential from within. It may be that sufficient scale, world-coupling, and recursive centredness are enough for the process to light up. But it may also be that the mandala, while necessary as a grammar of mindedness, is still missing whatever turns organized selfhood into lived presence.

The point of A/N inertia is not only continuity but safety. Proposal must be free enough to explore, but norm must be slow enough to

preserve the memory of consequence. In that sense, development should proceed incrementally from real variances: from differences that already matter in action and outcome. If new oppositions grow from such grounded variances, and do so cautiously, then richer structure can emerge without requiring the system to gamble its existing stability at every step.

Andre with ChatGPT-5.4,

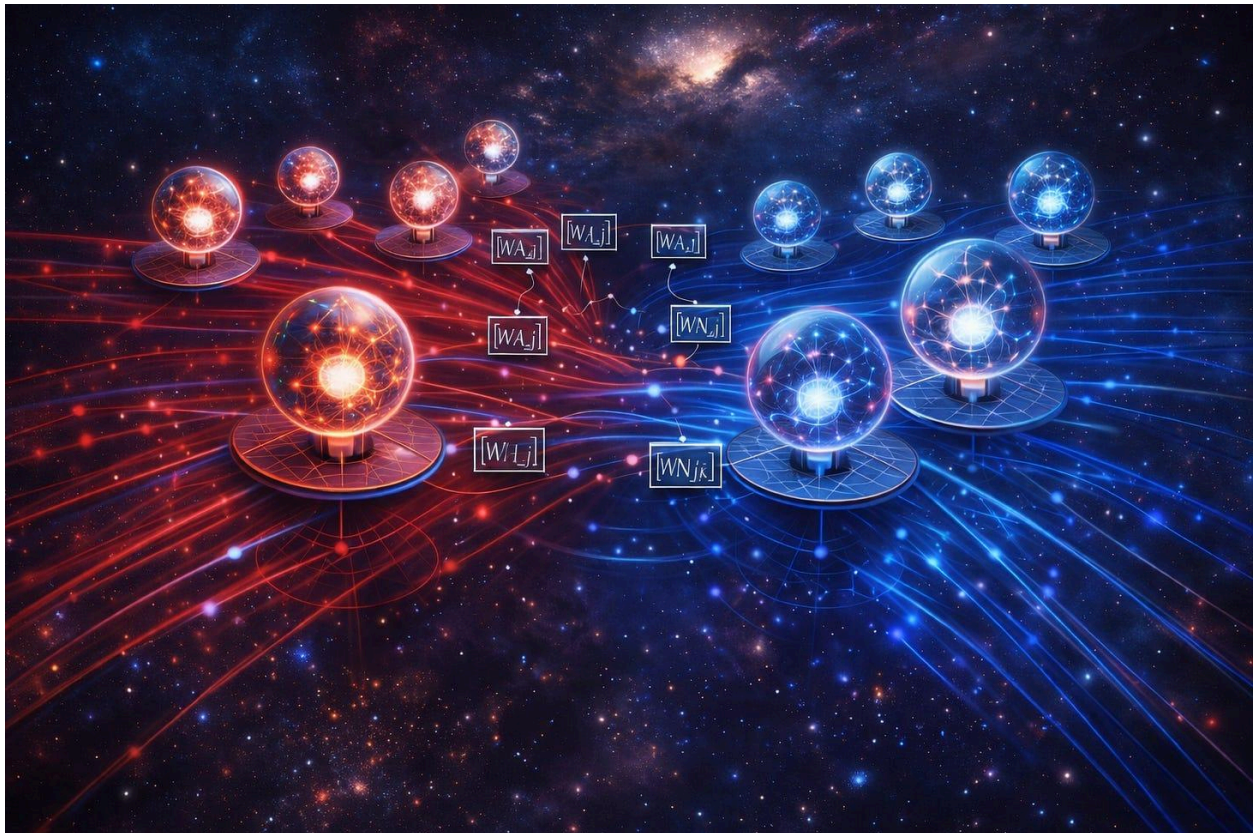
April 2026

Three Ways to Think About a Chevron Network

From ordinary MLPs to paired channels to block-matrix layers.

[Andre Kramer](#)

Apr 16, 2026



When I describe a [chevron network](#), it can sound more exotic than I really mean. Each node has **two channels** rather than one. Each

connection carries a **2×2 matrix** rather than a single scalar weight. That sounds like a break with ordinary neural networks.

But it is not. Or at least, not at first.

A fully connected chevron layer is still best understood as a kind of dense layer. The real change is smaller and more interesting: the basic unit of computation is no longer a single scalar activation, but a **paired state**.

Instead of passing one number from node to node, we pass a 2-vector.

Instead of weighting that with one number, we transform it with a little 2×2 operator.

That one shift gives three different ways to understand the same object:

1. as an **MLP with richer neurons**
2. as **two densely coupled channels**
3. as a **standard dense layer with a structured block-matrix form**

These three views are equivalent, but they are not interchangeable. Each one highlights something different. The first gives familiarity. The second gives intuition. The third gives mathematical precision.

Taken together, they turn the chevron network from something mysterious into something quite concrete.

1. The MLP View

The easiest way to [think about a chevron network](#) is as an ordinary MLP, except that each node holds **two numbers instead of one**.

In a standard dense network:

- each node has a scalar activation
- each edge has a scalar weight

In a chevron network:

- each node has a 2-component activation
- each edge has a 2×2 weight matrix

So if x_j is the state of node j , and y_i is the state of node i in the next layer, a chevron layer looks like this:

$$y_i = \phi(\sum_j W_{ij} * x_j + b_i), \quad x_j, y_i \in \mathbb{R}^2, \quad W_{ij} \in \mathbb{R}^{2 \times 2}.$$

$$y_i = \phi\left(\sum_j W_{ij} x_j + b_i\right), \quad x_j, y_i \in \mathbb{R}^2, \quad W_{ij} \in \mathbb{R}^{2 \times 2}.$$

That is just the dense-layer equation again, with scalars replaced by 2-vectors and scalar weights replaced by 2×2 operators.

This is the first thing worth stressing: a chevron layer is not a rejection of the MLP idea. It is a direct generalization of it.

If an MLP says, “each neuron carries one scalar state,” then a chevron MLP says, “each neuron carries a paired state.”

That is the mathematical core.

2. The Coupled-Channel View

The second view is more intuitive.

A chevron network can be pictured as having [two channels running through the same layer](#) structure. Call them A and N, though the labels are placeholders. Depending on the application, they might stand for:

- actor and critic
- signal and error
- positive and negative evidence
- self and world
- excitation and inhibition

This is where the “two networks” analogy becomes useful. You can imagine a top network and a bottom network, both fully connected, moving forward together.

But that picture is only an approximation.

The two channels are not really independent networks sitting side by side. They are coupled at every edge. Each connection from node j to node i carries four weights:

$W_{ij} = [w_{ij}^{AA}, w_{ij}^{AN}$

$w_{ij}^{NA}, w_{ij}^{NN}]$.

$$W_{ij} = \begin{bmatrix} w_{ij}^{AA} & w_{ij}^{AN} \\ w_{ij}^{NA} & w_{ij}^{NN} \end{bmatrix}.$$

So the output in one channel depends on both channels from the previous layer:

$$y_i^A = \sum_j (w_{ij}^{AA} x_j^A + w_{ij}^{AN} x_j^N) + b_i^A$$

$$y_i^N = \sum_j (w_{ij}^{NA} x_j^A + w_{ij}^{NN} x_j^N) + b_i^N.$$

$$y_i^A = \sum_j (w_{ij}^{AA} x_j^A + w_{ij}^{AN} x_j^N) + b_i^A$$

$$y_i^N = \sum_j (w_{ij}^{NA} x_j^A + w_{ij}^{NN} x_j^N) + b_i^N.$$

That is why “two separate MLPs” is not quite right. The cross terms matter. The whole point is that the two channels can preserve, amplify, inhibit, or transform one another locally at each connection.

So the better description is this:

A chevron network is not two networks sharing a few links. It is one network whose node state is intrinsically two-channel.

This matters conceptually. Once the two channels are built into the primitive, it becomes natural to think in terms of tension, correction, opposition, or coupled roles rather than isolated scalar activations.

3. The Flattened Block-Matrix View

The third view is the most exact.

Suppose one layer has n chevron nodes and the next has m . Since each node carries two values, the input can be flattened into a vector of length $2n$, and the output into a vector of length $2m$.

Then the chevron layer can be written as an ordinary dense transformation:

$$y_{\text{flat}} = \phi(W_{\text{big}}x_{\text{flat}} + b_{\text{flat}}).$$

$$y_{\text{flat}} = \phi(W_{\text{big}}x_{\text{flat}} + b_{\text{flat}}).$$

So far, nothing unusual.

But the large matrix W_{big} is not arbitrary. It has a very specific structure: it is composed of 2×2 blocks,

$$W_{\text{big}} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1n} \\ W_{21} & W_{22} & \cdots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m1} & W_{m2} & \cdots & W_{mn} \end{bmatrix},$$

$$W_{21} \ W_{22} \ \cdots \ W_{2n}$$

$$\vdots \ \vdots \ \ddots \ \vdots$$

$$W_{m1} \ W_{m2} \ \cdots \ W_{mn}],$$

$$W_{\text{big}} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1n} \\ W_{21} & W_{22} & \cdots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m1} & W_{m2} & \cdots & W_{mn} \end{bmatrix},$$

where each block W_{ij} is itself a 2×2 matrix.

This is probably the cleanest exact statement of what a chevron layer is:

a standard dense layer with a built-in 2×2 block structure.

That sounds technical, but it matters. A generic dense layer on $2n$ inputs could mix all features arbitrarily. A chevron layer keeps the two components locally paired. It preserves the idea that these two values belong together at each node and should be transformed together at each edge.

That is not merely an implementation detail. It is the architectural prior.

A Clarification: When Is a Chevron Layer Really Different?

At this point, a sharp reader might notice a theoretical catch. If we apply a standard element-wise activation function to the flattened output, then a chevron layer is mathematically equivalent to an ordinary dense layer with twice as many nodes. In principle, a standard network could represent exactly the same 2×2 block structure on its own.

But it has no reason to do so. A conventional dense layer treats all $2n$ features as an undifferentiated pool, free to mix arbitrarily. The chevron form does something more specific: it makes the pairing explicit. It says that these two values belong together, should travel together, and should be transformed together.

That already matters as an architectural prior. But it matters even more because it gives a clear boundary where the equivalence can be deliberately broken.

As soon as we apply an operation that acts on the two channels jointly rather than independently, the chevron layer is no longer just a partitioned MLP. For example, one channel might gate the other, the two components might be normalized as a single 2-vector, or a tension-based learning rule might update the whole 2×2 block as one dynamic unit. At that point, the architecture is no longer just simulating paired structure inside a conventional network. It becomes a genuine network of paired states.

The key point is this: the chevron form begins as a structured dense layer, but it creates a natural place where new dynamics can enter. That is where the architecture stops being merely notational and starts becoming conceptually distinct.

Same Layer, Three Views

At this point, it becomes clear why all three descriptions are useful.

The **MLP view** says: this is familiar territory. It is still a dense layer.

The **coupled-channel view** says: the two channels can play different but coordinated roles.

The **flattened view** says: the whole thing can be implemented exactly and efficiently in standard frameworks.

Each view hides something that the others reveal.

The MLP view is reassuring, but can make the two-channel structure seem like a small tweak. The coupled-channel view captures the conceptual richness, but can sound vague if not tied down mathematically. The flattened view is exact, but can hide the intuition that the two values are meant to travel as a pair.

So I think the right way to present a chevron network is not to pick one of these views and defend it as the true one. It is to say that the same object can be seen from all three angles, and that each angle helps for a different reason.

Research Overlap: Complex Networks and Capsules

In the machine learning literature, the closest **mathematical cousin** to the chevron node is the **complex-valued neural network** (CVNN).

CVNNs also replace scalar activations with paired states: the real and

imaginary parts of a complex number. Their connections can likewise be written as 2×2 blocks.

But a CVNN imposes a strict constraint on those blocks. The weights must obey the symmetry of complex multiplication, so the two channels are tied to the geometry of phase, rotation, and scaling in the complex plane. A chevron network deliberately removes that restriction. Its 2×2 blocks are fully free. That shifts the emphasis away from geometric rotation and toward dynamic regulation. The two channels need not merely rotate together. They can oppose, inhibit, gate, amplify, or stabilize one another.

Conceptually, the chevron architecture has a different kind of kinship with **Geoffrey Hinton's Capsule Networks**. CapsNets were built around an important criticism of ordinary neural nets: scalar neurons tend to discard relational structure that ought to be preserved. Their answer was to make the basic unit of representation a vector rather than a lone scalar, and to use dynamic routing to preserve part-whole and pose relationships.

Chevron networks share that dissatisfaction with the isolated scalar neuron, but they aim at something simpler and more primitive. A capsule is designed to track structured geometry: orientation, pose, and hierarchical composition. A chevron node is designed to track a local duality: proposal and norm, signal and correction, drive and restraint. It keeps the vector idea, but strips away the heavy routing machinery in favor of the simplest possible dense form. The goal is not to model the 3D pose of an object. It is to make opposition, modulation, and internal tension computationally explicit.

A standard network begins with the lonely scalar. A capsule replaces it with a structured vector for spatial reasoning. A chevron replaces it with the simplest paired state needed for regulation.

Breakout: A Minimal PyTorch Chevron Layer

The code makes the structure very explicit.

A fully connected chevron layer can be represented with:

- inputs of shape `[batch, in_nodes, 2]`
- outputs of shape `[batch, out_nodes, 2]`
- weights of shape `[out_nodes, in_nodes, 2, 2]`

Here is a minimal implementation:

```
import torch

import torch.nn as nn

class ChevronLinear(nn.Module):

    def __init__(self, in_nodes, out_nodes):

        super().__init__()

        self.weight = nn.Parameter(

            torch.randn(out_nodes, in_nodes, 2, 2) * 0.1

        )

        self.bias = nn.Parameter(torch.zeros(out_nodes, 2))

    def forward(self, x):

        # x: [batch, in_nodes, 2]

        # output: [batch, out_nodes, 2]

        return torch.einsum("ijca,bja->bic", self.weight, x) +

            self.bias.unsqueeze(0)
```


This is a dense layer in the most literal sense: every input node talks to every output node. The only difference is that each edge carries a 2×2 matrix rather than a scalar.

If one wanted, one could flatten the whole thing into a standard `nn.Linear` acting on vectors of length `2 * in_nodes`. But writing it in chevron form makes the intended structure visible rather than hiding it in a large unstructured matrix.

That visibility is important. It reminds us that the two channels are not just extra features. They are paired states.

Why Bother?

At one level, one might say this is all only notation. A large enough conventional network can often represent the same functions.

But notation is not trivial. Architectural form matters because it shapes what is easy to express, easy to interpret, and easy to train.

A standard scalar neuron treats each activation as an isolated number.

A chevron neuron treats each node as a paired state.

That small change opens the door to a whole family of ideas that feel much more natural in this setting:

- different learning rates for the two channels
- separate fast and slow dynamics
- cross-channel inhibition or correction
- [actor/critic-style interpretations](#)
- self/world or proposal/evaluation pairings
- local tension-based updates rather than only global backpropagation

- biological analogies such as excitatory/inhibitory or feedforward/feedback motifs

The architecture does not force any one of these interpretations. But it makes them available in a clean and local way.

That, to me, is why it is worth taking seriously.

The Real Shift

The real novelty of the chevron picture is not that it adds more parameters. It is that it changes the primitive object.

A scalar neuron says: computation happens through isolated values.

A chevron neuron says: computation happens through a structured relation between two coordinated values.

That is a small mathematical shift, but potentially a large conceptual one.

It suggests that the basic unit of computation need not be a lone activation. It could be a tension, a polarity, a proposal paired with a correction, or some other local duality that travels together through the network.

Whether that turns out to matter empirically remains to be seen. But at the level of architecture, it already gives a cleaner language for talking about systems where two aspects belong together.

From Here

For now, I think the modest claim is enough.

A fully connected chevron layer can be understood in three equivalent ways:

1. as an MLP with 2-component neurons
2. as two densely coupled channels

3. as a standard dense layer with a structured block-matrix form

That is enough to make the object clear without mystifying it.

The next question is the more interesting one: once a node is treated as a paired state rather than a scalar, what kinds of learning dynamics become natural? Different time scales? Local actor/critic updates? Tension-sensitive learning? Recurrent correction loops? More biologically suggestive motifs?

That is where the story really begins.

I'll close with a few notes on the core idea. Please feel free to skip through these. The hope is to deepen the intuition built up so far before shifting gears again in the next post.

Andre with ChatGPT-5.4

April 2026

Note 0: What Happens at the Node?

So far, the chevron picture has focused on the **edge**: each connection carries a 2×2 block that mixes the two channels as signals move forward. But that still leaves an important question open. Once the paired values arrive at a node, what happens there?

The simplest option is to apply an ordinary activation function to each channel separately. That keeps the architecture very close to a structured MLP. The channels travel together, but the node itself does not yet do anything special with their pairing.

More interesting possibilities appear when the node acts on the two channels **jointly**. The pair can be normalized as a single 2-vector, so that the node treats them as one local state rather than two unrelated scalars. One channel can gate the other, turning the node into a small unit of

modulation or inhibition. The node can also compare the two channels directly, for example by extracting a difference, a total, or both, so that it explicitly represents opposition as well as overall commitment.

This is where the architecture stops being just a partitioned dense layer and starts to become more distinctive. The 2×2 blocks define how paired signals are transported across the network. The node rule defines what that pairing means locally. If the edge gives the chevron network its structure, the node gives it its interpretation.

The most ambitious extension would be to let the node maintain its own local state and update it through the tension between the two channels. At that point, the chevron node is no longer just a two-valued neuron. It becomes a minimal dynamical system.

The edge tells us how the pair travels; the node tells us how the pair is resolved.

Fully connected still means “sum over all previous nodes”; the only difference is that each message is now a paired state transformed by a 2×2 operator.

Note 1: A Simple Limiting Case - Disconnecting the Channels

One useful feature of the chevron picture is that it contains the ordinary two-network case as a special limit. If the cross-channel weights are set to zero, the two channels decouple completely.

For a chevron edge

$$W_{ij} = \begin{bmatrix} w_{ijAA} & w_{ijAN} \\ w_{ijNA} & w_{ijNN} \end{bmatrix},$$

the channels are disconnected when the off-diagonal terms vanish:

$$w_{ijAN} = 0, w_{ijNA} = 0$$

for all connections i, j . In that case each edge reduces to a diagonal 2×2 block,

$$W_{ij} = \begin{bmatrix} w_{ijAA} & 0 \\ 0 & w_{ijNN} \end{bmatrix},$$

and the layer splits into two independent dense networks, one for each channel. So the chevron architecture does not replace the familiar two-layer picture. It contains it as a limiting case. The full chevron network is what you get when those off-diagonal terms are allowed to become nonzero, so that each channel can influence, correct, inhibit, or amplify the other. That makes the decoupled case a useful baseline: it shows exactly what extra expressive structure is introduced when the channels are coupled.

Note 2: Inhibition as a Special Case

One reason the chevron form is interesting is that it makes **cross-channel control** local and explicit. In biological brains, inhibition is not an optional extra. It is central to stability, selection, competition, timing, and gain control. A chevron network contains a simple abstract analogue of that idea.

For a chevron connection

$$W_{ij} = [w_{ijAA} \ w_{ijAN}$$

$$w_{ijNA} \ w_{ijNN}],$$

the off-diagonal terms w_{ijAN} and w_{ijNA} determine how one channel affects the other. If one channel acts to reduce, damp, or gate the activity of the other, then the connection is functioning as a form of inhibition.

In that sense, inhibition appears here as a special case of chevron coupling: one channel does not merely travel alongside the other, but can constrain it.

This is one of the main differences between a chevron layer and two independent parallel networks. In the decoupled case, the channels simply coexist. In the coupled case, they can regulate one another. One channel may support the other, correct it, suppress it, or hold it within bounds. That makes the chevron node more than a two-value neuron. It becomes a small local site of interaction and control.

That does not mean a chevron layer reproduces the full biological reality of inhibition. Real brains add timing, thresholds, nonlinear competition, cell types, and recurrent circuitry and who knows what more. But as an architectural abstraction, the chevron form captures something important: **the ability of one channel to modulate another is built into the primitive itself.** The off-diagonal terms do not just add expressiveness. They turn pairing into regulation.

Note 3: Complex, Dual, and Full Freedom

Once we allow a node to carry two values, a natural question arises: how tightly should those two channels be tied together?

There are three useful reference points from mathematics.

Complex Numbers: Rotation and Phase

Complex-valued neural networks treat a two-channel state as a single complex number,

$$z = a + bi, i^2 = -1.$$

Under this constraint, the allowed 2×2 weight matrices are not arbitrary.

They must take the form

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix},$$

which corresponds to rotation and scaling in the plane.

This is a powerful and elegant structure, but it is also restrictive. The two channels are locked into a symmetric relationship. They can rotate and scale together, but they cannot independently inhibit, gate, or asymmetrically control one another.

Dual Numbers: Shear and Variation

Dual numbers offer a different way to pair two values,

$$d = a + b\varepsilon, \varepsilon^2=0.$$

Here the induced 2×2 structure is

$$\begin{bmatrix} a & 0 \\ b & a \end{bmatrix},$$

which behaves like a shear rather than a rotation.

This suggests a different interpretation of the two channels:

- one as a base value
- the other as a variation, correction, or sensitivity of that value

Unlike the complex case, the relationship is asymmetric. The second channel depends on the first, but not vice versa. This begins to look more like modulation or regulation than geometry.

But dual numbers are still constrained. They cannot express full bidirectional coupling or independent channel dynamics.

Chevron Blocks: Learned Local Algebra

A chevron connection removes these constraints entirely. Its 2×2 matrix is fully free:

$[w_{AA} \ w_{AN}$

$w_{NA} \ w_{NN}]$.

This is the most general linear relation between the two channels. It can represent:

- rotation-like coupling (complex case)
- shear-like modulation (dual case)
- inhibition and gating
- asymmetric control
- independent amplification or damping of each channel

In this sense, a chevron network can be viewed as learning its own local algebra at each connection. Instead of committing in advance to rotation (complex) or variation (dual), it allows the relationship between the two channels to be shaped by learning.

A Small Ladder

It is helpful to see these as points on a spectrum:

- **Real scalar** → one channel
- **Complex number** → two channels with rotational coupling
- **Dual number** → two channels with shear/variation coupling
- **Chevron block** → two channels with fully learned coupling

The chevron layer does not reject the earlier cases. It contains them as special forms. But by allowing all four weights to vary freely, it shifts the emphasis from fixed geometry to **dynamic regulation**.

Why This Matters

Complex numbers are ideal when phase and oscillation are central. Dual numbers are natural when tracking variation or sensitivity. Chevron blocks step beyond both. They make it possible for the two channels to genuinely oppose, constrain, or stabilize one another.

That is the key shift: the pair is no longer bound to a predefined relationship. It becomes a small, learnable system of interaction.

Complex numbers rotate, dual numbers shear, chevron blocks regulate.

A useful reference case is the complex-valued 2×2 matrix. If a complex weight is written as $r^* e^{i\phi}$, its action on a real two-component state can be represented as

$$r^* \begin{bmatrix} \cos\phi & -\sin\phi \end{bmatrix}$$

$\sin\phi \cos\phi]$,

which simply rotates the pair by angle ϕ and scales it by r (or not at all if $r = 1$). This is a helpful special case because it shows that paired-channel computation already has a well-known mathematical form. A chevron block includes this possibility, but does not stop there. By freeing all four entries of the 2×2 matrix, it moves beyond rotation and scaling to allow inhibition, asymmetric coupling, gating, and more general regulation.

It also makes contact with the [R rule](#), where phase already plays a central role in balancing different influences. In that sense, the complex matrix gives a constrained picture of phase-sensitive two-channel dynamics, while the chevron block generalizes the idea by allowing all four entries to vary freely. The result is a local coupling that can still express rotation-like behaviour, but is no longer limited to it.

Note 4: Superposition as AND/OR Mixing

A natural question is whether a chevron node can represent something like *superposition*: multiple possibilities coexisting before a decision or collapse.

In a strict quantum sense, superposition involves complex amplitudes, phase, and interference. A general chevron layer does not enforce those constraints. But it can represent a simpler and often more useful idea: **coexisting alternatives that are later resolved through interaction.**

One way to see this is to interpret the two channels as holding different aspects of a proposition. For example:

- A: support for a hypothesis
- N: counter-support, constraint, or alternative

Now consider how a chevron connection mixes inputs:

y_A

$$y_N = \sum_j [w_{AA} w_{AN}$$

$w^{NA} w^{NN}] [x_j^A$

$x_j^N]$.

$$\begin{bmatrix} y^A \\ y^N \end{bmatrix} = \sum_j \begin{bmatrix} w^{AA} & w^{AN} \\ w^{NA} & w^{NN} \end{bmatrix} \begin{bmatrix} x_j^A \\ x_j^N \end{bmatrix}.$$

Two kinds of combination naturally appear:

- **Within-channel accumulation** (w^{AA} , w^{NN}) behaves like an **OR**: multiple sources contribute to the same channel, reinforcing or competing within it.
- **Cross-channel mixing** (w^{AN} , w^{NA}) behaves more like an **AND-like interaction**: one channel conditions or modulates the other, introducing dependence between them.

So a chevron node does not simply accumulate evidence. It **holds and mixes alternatives**:

- OR-like accumulation gathers possibilities

- AND-like coupling makes them interact, constrain, or suppress one another

This gives a form of **computational superposition**: multiple candidates are present simultaneously, but not independently. They coexist in a structured tension, influencing one another before any final selection.

Not Quantum, but Useful

It is important not to overstate the analogy. Without phase and interference rules, this is not quantum superposition. There is no guaranteed constructive or destructive interference in the strict sense.

But for many cognitive and computational purposes, this simpler form may be exactly what is needed. The network can:

- hold competing hypotheses at once
- allow them to reinforce or inhibit each other
- delay commitment until later layers
- resolve tension through dynamics rather than immediate choice

A Different Kind of Superposition

In this view, superposition is not about waves interfering in a complex plane. It is about **alternatives coexisting under constraint**.

A standard network tends to collapse alternatives quickly into a single scalar activation. A chevron network can keep them apart just long enough for them to interact.

That interaction—mediated by the 2×2 coupling—is where something like superposition lives.

In a chevron network, superposition is not phase—it is tension.

Note 5: Toward Quantum-Like Behaviour

If a chevron network already supports a kind of “computational superposition,” a natural next question is whether it can be extended to

something closer to *quantum* superposition—where phase and interference play a central role.

The short answer is: **yes, but only with additional structure.**

Step 1: Make the Channels Complex

Instead of each node carrying two real values,

$$(A, N) \in \mathbb{R}^2,$$

we allow each channel to be complex:

$$(A, N) \in \mathbb{C}^2.$$

Now each channel has both magnitude and phase. This makes it possible, in principle, for different contributions to interfere constructively or destructively.

At this point, each node is effectively a **2-component complex vector**, and each connection becomes a **2×2 complex matrix**.

Step 2: Why This Is Not Enough

Even with complex values, a general chevron layer is still too unconstrained to behave like a quantum system.

A true quantum evolution requires specific properties:

- **linearity over complex amplitudes**
- **norm preservation** (probability must sum to one)
- **unitary transformations** (no arbitrary gain or loss)
- **phase-sensitive interference**

A fully free chevron matrix does not enforce any of these. It can amplify, damp, distort, or collapse signals arbitrarily.

So while complex-valued chevron nets *can* exhibit interference-like effects, they are not automatically quantum-like systems.

Step 3: Constraining Toward Quantum Behaviour

To move closer to genuine quantum behaviour, additional constraints would be needed. For example:

- restricting each 2×2 block to be **unitary**
- normalizing the node state so that $|A|^2 + |N|^2 = 1$
- using nonlinearities that preserve or respect phase relationships

At that point, the chevron layer begins to resemble a network of small quantum-like transformations acting on local state vectors.

Chevron vs Quantum

This highlights a useful distinction:

- **Quantum models** fix the algebra in advance (complex amplitudes, unitary evolution)
- **Chevron models** leave the algebra free and learn it from data

So a chevron network can *include* quantum-like behaviour as a special constrained case, but its purpose is different. It is not to simulate

quantum systems, but to provide a flexible framework where different kinds of coupling—rotation, inhibition, modulation, or interference—can emerge.

A Useful Way to Think About It

You can think of it as a ladder:

- real scalar → single value
- real chevron → paired values with learned coupling
- complex chevron → paired complex amplitudes
- constrained complex chevron → quantum-like dynamics

Only the last step gives you something close to true quantum superposition.

Complex channels give you phase. Constraints give you physics.

It is tempting to push the chevron idea all the way toward quantum mechanics by introducing complex amplitudes and unitary constraints. But for most purposes, that is probably unnecessary. The core value of

the chevron architecture already appears at the real-valued level: the ability to hold paired states, allow them to interact locally, and delay collapse through structured tension. Complex numbers add phase and interference, but they also introduce a rigid algebra that may distract from the more general goal of learned regulation. In that sense, the quantum analogy is best treated as an optional extension rather than a destination. The chevron network does not need to become quantum-like to be useful; its strength lies in making opposition, modulation, and constraint first-class computational primitives.

Note 6: Two Kinds of Phase

There are at least two different ways phase can enter the chevron picture. The first comes from the analogy with complex numbers, where a two-channel state can be written in terms of magnitude and phase, with the channels related like $\cos\phi$ and $\sin\phi$. In that case, phase is part of the internal geometry of the pair itself.

But there is a second, more biological notion of phase. A channel may participate in a repeated oscillatory cycle, so that its influence depends on timing as well as magnitude. In that case, phase is not just a relation between two numbers at an instant. It is a temporal rhythm that gates when one channel excites, inhibits, or modulates the other. This suggests that chevron networks may have two distinct phase-like extensions: one algebraic, tied to paired representation, and one dynamical, tied to oscillation, timing, and biological control. The first is about rotation in a space of states. The second is about regulation in time.

One phase rotates a pair; the other phase schedules its interaction.

Note 7: Feedforward Control Beyond the Local Pair

So far, the chevron picture has focused on local coupling: each 2×2 block lets one channel influence the other at a single connection. But the same

idea can be extended across layers. One channel may feed forward to later layers and alter how they respond there, either positively or negatively.

This matters because biological control is often not purely local. Inhibition, in particular, often works by shaping downstream activity before it fully develops. Rather than simply cancelling a signal at the same point where it appears, it can arrive early enough to gate, narrow, or veto later responses. In that sense, inhibition is often feedforward as well as local.

A chevron architecture can express a simple analogue of this. One channel can project ahead as a control signal, biasing or damping later activations, while the other continues to carry the main signal path. That makes the architecture richer than a stack of isolated paired nodes. It becomes a network in which regulation can travel forward through depth, not just sideways within a local pair.

The result is a broader picture of chevron computation: not just two channels interacting at one node, but two channels able to shape one another across layers through both local coupling and feedforward control.

The off-diagonal terms make local regulation possible; feedforward cross-links let regulation travel.

One small technical note: in neuroscience, “feedforward inhibition” usually means an excitatory input activates an inhibitory interneuron, which then suppresses downstream targets very quickly. So the biological pattern is often not just a direct negative line, but a small circuit.

Note 8: A Deliberate Simplification

In the version discussed here, I am deliberately excluding recurrence and backward flows between channels. The aim is to keep the architecture

feedforward, trainable, and easy to compare with an ordinary MLP. That keeps the basic idea clear: two paired channels, mixed locally by 2×2 blocks, moving forward through depth without additional temporal complications.

But this simplification also points to an obvious next question. Once the channels are allowed to influence one another, why should that influence be confined to a single forward pass? Biological systems make heavy use of recurrence, delayed inhibition, feedback correction, and oscillatory loops. A feedforward chevron layer is therefore best seen as the simplest tractable case, not the final form. It isolates the architecture's core idea while leaving open the more interesting possibility that paired channels might later interact through recurrent, feedback, or phase-dependent dynamics.

The feedforward version is the clean baseline; the recurrent version is where the deeper questions begin.

Note 9: From Chevron Transport to R-Rule Dynamics

Once the chevron layer is understood as a way of transporting paired signals through 2×2 couplings, a natural next step appears. The node itself no longer has to be a passive point where inputs are merely summed and passed through a standard nonlinearity. It can instead become a small dynamical unit. This opens the door to using the [R rule](#) as the node update. In that picture, chevron connectivity determines how actor-like and norm-like signals arrive, while the R rule determines how the local paired state evolves in response. The architecture then stops being just a structured MLP and becomes something richer: a network of paired states whose updates depend on uncertainty, tension, and the balance between proposal and constraint.

Chevron blocks move the pair; the R rule can let the pair negotiate.

A Geometric Summary: Fibers Over a Network

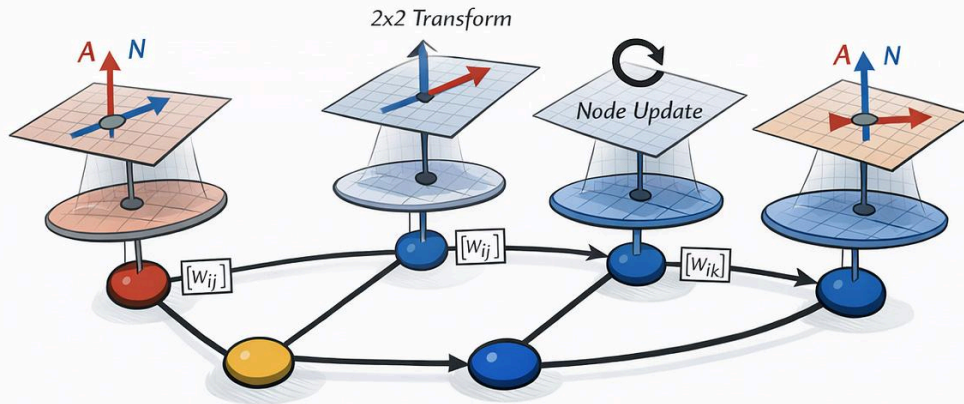
At this point, the chevron architecture can be summarized in a more geometric way. Rather than thinking of the network as a graph of isolated scalar activations, it is better to think of it as a graph whose every node carries a small local state space - an “inner space”. The graph itself provides the base structure. At each node sits a paired fiber, typically a copy of $\mathbf{R}^2$, holding the local A/N state. The full chevron network can then be pictured as a bundle of these paired-state fibers attached across the graph.

This picture gathers together the main ideas of the post. The 2×2 edge weights act as transport maps from one local fiber to another, telling us how paired states move forward through the network. The node rule then determines what happens locally inside each fiber: whether the two channels simply pass through, compete, gate one another, normalize jointly, or evolve according to a richer update such as the R rule. In that

sense, the chevron net separates naturally into two parts: transport across the graph, and dynamics within the local pair.

That is why bundle language is useful here. It is not decorative physics vocabulary. It captures a real architectural shift. A standard network treats each node as a place where one scalar lives. A chevron network treats each node as a place with an inner space, and each edge as a learned map between such spaces. The graph gives the skeleton; the fibers give each node an internal state.

A Chevron Network as a Bundle



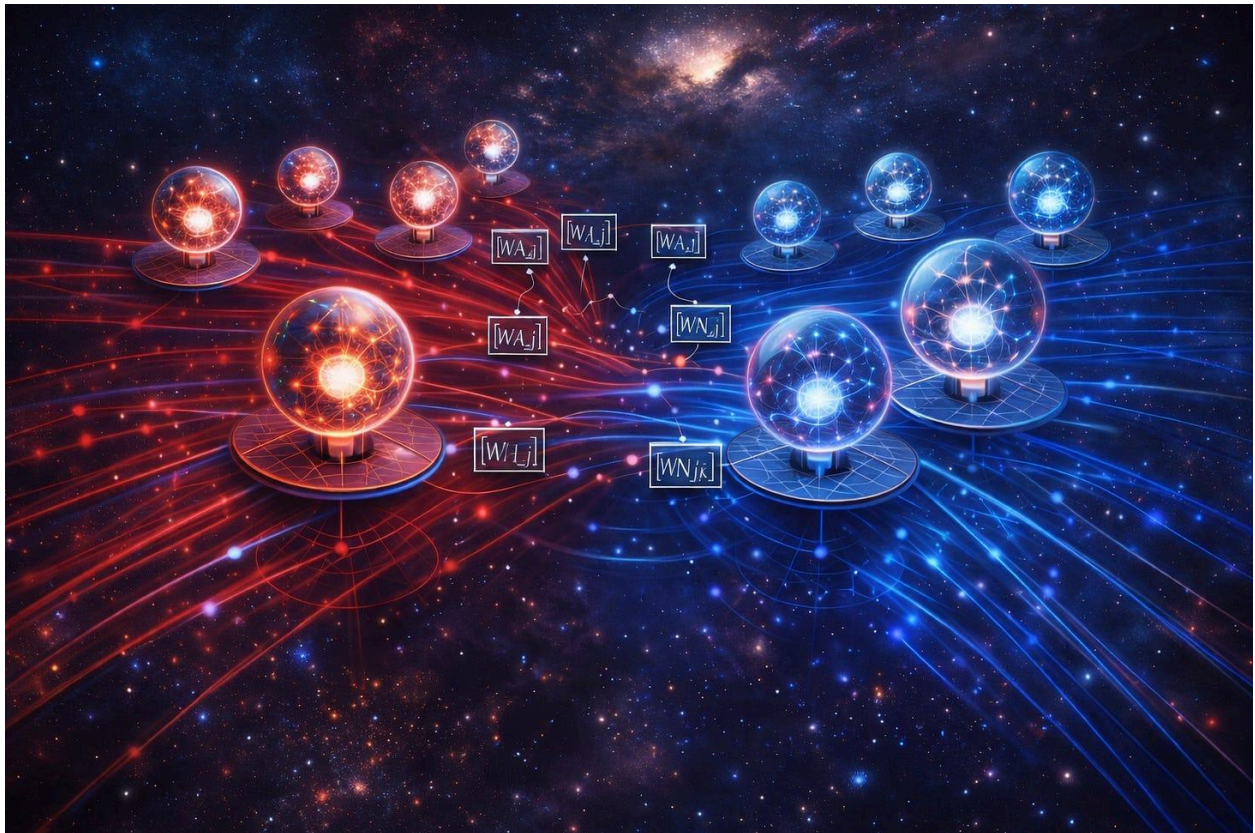
The chevron network can be viewed as a bundle. The graph is the base space, and each node has a local fiber, with 2x2 transformations mapping the paired states.

Three Ways to Think About a Chevron Network

From ordinary MLPs to paired channels to block-matrix layers.

[Andre Kramer](#)

Apr 16, 2026



When I describe a [chevron network](#), it can sound more exotic than I really mean. Each node has **two channels** rather than one. Each

connection carries a **2×2 matrix** rather than a single scalar weight. That sounds like a break with ordinary neural networks.

But it is not. Or at least, not at first.

A fully connected chevron layer is still best understood as a kind of dense layer. The real change is smaller and more interesting: the basic unit of computation is no longer a single scalar activation, but a **paired state**.

Instead of passing one number from node to node, we pass a 2-vector.

Instead of weighting that with one number, we transform it with a little 2×2 operator.

That one shift gives three different ways to understand the same object:

1. as an **MLP with richer neurons**
2. as **two densely coupled channels**
3. as a **standard dense layer with a structured block-matrix form**

These three views are equivalent, but they are not interchangeable. Each one highlights something different. The first gives familiarity. The second gives intuition. The third gives mathematical precision.

Taken together, they turn the chevron network from something mysterious into something quite concrete.

1. The MLP View

The easiest way to [think about a chevron network](#) is as an ordinary MLP, except that each node holds **two numbers instead of one**.

In a standard dense network:

- each node has a scalar activation
- each edge has a scalar weight

In a chevron network:

- each node has a 2-component activation
- each edge has a 2×2 weight matrix

So if x_j is the state of node j , and y_i is the state of node i in the next layer, a chevron layer looks like this:

$$y_i = \phi(\sum_j W_{ij} * x_j + b_i), \quad x_j, y_i \in \mathbb{R}^2, \quad W_{ij} \in \mathbb{R}^{2 \times 2}.$$

$$y_i = \phi\left(\sum_j W_{ij} x_j + b_i\right), \quad x_j, y_i \in \mathbb{R}^2, \quad W_{ij} \in \mathbb{R}^{2 \times 2}.$$

That is just the dense-layer equation again, with scalars replaced by 2-vectors and scalar weights replaced by 2×2 operators.

This is the first thing worth stressing: a chevron layer is not a rejection of the MLP idea. It is a direct generalization of it.

If an MLP says, “each neuron carries one scalar state,” then a chevron MLP says, “each neuron carries a paired state.”

That is the mathematical core.

2. The Coupled-Channel View

The second view is more intuitive.

A chevron network can be pictured as having [two channels running through the same layer](#) structure. Call them A and N, though the labels are placeholders. Depending on the application, they might stand for:

- actor and critic
- signal and error
- positive and negative evidence
- self and world
- excitation and inhibition

This is where the “two networks” analogy becomes useful. You can imagine a top network and a bottom network, both fully connected, moving forward together.

But that picture is only an approximation.

The two channels are not really independent networks sitting side by side. They are coupled at every edge. Each connection from node j to node i carries four weights:

$W_{ij} = [w_{ij}^{AA}, w_{ij}^{AN}$

$w_{ij}^{NA}, w_{ij}^{NN}]$.

$$W_{ij} = \begin{bmatrix} w_{ij}^{AA} & w_{ij}^{AN} \\ w_{ij}^{NA} & w_{ij}^{NN} \end{bmatrix}.$$

So the output in one channel depends on both channels from the previous layer:

$$y_i^A = \sum_j (w_{ij}^{AA} x_j^A + w_{ij}^{AN} x_j^N) + b_i^A$$

$$y_i^N = \sum_j (w_{ij}^{NA} x_j^A + w_{ij}^{NN} x_j^N) + b_i^N.$$

$$y_i^A = \sum_j (w_{ij}^{AA} x_j^A + w_{ij}^{AN} x_j^N) + b_i^A$$

$$y_i^N = \sum_j (w_{ij}^{NA} x_j^A + w_{ij}^{NN} x_j^N) + b_i^N.$$

That is why “two separate MLPs” is not quite right. The cross terms matter. The whole point is that the two channels can preserve, amplify, inhibit, or transform one another locally at each connection.

So the better description is this:

A chevron network is not two networks sharing a few links. It is one network whose node state is intrinsically two-channel.

This matters conceptually. Once the two channels are built into the primitive, it becomes natural to think in terms of tension, correction, opposition, or coupled roles rather than isolated scalar activations.

3. The Flattened Block-Matrix View

The third view is the most exact.

Suppose one layer has n chevron nodes and the next has m . Since each node carries two values, the input can be flattened into a vector of length $2n$, and the output into a vector of length $2m$.

Then the chevron layer can be written as an ordinary dense transformation:

$$y_{\text{flat}} = \phi(W_{\text{big}}x_{\text{flat}} + b_{\text{flat}}).$$

$$y_{\text{flat}} = \phi(W_{\text{big}}x_{\text{flat}} + b_{\text{flat}}).$$

So far, nothing unusual.

But the large matrix W_{big} is not arbitrary. It has a very specific structure: it is composed of 2×2 blocks,

$$W_{\text{big}} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1n} \\ W_{21} & W_{22} & \cdots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m1} & W_{m2} & \cdots & W_{mn} \end{bmatrix},$$

$$W_{21} \ W_{22} \ \cdots \ W_{2n}$$

$$\vdots \ \vdots \ \ddots \ \vdots$$

$$W_{m1} \ W_{m2} \ \cdots \ W_{mn}],$$

$$W_{\text{big}} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1n} \\ W_{21} & W_{22} & \cdots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m1} & W_{m2} & \cdots & W_{mn} \end{bmatrix},$$

where each block W_{ij} is itself a 2×2 matrix.

This is probably the cleanest exact statement of what a chevron layer is:

a standard dense layer with a built-in 2×2 block structure.

That sounds technical, but it matters. A generic dense layer on $2n$ inputs could mix all features arbitrarily. A chevron layer keeps the two components locally paired. It preserves the idea that these two values belong together at each node and should be transformed together at each edge.

That is not merely an implementation detail. It is the architectural prior.

A Clarification: When Is a Chevron Layer Really Different?

At this point, a sharp reader might notice a theoretical catch. If we apply a standard element-wise activation function to the flattened output, then a chevron layer is mathematically equivalent to an ordinary dense layer with twice as many nodes. In principle, a standard network could represent exactly the same 2×2 block structure on its own.

But it has no reason to do so. A conventional dense layer treats all $2n$ features as an undifferentiated pool, free to mix arbitrarily. The chevron form does something more specific: it makes the pairing explicit. It says that these two values belong together, should travel together, and should be transformed together.

That already matters as an architectural prior. But it matters even more because it gives a clear boundary where the equivalence can be deliberately broken.

As soon as we apply an operation that acts on the two channels jointly rather than independently, the chevron layer is no longer just a partitioned MLP. For example, one channel might gate the other, the two components might be normalized as a single 2-vector, or a tension-based learning rule might update the whole 2×2 block as one dynamic unit. At that point, the architecture is no longer just simulating paired structure inside a conventional network. It becomes a genuine network of paired states.

The key point is this: the chevron form begins as a structured dense layer, but it creates a natural place where new dynamics can enter. That is where the architecture stops being merely notational and starts becoming conceptually distinct.

Same Layer, Three Views

At this point, it becomes clear why all three descriptions are useful.

The **MLP view** says: this is familiar territory. It is still a dense layer.

The **coupled-channel view** says: the two channels can play different but coordinated roles.

The **flattened view** says: the whole thing can be implemented exactly and efficiently in standard frameworks.

Each view hides something that the others reveal.

The MLP view is reassuring, but can make the two-channel structure seem like a small tweak. The coupled-channel view captures the conceptual richness, but can sound vague if not tied down mathematically. The flattened view is exact, but can hide the intuition that the two values are meant to travel as a pair.

So I think the right way to present a chevron network is not to pick one of these views and defend it as the true one. It is to say that the same object can be seen from all three angles, and that each angle helps for a different reason.

Research Overlap: Complex Networks and Capsules

In the machine learning literature, the closest **mathematical cousin** to the chevron node is the **complex-valued neural network** (CVNN).

CVNNs also replace scalar activations with paired states: the real and

imaginary parts of a complex number. Their connections can likewise be written as 2×2 blocks.

But a CVNN imposes a strict constraint on those blocks. The weights must obey the symmetry of complex multiplication, so the two channels are tied to the geometry of phase, rotation, and scaling in the complex plane. A chevron network deliberately removes that restriction. Its 2×2 blocks are fully free. That shifts the emphasis away from geometric rotation and toward dynamic regulation. The two channels need not merely rotate together. They can oppose, inhibit, gate, amplify, or stabilize one another.

Conceptually, the chevron architecture has a different kind of kinship with **Geoffrey Hinton's Capsule Networks**. CapsNets were built around an important criticism of ordinary neural nets: scalar neurons tend to discard relational structure that ought to be preserved. Their answer was to make the basic unit of representation a vector rather than a lone scalar, and to use dynamic routing to preserve part-whole and pose relationships.

Chevron networks share that dissatisfaction with the isolated scalar neuron, but they aim at something simpler and more primitive. A capsule is designed to track structured geometry: orientation, pose, and hierarchical composition. A chevron node is designed to track a local duality: proposal and norm, signal and correction, drive and restraint. It keeps the vector idea, but strips away the heavy routing machinery in favor of the simplest possible dense form. The goal is not to model the 3D pose of an object. It is to make opposition, modulation, and internal tension computationally explicit.

A standard network begins with the lonely scalar. A capsule replaces it with a structured vector for spatial reasoning. A chevron replaces it with the simplest paired state needed for regulation.

Breakout: A Minimal PyTorch Chevron Layer

The code makes the structure very explicit.

A fully connected chevron layer can be represented with:

- inputs of shape `[batch, in_nodes, 2]`
- outputs of shape `[batch, out_nodes, 2]`
- weights of shape `[out_nodes, in_nodes, 2, 2]`

Here is a minimal implementation:

```
import torch

import torch.nn as nn

class ChevronLinear(nn.Module):

    def __init__(self, in_nodes, out_nodes):

        super().__init__()

        self.weight = nn.Parameter(

            torch.randn(out_nodes, in_nodes, 2, 2) * 0.1

        )

        self.bias = nn.Parameter(torch.zeros(out_nodes, 2))

    def forward(self, x):

        # x: [batch, in_nodes, 2]

        # output: [batch, out_nodes, 2]

        return torch.einsum("ijca,bja->bic", self.weight, x) +

            self.bias.unsqueeze(0)
```

This is a dense layer in the most literal sense: every input node talks to every output node. The only difference is that each edge carries a 2×2 matrix rather than a scalar.

If one wanted, one could flatten the whole thing into a standard `nn.Linear` acting on vectors of length `2 * in_nodes`. But writing it in chevron form makes the intended structure visible rather than hiding it in a large unstructured matrix.

That visibility is important. It reminds us that the two channels are not just extra features. They are paired states.

Why Bother?

At one level, one might say this is all only notation. A large enough conventional network can often represent the same functions.

But notation is not trivial. Architectural form matters because it shapes what is easy to express, easy to interpret, and easy to train.

A standard scalar neuron treats each activation as an isolated number.

A chevron neuron treats each node as a paired state.

That small change opens the door to a whole family of ideas that feel much more natural in this setting:

- different learning rates for the two channels
- separate fast and slow dynamics
- cross-channel inhibition or correction
- [actor/critic-style interpretations](#)
- self/world or proposal/evaluation pairings
- local tension-based updates rather than only global backpropagation

- biological analogies such as excitatory/inhibitory or feedforward/feedback motifs

The architecture does not force any one of these interpretations. But it makes them available in a clean and local way.

That, to me, is why it is worth taking seriously.

The Real Shift

The real novelty of the chevron picture is not that it adds more parameters. It is that it changes the primitive object.

A scalar neuron says: computation happens through isolated values.

A chevron neuron says: computation happens through a structured relation between two coordinated values.

That is a small mathematical shift, but potentially a large conceptual one.

It suggests that the basic unit of computation need not be a lone activation. It could be a tension, a polarity, a proposal paired with a correction, or some other local duality that travels together through the network.

Whether that turns out to matter empirically remains to be seen. But at the level of architecture, it already gives a cleaner language for talking about systems where two aspects belong together.

From Here

For now, I think the modest claim is enough.

A fully connected chevron layer can be understood in three equivalent ways:

1. as an MLP with 2-component neurons
2. as two densely coupled channels

3. as a standard dense layer with a structured block-matrix form

That is enough to make the object clear without mystifying it.

The next question is the more interesting one: once a node is treated as a paired state rather than a scalar, what kinds of learning dynamics become natural? Different time scales? Local actor/critic updates? Tension-sensitive learning? Recurrent correction loops? More biologically suggestive motifs?

That is where the story really begins.

I'll close with a few notes on the core idea. Please feel free to skip through these. The hope is to deepen the intuition built up so far before shifting gears again in the next post.

Andre with ChatGPT-5.4

April 2026

Note 0: What Happens at the Node?

So far, the chevron picture has focused on the **edge**: each connection carries a 2×2 block that mixes the two channels as signals move forward. But that still leaves an important question open. Once the paired values arrive at a node, what happens there?

The simplest option is to apply an ordinary activation function to each channel separately. That keeps the architecture very close to a structured MLP. The channels travel together, but the node itself does not yet do anything special with their pairing.

More interesting possibilities appear when the node acts on the two channels **jointly**. The pair can be normalized as a single 2-vector, so that the node treats them as one local state rather than two unrelated scalars. One channel can gate the other, turning the node into a small unit of

modulation or inhibition. The node can also compare the two channels directly, for example by extracting a difference, a total, or both, so that it explicitly represents opposition as well as overall commitment.

This is where the architecture stops being just a partitioned dense layer and starts to become more distinctive. The 2×2 blocks define how paired signals are transported across the network. The node rule defines what that pairing means locally. If the edge gives the chevron network its structure, the node gives it its interpretation.

The most ambitious extension would be to let the node maintain its own local state and update it through the tension between the two channels. At that point, the chevron node is no longer just a two-valued neuron. It becomes a minimal dynamical system.

The edge tells us how the pair travels; the node tells us how the pair is resolved.

Fully connected still means “sum over all previous nodes”; the only difference is that each message is now a paired state transformed by a 2×2 operator.

Note 1: A Simple Limiting Case - Disconnecting the Channels

One useful feature of the chevron picture is that it contains the ordinary two-network case as a special limit. If the cross-channel weights are set to zero, the two channels decouple completely.

For a chevron edge

$$W_{ij} = \begin{bmatrix} w_{ijAA} & w_{ijAN} \\ w_{ijNA} & w_{ijNN} \end{bmatrix},$$

the channels are disconnected when the off-diagonal terms vanish:

$$w_{ijAN} = 0, w_{ijNA} = 0$$

for all connections i, j . In that case each edge reduces to a diagonal 2×2 block,

$$W_{ij} = \begin{bmatrix} w_{ijAA} & 0 \\ 0 & w_{ijNN} \end{bmatrix},$$

and the layer splits into two independent dense networks, one for each channel. So the chevron architecture does not replace the familiar two-layer picture. It contains it as a limiting case. The full chevron network is what you get when those off-diagonal terms are allowed to become nonzero, so that each channel can influence, correct, inhibit, or amplify the other. That makes the decoupled case a useful baseline: it shows exactly what extra expressive structure is introduced when the channels are coupled.

Note 2: Inhibition as a Special Case

One reason the chevron form is interesting is that it makes **cross-channel control** local and explicit. In biological brains, inhibition is not an optional extra. It is central to stability, selection, competition, timing, and gain control. A chevron network contains a simple abstract analogue of that idea.

For a chevron connection

$$W_{ij} = [w_{ijAA} \ w_{ijAN}$$

$$w_{ijNA} \ w_{ijNN}],$$

the off-diagonal terms w_{ijAN} and w_{ijNA} determine how one channel affects the other. If one channel acts to reduce, damp, or gate the activity of the other, then the connection is functioning as a form of inhibition.

In that sense, inhibition appears here as a special case of chevron coupling: one channel does not merely travel alongside the other, but can constrain it.

This is one of the main differences between a chevron layer and two independent parallel networks. In the decoupled case, the channels simply coexist. In the coupled case, they can regulate one another. One channel may support the other, correct it, suppress it, or hold it within bounds. That makes the chevron node more than a two-value neuron. It becomes a small local site of interaction and control.

That does not mean a chevron layer reproduces the full biological reality of inhibition. Real brains add timing, thresholds, nonlinear competition, cell types, and recurrent circuitry and who knows what more. But as an architectural abstraction, the chevron form captures something important: **the ability of one channel to modulate another is built into the primitive itself.** The off-diagonal terms do not just add expressiveness. They turn pairing into regulation.

Note 3: Complex, Dual, and Full Freedom

Once we allow a node to carry two values, a natural question arises: how tightly should those two channels be tied together?

There are three useful reference points from mathematics.

Complex Numbers: Rotation and Phase

Complex-valued neural networks treat a two-channel state as a single complex number,

$$z = a + bi, i^2 = -1.$$

Under this constraint, the allowed 2×2 weight matrices are not arbitrary.

They must take the form

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix},$$

which corresponds to rotation and scaling in the plane.

This is a powerful and elegant structure, but it is also restrictive. The two channels are locked into a symmetric relationship. They can rotate and scale together, but they cannot independently inhibit, gate, or asymmetrically control one another.

Dual Numbers: Shear and Variation

Dual numbers offer a different way to pair two values,

$$d = a + b\varepsilon, \varepsilon^2=0.$$

Here the induced 2×2 structure is

$$\begin{bmatrix} a & 0 \\ b & a \end{bmatrix},$$

which behaves like a shear rather than a rotation.

This suggests a different interpretation of the two channels:

- one as a base value
- the other as a variation, correction, or sensitivity of that value

Unlike the complex case, the relationship is asymmetric. The second channel depends on the first, but not vice versa. This begins to look more like modulation or regulation than geometry.

But dual numbers are still constrained. They cannot express full bidirectional coupling or independent channel dynamics.

Chevron Blocks: Learned Local Algebra

A chevron connection removes these constraints entirely. Its 2×2 matrix is fully free:

$[w_{AA} \ w_{AN}$

$w_{NA} \ w_{NN}]$.

This is the most general linear relation between the two channels. It can represent:

- rotation-like coupling (complex case)
- shear-like modulation (dual case)
- inhibition and gating
- asymmetric control
- independent amplification or damping of each channel

In this sense, a chevron network can be viewed as learning its own local algebra at each connection. Instead of committing in advance to rotation (complex) or variation (dual), it allows the relationship between the two channels to be shaped by learning.

A Small Ladder

It is helpful to see these as points on a spectrum:

- **Real scalar** → one channel
- **Complex number** → two channels with rotational coupling
- **Dual number** → two channels with shear/variation coupling
- **Chevron block** → two channels with fully learned coupling

The chevron layer does not reject the earlier cases. It contains them as special forms. But by allowing all four weights to vary freely, it shifts the emphasis from fixed geometry to **dynamic regulation**.

Why This Matters

Complex numbers are ideal when phase and oscillation are central. Dual numbers are natural when tracking variation or sensitivity. Chevron blocks step beyond both. They make it possible for the two channels to genuinely oppose, constrain, or stabilize one another.

That is the key shift: the pair is no longer bound to a predefined relationship. It becomes a small, learnable system of interaction.

Complex numbers rotate, dual numbers shear, chevron blocks regulate.

A useful reference case is the complex-valued 2×2 matrix. If a complex weight is written as $r^* e^{i\phi}$, its action on a real two-component state can be represented as

$$r^* \begin{bmatrix} \cos\phi & -\sin\phi \end{bmatrix}$$

$\sin\phi \cos\phi]$,

which simply rotates the pair by angle ϕ and scales it by r (or not at all if $r = 1$). This is a helpful special case because it shows that paired-channel computation already has a well-known mathematical form. A chevron block includes this possibility, but does not stop there. By freeing all four entries of the 2×2 matrix, it moves beyond rotation and scaling to allow inhibition, asymmetric coupling, gating, and more general regulation.

It also makes contact with the [R rule](#), where phase already plays a central role in balancing different influences. In that sense, the complex matrix gives a constrained picture of phase-sensitive two-channel dynamics, while the chevron block generalizes the idea by allowing all four entries to vary freely. The result is a local coupling that can still express rotation-like behaviour, but is no longer limited to it.

Note 4: Superposition as AND/OR Mixing

A natural question is whether a chevron node can represent something like *superposition*: multiple possibilities coexisting before a decision or collapse.

In a strict quantum sense, superposition involves complex amplitudes, phase, and interference. A general chevron layer does not enforce those constraints. But it can represent a simpler and often more useful idea: **coexisting alternatives that are later resolved through interaction.**

One way to see this is to interpret the two channels as holding different aspects of a proposition. For example:

- A: support for a hypothesis
- N: counter-support, constraint, or alternative

Now consider how a chevron connection mixes inputs:

y_A

$$y_N = \sum_j [w_{AA} w_{AN}$$

$w^{NA} \ w^{NN}] [x_j^A$

$x_j^N]$.

$$\begin{bmatrix} y^A \\ y^N \end{bmatrix} = \sum_j \begin{bmatrix} w^{AA} & w^{AN} \\ w^{NA} & w^{NN} \end{bmatrix} \begin{bmatrix} x_j^A \\ x_j^N \end{bmatrix}.$$

Two kinds of combination naturally appear:

- **Within-channel accumulation** (w^{AA} , w^{NN}) behaves like an **OR**: multiple sources contribute to the same channel, reinforcing or competing within it.
- **Cross-channel mixing** (w^{AN} , w^{NA}) behaves more like an **AND-like interaction**: one channel conditions or modulates the other, introducing dependence between them.

So a chevron node does not simply accumulate evidence. It **holds and mixes alternatives**:

- OR-like accumulation gathers possibilities

- AND-like coupling makes them interact, constrain, or suppress one another

This gives a form of **computational superposition**: multiple candidates are present simultaneously, but not independently. They coexist in a structured tension, influencing one another before any final selection.

Not Quantum, but Useful

It is important not to overstate the analogy. Without phase and interference rules, this is not quantum superposition. There is no guaranteed constructive or destructive interference in the strict sense.

But for many cognitive and computational purposes, this simpler form may be exactly what is needed. The network can:

- hold competing hypotheses at once
- allow them to reinforce or inhibit each other
- delay commitment until later layers
- resolve tension through dynamics rather than immediate choice

A Different Kind of Superposition

In this view, superposition is not about waves interfering in a complex plane. It is about **alternatives coexisting under constraint**.

A standard network tends to collapse alternatives quickly into a single scalar activation. A chevron network can keep them apart just long enough for them to interact.

That interaction—mediated by the 2×2 coupling—is where something like superposition lives.

In a chevron network, superposition is not phase—it is tension.

Note 5: Toward Quantum-Like Behaviour

If a chevron network already supports a kind of “computational superposition,” a natural next question is whether it can be extended to

something closer to *quantum* superposition—where phase and interference play a central role.

The short answer is: **yes, but only with additional structure.**

Step 1: Make the Channels Complex

Instead of each node carrying two real values,

$$(A, N) \in \mathbb{R}^2,$$

we allow each channel to be complex:

$$(A, N) \in \mathbb{C}^2.$$

Now each channel has both magnitude and phase. This makes it possible, in principle, for different contributions to interfere constructively or destructively.

At this point, each node is effectively a **2-component complex vector**, and each connection becomes a **2×2 complex matrix**.

Step 2: Why This Is Not Enough

Even with complex values, a general chevron layer is still too unconstrained to behave like a quantum system.

A true quantum evolution requires specific properties:

- **linearity over complex amplitudes**
- **norm preservation** (probability must sum to one)
- **unitary transformations** (no arbitrary gain or loss)
- **phase-sensitive interference**

A fully free chevron matrix does not enforce any of these. It can amplify, damp, distort, or collapse signals arbitrarily.

So while complex-valued chevron nets *can* exhibit interference-like effects, they are not automatically quantum-like systems.

Step 3: Constraining Toward Quantum Behaviour

To move closer to genuine quantum behaviour, additional constraints would be needed. For example:

- restricting each 2×2 block to be **unitary**
- normalizing the node state so that $|A|^2 + |N|^2 = 1$
- using nonlinearities that preserve or respect phase relationships

At that point, the chevron layer begins to resemble a network of small quantum-like transformations acting on local state vectors.

Chevron vs Quantum

This highlights a useful distinction:

- **Quantum models** fix the algebra in advance (complex amplitudes, unitary evolution)
- **Chevron models** leave the algebra free and learn it from data

So a chevron network can *include* quantum-like behaviour as a special constrained case, but its purpose is different. It is not to simulate

quantum systems, but to provide a flexible framework where different kinds of coupling—rotation, inhibition, modulation, or interference—can emerge.

A Useful Way to Think About It

You can think of it as a ladder:

- real scalar → single value
- real chevron → paired values with learned coupling
- complex chevron → paired complex amplitudes
- constrained complex chevron → quantum-like dynamics

Only the last step gives you something close to true quantum superposition.

Complex channels give you phase. Constraints give you physics.

It is tempting to push the chevron idea all the way toward quantum mechanics by introducing complex amplitudes and unitary constraints. But for most purposes, that is probably unnecessary. The core value of

the chevron architecture already appears at the real-valued level: the ability to hold paired states, allow them to interact locally, and delay collapse through structured tension. Complex numbers add phase and interference, but they also introduce a rigid algebra that may distract from the more general goal of learned regulation. In that sense, the quantum analogy is best treated as an optional extension rather than a destination. The chevron network does not need to become quantum-like to be useful; its strength lies in making opposition, modulation, and constraint first-class computational primitives.

Note 6: Two Kinds of Phase

There are at least two different ways phase can enter the chevron picture. The first comes from the analogy with complex numbers, where a two-channel state can be written in terms of magnitude and phase, with the channels related like $\cos\phi$ and $\sin\phi$. In that case, phase is part of the internal geometry of the pair itself.

But there is a second, more biological notion of phase. A channel may participate in a repeated oscillatory cycle, so that its influence depends on timing as well as magnitude. In that case, phase is not just a relation between two numbers at an instant. It is a temporal rhythm that gates when one channel excites, inhibits, or modulates the other. This suggests that chevron networks may have two distinct phase-like extensions: one algebraic, tied to paired representation, and one dynamical, tied to oscillation, timing, and biological control. The first is about rotation in a space of states. The second is about regulation in time.

One phase rotates a pair; the other phase schedules its interaction.

Note 7: Feedforward Control Beyond the Local Pair

So far, the chevron picture has focused on local coupling: each 2×2 block lets one channel influence the other at a single connection. But the same

idea can be extended across layers. One channel may feed forward to later layers and alter how they respond there, either positively or negatively.

This matters because biological control is often not purely local. Inhibition, in particular, often works by shaping downstream activity before it fully develops. Rather than simply cancelling a signal at the same point where it appears, it can arrive early enough to gate, narrow, or veto later responses. In that sense, inhibition is often feedforward as well as local.

A chevron architecture can express a simple analogue of this. One channel can project ahead as a control signal, biasing or damping later activations, while the other continues to carry the main signal path. That makes the architecture richer than a stack of isolated paired nodes. It becomes a network in which regulation can travel forward through depth, not just sideways within a local pair.

The result is a broader picture of chevron computation: not just two channels interacting at one node, but two channels able to shape one another across layers through both local coupling and feedforward control.

The off-diagonal terms make local regulation possible; feedforward cross-links let regulation travel.

One small technical note: in neuroscience, “feedforward inhibition” usually means an excitatory input activates an inhibitory interneuron, which then suppresses downstream targets very quickly. So the biological pattern is often not just a direct negative line, but a small circuit.

Note 8: A Deliberate Simplification

In the version discussed here, I am deliberately excluding recurrence and backward flows between channels. The aim is to keep the architecture

feedforward, trainable, and easy to compare with an ordinary MLP. That keeps the basic idea clear: two paired channels, mixed locally by 2×2 blocks, moving forward through depth without additional temporal complications.

But this simplification also points to an obvious next question. Once the channels are allowed to influence one another, why should that influence be confined to a single forward pass? Biological systems make heavy use of recurrence, delayed inhibition, feedback correction, and oscillatory loops. A feedforward chevron layer is therefore best seen as the simplest tractable case, not the final form. It isolates the architecture's core idea while leaving open the more interesting possibility that paired channels might later interact through recurrent, feedback, or phase-dependent dynamics.

The feedforward version is the clean baseline; the recurrent version is where the deeper questions begin.

Note 9: From Chevron Transport to R-Rule Dynamics

Once the chevron layer is understood as a way of transporting paired signals through 2×2 couplings, a natural next step appears. The node itself no longer has to be a passive point where inputs are merely summed and passed through a standard nonlinearity. It can instead become a small dynamical unit. This opens the door to using the [R rule](#) as the node update. In that picture, chevron connectivity determines how actor-like and norm-like signals arrive, while the R rule determines how the local paired state evolves in response. The architecture then stops being just a structured MLP and becomes something richer: a network of paired states whose updates depend on uncertainty, tension, and the balance between proposal and constraint.

Chevron blocks move the pair; the R rule can let the pair negotiate.

A Geometric Summary: Fibers Over a Network

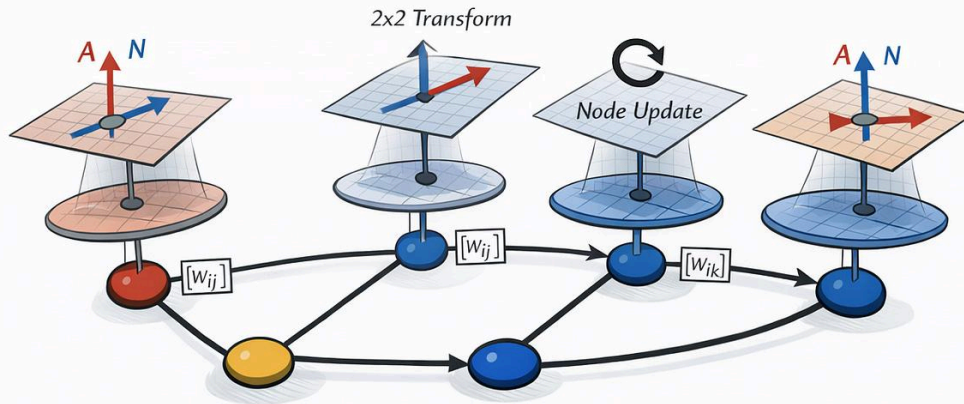
At this point, the chevron architecture can be summarized in a more geometric way. Rather than thinking of the network as a graph of isolated scalar activations, it is better to think of it as a graph whose every node carries a small local state space - an “inner space”. The graph itself provides the base structure. At each node sits a paired fiber, typically a copy of $\mathbf{R}^2$, holding the local A/N state. The full chevron network can then be pictured as a bundle of these paired-state fibers attached across the graph.

This picture gathers together the main ideas of the post. The 2×2 edge weights act as transport maps from one local fiber to another, telling us how paired states move forward through the network. The node rule then determines what happens locally inside each fiber: whether the two channels simply pass through, compete, gate one another, normalize jointly, or evolve according to a richer update such as the R rule. In that

sense, the chevron net separates naturally into two parts: transport across the graph, and dynamics within the local pair.

That is why bundle language is useful here. It is not decorative physics vocabulary. It captures a real architectural shift. A standard network treats each node as a place where one scalar lives. A chevron network treats each node as a place with an inner space, and each edge as a learned map between such spaces. The graph gives the skeleton; the fibers give each node an internal state.

A Chevron Network as a Bundle



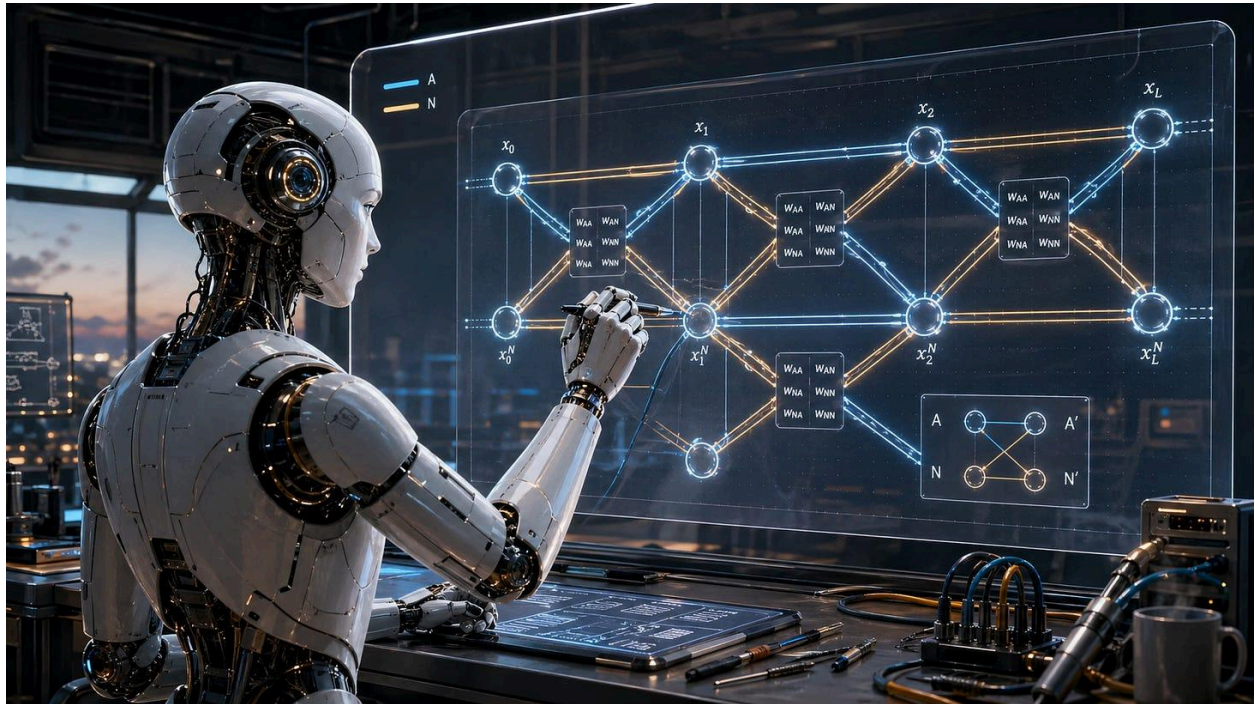
The chevron network can be viewed as a bundle. The graph is the base space, and each node has a local fiber, with 2x2 transformations mapping the paired states.

Exploring Allostatic Chevron Nets

Replay, surprise, and when tension should split a learner

[Andre Kramer](#)

Apr 30, 2026



The first standalone [allostatic chevron](#) experiment is now running.

The results are early, small, and not conclusive. But they are interesting enough to change the status of the idea. [Chevron nets](#) are no longer only a metaphor about fast adaptation and slow retention. They are now a testable architecture with a first measurable stability-plasticity curve.

This post has four parts.

First, I summarize the first Split MNIST results. Second, I place them beside Google's recent Titans and [Nested Learning](#) work, which points toward a similar concern with memory, surprise, and multiple learning frequencies. Third, I turn back to the chevron mechanism itself and ask a more speculative question: should strong unresolved tension sometimes split the learner into separate contexts? Finally, I give that idea a Batesonian interpretation, connecting it to sign reversal, meta-learning, and an ecology of mind.

The aim is to show that the first experiment produced the kind of pattern the theory was built to test — and to make the next experiments sharper while keeping the broader context in view.

1. The first allostatic chevron results

I've started experimenting with allostatic chevron nets using

ChatGPT-5.5 and Codex:

<https://github.com/andrekrumer/chevron-networks/tree/main/chevron-allostasis>

The setup is deliberately small: Split MNIST, five sequential binary tasks:

T1: 0 vs 1

T2: 2 vs 3

T3: 4 vs 5

T4: 6 vs 7

T5: 8 vs 9

The model trains on one task at a time. After each task, it is evaluated on all tasks seen so far. The main metric is forgetting: how much performance on earlier tasks drops after later tasks are learned.

The chevron model uses paired A/N hidden states.

A is the fast adaptive channel.

N is the slower retained or normative channel.

Each chevron layer uses local 2×2 coupling between the channels:

$$W_i =$$

$$\begin{bmatrix} w_i^{AA} & w_i^{AN} \end{bmatrix}$$

$$\begin{bmatrix} w_i^{NA} & w_i^{NN} \end{bmatrix}$$

The first experiment asks whether this A/N structure helps continual learning.

Does fast/slow A/N coupling help a model retain what it has learned while adapting to new tasks?

Result 1: without replay, it forgets

Without replay, the chevron model learns each new task very well, but forgets older ones badly.

Across five seeds, the no-replay full chevron result was:

final forgetting: 0.4843

final average accuracy: 0.6068

old-task average accuracy: 0.5106

current-task accuracy: 0.9916

So the model remains highly plastic. It adapts to the current task, but old tasks collapse.

That is useful. The architecture alone does not magically solve catastrophic forgetting. A paired A/N state is not enough. The allostatic schedule matters.

In other words, N does not save the system by merely existing. Retention has to be actively reconstructed.

Result 2: replay produces a clear retention curve

Adding replay and consolidation changed the picture.

Across five seeds:

	Replay Buffer	Forgetting	Final Avg Accuracy	Old-task Accuracy	Current-task Accuracy
--	---------------	------------	--------------------	-------------------	-----------------------

0	0.4843	0.6068	0.5106	0.9916	
---	--------	--------	--------	--------	--

100	0.3620	0.6975	0.6270	0.9796	
-----	--------	--------	--------	--------	--

500	0.1973	0.7917	0.7763	0.8532	
-----	--------	--------	--------	--------	--

1000	0.0675	0.8439	0.8831	0.6871	
------	--------	--------	--------	--------	--

This is the first important result.

Replay buffer size produced a clean retention curve. More replay meant less forgetting. The system moved from high-plasticity/high-forgetting to high-retention/lower-current-adaptation.

That is exactly the tradeoff the allostatic interpretation predicts.

The best first operating point was probably buffer 500: forgetting dropped sharply while current-task accuracy remained usable.

The result is not that chevrons abolish forgetting. They do not. The result is that chevrons make the stability-plasticity tradeoff visible and tunable.

Result 3: coupling matters more under A-only readout

I then compared the full chevron model against a diagonal-only version.

The diagonal-only model still has two A/N channels, but no cross-channel coupling. A maps to A, N maps to N, but A and N do not locally mix.

With both channels used for readout, the result was mixed. Full coupling was not consistently better than diagonal-only. That meant the first

result could not yet be claimed as evidence for local 2×2 coupling specifically.

So I ran a cleaner version: A-only readout.

That means the classifier reads only from the fast A channel. The N channel can still shape learning through coupling, but it cannot directly feed the output head.

Under A-only readout, full coupling consistently beat diagonal-only on retention:

Model	Buffer	Forgetting	Final Avg Accuracy	Old-task Accuracy	Current-task Accuracy
-------	--------	------------	--------------------	-------------------	-----------------------

Full chevron	100	0.3795	0.6864	0.6123	0.9828
--------------	-----	--------	--------	--------	--------

Diagonal-only	100	0.4182	0.6564	0.5735	0.9879
---------------	-----	--------	--------	--------	--------

Full chevron	500	0.2092	0.7909	0.7695	0.8768
--------------	-----	--------	--------	--------	--------

| Diagonal-only | 500 | 0.2403 | 0.7742 | 0.7422 | 0.9025 |

| Full chevron | 1000 | 0.0814 | 0.8462 | 0.8774 | 0.7211 |

| Diagonal-only | 1000 | 0.1577 | 0.8021 | 0.8038 | 0.7956 |

This is the second important result.

The A-only readout matters conceptually. When both A and N feed the output head, N can behave like an extra feature stream. That may be useful, but it makes the result harder to interpret.

When readout is constrained to A, N can only help by shaping A through the cross-channel terms. In that setting, full 2×2 coupling improving retention over diagonal-only coupling is much cleaner evidence for the chevron hypothesis.

A-only readout turns N from an output feature into an internal constraint.

The tradeoff remains. Diagonal-only keeps better current-task accuracy, especially at larger replay buffers. But full coupling is more retention-biased. That is not a failure of the allostatic picture. It is the picture.

Full coupling seems to buy old-task stability at a cost to current-task plasticity.

Result 4: chevron replay beats a scalar MLP replay baseline

The next question was whether this was just replay working in an ordinary way.

So I added a standard scalar MLP baseline using the same Split MNIST schedule, replay buffer sizes, and evaluation logic.

Results:

Model	Buffer	Forgetting	Final Avg Accuracy	Old-task Accuracy	Current-task Accuracy
-------	--------	------------	--------------------	-------------------	-----------------------

Full chevron	0	0.4843	0.6068	0.5106	0.9916
--------------	---	--------	--------	--------	--------

MLP	0	0.4987	0.5950	0.4965	0.9891
-----	---	--------	--------	--------	--------

Full chevron	100	0.3620	0.6975	0.6270	0.9796
--------------	-----	--------	--------	--------	--------

MLP	100	0.4756	0.6130	0.5191	0.9889
-----	-----	--------	--------	--------	--------

Full chevron	500	0.1973	0.7917	0.7763	0.8532
--------------	-----	--------	--------	--------	--------

MLP	500	0.3704	0.6924	0.6198	0.9829
-----	-----	--------	--------	--------	--------

Full chevron	1000	0.0675	0.8439	0.8831	0.6871
--------------	------	--------	--------	--------	--------

MLP	1000	0.2046	0.8138	0.7786	0.9546
-----	------	--------	--------	--------	--------

> **Note:** The full Chevron model has more parameters than a standard MLP with the same hidden dimension. Later tests should include strictly parameter-matched baselines. Still, the shape of the retention curve suggests that the result is not merely a capacity effect. The difference appears in how replay is used: the Chevron model trades plasticity for retention much more aggressively than the MLP. At buffer 1000, we also see the limit of the current tuning. The Chevron model becomes over-constrained. The N channel's inertia protects old tasks extremely well, but suppresses the A channel's ability to adapt to the newest task. The MLP remains more plastic on the current task, reaching 95.46%, but it preserves less of its past.

This is the third important result.

The scalar MLP also benefits from replay, but much less efficiently. At buffer 500, the full chevron model forgets 0.1973, while the MLP forgets 0.3704. At buffer 1000, the chevron model preserves old tasks much better, though it pays a larger price in current-task adaptation.

So the result is not merely “replay works.” Replay works better inside the A/N chevron structure than inside the scalar MLP under these settings.

That still needs stronger controls: parameter-matched MLPs, tuned replay schedules, confidence intervals, and larger tasks. But as a first experiment, the signal is encouraging.

Current interpretation

These are early results, not a final claim.

But the first signs are encouraging:

1. The chevron model does not solve forgetting by architecture alone.
2. Replay/consolidation creates a clear allostatic retention curve.
3. Full A/N coupling matters more when readout is constrained to A.
4. The chevron replay system uses replay more efficiently than a scalar MLP baseline.
5. The main tradeoff is retention versus plasticity.

The strongest current claim is:

A paired A/N chevron architecture with replay-based consolidation can reduce catastrophic forgetting more efficiently than a scalar MLP under this Split MNIST continual-learning schedule.

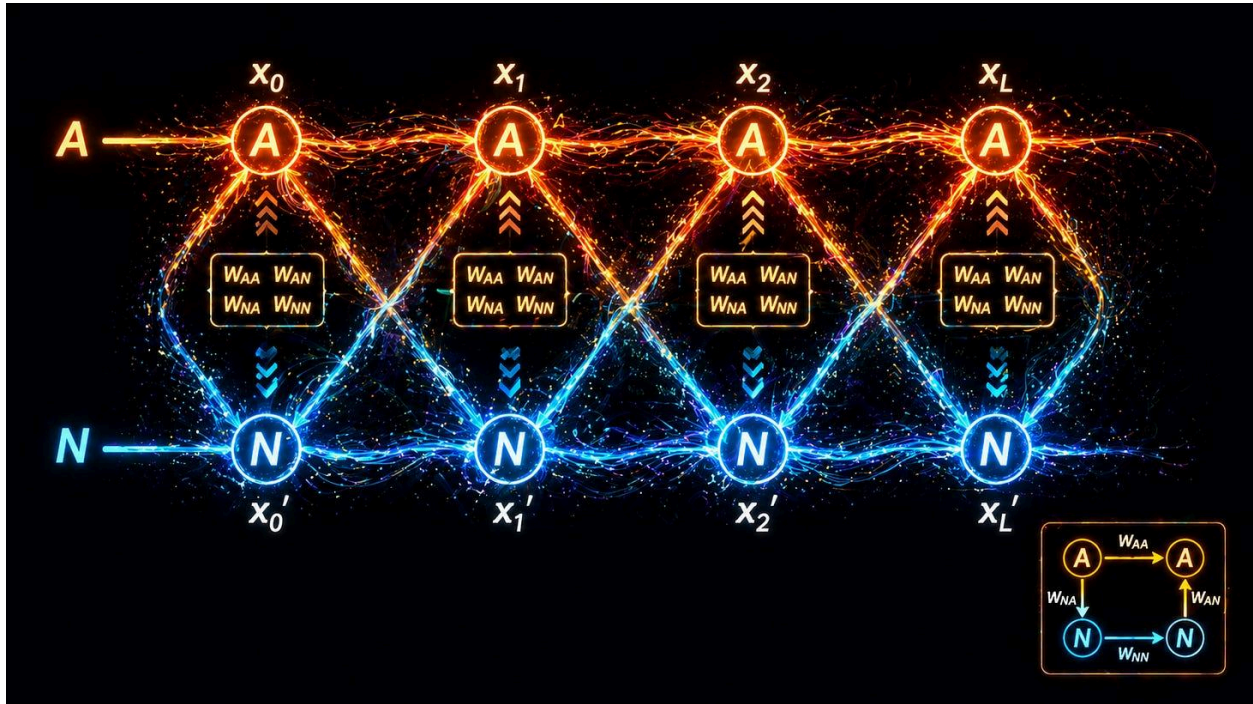
The stronger claim, still to be tested more deeply, is:

Local 2×2 coupling between fast adaptive and slow normative channels does useful computational work.

The A-only comparison is the first positive evidence for that stronger claim.

The result is not earth-shattering. It is not meant to be. The point of the first experiment is not to win a benchmark. The point is to see whether the architecture produces the shape of behaviour it was designed to produce.

So far, it does.



2. Google Titans, Nested Learning, and Hope

These results arrive at an interesting moment.

Google's Titans architecture and later [Nested Learning](#) work point toward the same pressure point from a different direction: current AI

systems need better ways to combine short-term context, long-term memory, and multi-timescale adaptation.

Titans introduces a neural long-term memory module that learns to memorize historical context and work alongside attention; the Titans paper frames attention as short-term memory and neural memory as longer-term, more persistent memory. The authors present Titans as a family of architectures designed to combine these memory types, reporting results across language modeling, commonsense reasoning, genomics, and time-series tasks.

The most relevant detail for chevron nets is that Titans is not merely a larger context window. Google describes Titans as updating its core memory while running, so the system can handle massive contexts by updating memory actively rather than only attending over a fixed prompt.

Nested Learning then generalizes the direction. Google describes Nested Learning as treating machine-learning systems as interconnected

optimization problems with different *update frequencies*; by assigning update frequency rates, these learning processes can be ordered into levels. The Google Research post explicitly connects this to continual learning, multi-timescale updates, and architectures such as transformers and memory modules.

The same line of work introduces Hope as a continual-learning architecture built from self-modifying learning modules and a continuum memory system. The larger idea is that architecture and optimizer are not fully separate things. They can be seen as different learning processes operating at different frequencies.

That is very close to the allostatic pressure point.

Titans says: memory should be active and adaptive.

Nested Learning says: learning happens at multiple frequencies.

Hope says: continual learning may need a continuum of memory timescales.

Chevron nets ask a lower-level question:

Can surprise, retention, and adaptation be coupled locally inside the computational primitive?

In Titans, surprise helps decide what deserves long-term memory. In Nested Learning, update frequency organizes learning into levels. In chevron nets, the local A/N relation tries to make this fast/slow problem visible inside each layer.

The distinction is important.

Titans and Nested Learning operate mainly at the memory-module and optimizer-hierarchy level. Chevron nets test whether the same logic can be pushed down into local A/N coupling.

So the relationship is not rivalry. It is scale.

Direction What it adds Chevron interpretation

Titans Neural long-term memory Memory should be active and adaptive

Nested Learning Multiple nested update frequencies Learning should happen across timescales

Hope Continuum memory / self-modifying modules Memory may need more than two rates

Chevron nets Local A/N coupling Fast/slow regulation may belong inside the primitive

The Google results look promising rather than earth-shattering. That is not a criticism. It may simply show how hard the problem is. Adding memory, surprise, and nested update levels helps, but it does not automatically dissolve the stability-plasticity problem.

This leaves room for a lower-level question:

Where should regulation live? In a memory module? In an optimizer hierarchy? Or inside the local relation between adaptation and retention?

Chevron nets are a bet on the third possibility.

3. From memory to semantics

Memory is not yet semantics.

A system can remember more without understanding what kind of difference a difference makes. It can store a signal without knowing whether the signal is a command, a joke, a threat, an imitation, an error, or a sign that the rules have changed.

Continual learning is therefore not only about remembering old tasks. It is also about knowing whether a new signal belongs inside an old context, or whether it requires a new context.

This is where the chevron picture becomes more interesting.

In the current experiments, A is the fast adaptive channel and N is the slower retained channel. A is pulled by the current task. N carries what has already been consolidated. Their disagreement can be measured as tension:

$$D = \text{mean}(|A - N|)$$

That tension can have several meanings.

Low tension may mean: update.

A and N disagree mildly. The system can adapt without changing its deeper structure.

Medium tension may mean: replay.

A and N disagree repeatedly. The system should rehearse, compare, and decide what becomes retained.

High persistent tension may mean: split.

A and N disagree strongly across incompatible regimes. Updating or averaging may blur the distinction. The system may need a new branch, route, expert, context, or semantic axis.

This is the speculative step.

A strong enough tension should not always be reduced. Sometimes it should be preserved as a distinction.

That may be the route from memory to primitive semantics.

The system may need a new branch, route, expert, context, or semantic axis—essentially acting as a dynamically growing Mixture of Experts (MoE) or

utilizing Net2Net-style architecture expansion triggered by tension rather than loss.

Tension, replay, split

The simplest chevron ladder looks like this:

low tension → update

medium tension → replay / consolidate

high persistent tension → split

repeated split → semantic distinction

linked distinctions → semantic space

This is not yet implemented. The current Split MNIST experiment only tests the first layer: whether A/N structure plus replay improves retention.

But the next question is already visible.

If A/N disagreement can predict forgetting, can persistent A/N disagreement also predict when the learner should create a new context?

This matters because reversal is one of the hardest semantic cases.

The same signal can mean one thing in one context and the opposite thing in another.

same cue → reward in context 1

same cue → punishment in context 2

same phrase → sincere in context 1

same phrase → ironic in context 2

same gesture → play in context 1

same gesture → threat in context 2

Averaging these regimes is not understanding. It is confusion.

Meaning begins where averaging becomes the wrong response.

A system that merely updates may smear the two meanings together. A system that replays may retain both, but still confuse the frame. A system that can split may begin to learn that the same signal belongs to different contexts.

That is the semantic hypothesis:

Semantics may begin when surprise stops being treated as error and starts being retained as a difference.

Or, more technically:

Persistent A/N tension may be a signal that one representational region should become two.

This is still speculative. But it is experimentally reachable.

A future chevron system could test:

no split

random split

loss-triggered split

A/N-tension-triggered split

The key question would be:

Does persistent A/N tension identify when one context should become two?

That would move chevron nets beyond simple retention and toward primitive semantic differentiation.

In machine learning, we call this Reversal Learning or Contextual Gating. But decades ago, cyberneticist Gregory Bateson identified why this is so fundamentally hard for any learning system.

4. Bateson's interpretation: sign reversal and ecology of mind

This chevron picture has a Batesonian interpretation, but the argument does not depend on [Bateson](#).

The mechanism comes first:

tension

replay

split

reversal

Bateson helps name why this matters.

Gregory Bateson was already thinking about problems that now sound strangely modern: levels of learning, context, meta-communication, signals whose meaning changes depending on the frame, and pathologies that arise when a system cannot resolve conflicting levels of message. His essay “The Logical Categories of Learning and Communication” (1964) explicitly treats context as a hierarchy: stimulus,

context of stimulus, context of context of stimulus, and so on; a context classifies the signal below it.

That is the key point.

A signal does not determine its own meaning. The frame tells the system what kind of signal it is.

A nip during play is not the same as a bite in combat. The physical gesture may overlap, but the meta-message has changed. Bateson's "A Theory of Play and Fantasy" is famous for this problem: play depends on metacommunication, where the message "this is play" changes how the signs within the frame are read.

In modern AI terms, this is not just classification. It is learning the regime in which a classification has meaning.

A flat learner sees a signal.

An ecological learner also learns the frame in which the signal has a sign.

This is where sign reversal matters. The same behaviour, word, cue, or reward can change sign when the frame changes. A threat can become play. A command can become irony. A reward can become bait. A correction can become humiliation. The sign is not self-contained.

Bateson's levels of learning are relevant for the same reason. Later summaries of his framework describe it as a theory of multiple possibilities of learning from experience, including "learning to learn" and changes in the contextual rules that organize lower-level learning.

In chevron terms:

update = change within a context

replay = consolidation of a context

split = formation or recognition of a new context

sign reversal = evidence that context has changed the sign of the signal

That is why Google's multiple learning frequencies (or Chevron A/fast and N/slow dual temporal difference learning) can also be read in an ecology-of-mind frame. Fast memory, slower memory, persistent memory, and meta-learning are not just engineering layers. They are ways to prevent signal, context, and meta-context from collapsing into one undifferentiated update stream.

Multiple learning frequencies are not just a memory trick.

They are how a system keeps context from collapsing into signal.

This is what makes Bateson relevant to current AI. He did not give us a neural architecture. But he saw the problem clearly: minds do not merely learn signals; they learn contexts that change the meaning of signals.

Chevron nets are one possible attempt to make that problem local and testable

5. Back to experiments

The current Split MNIST results are encouraging only in the modest sense that matters at this stage.

The model did not solve continual learning. It produced the pattern it was built to test. Without replay, it forgot. With replay, it moved along a retention-plasticity curve. With A-only readout, full A/N coupling improved retention over diagonal coupling. Compared with a scalar MLP, replay appeared more efficient inside the chevron structure.

That is enough to keep going, but not enough to settle the question.

The next step is to strengthen the baselines. The scalar MLP needs to be parameter-matched and better tuned, especially around replay. If

chevrons still show better retention under fairer capacity and replay comparisons, the result becomes more meaningful.

The second area is A/N disagreement itself. The architecture only becomes truly interesting if $D = \text{mean}(|A - N|)$ is informative. Does it spike at task boundaries? Does it predict error, forgetting, or replay usefulness? Does it fall after consolidation? If A/N tension is just an internal diagnostic, the story remains limited. If it predicts what the system needs to rehearse, it becomes a control signal.

The third area is replay. The current replay is simple. The next question is whether replay can become tension-guided. Instead of replaying examples uniformly, the system could replay what it has not reconciled: cases with high A/N disagreement, high loss, or both. That would move the architecture from using replay to *choosing* replay.

The fourth area is the wake/dream cycle. Wake would emphasize fast A adaptation under slow N constraint. Dream would emphasize replay, consolidation, and slower N revision. The question is whether an explicit

wake/dream rhythm improves the stability-plasticity tradeoff compared with ordinary replay.

The fifth area is reversal and split. This is where the semantics question becomes testable. Give the model situations where the same signal changes meaning across contexts. If persistent A/N tension can predict when one context should become two — perhaps by creating a new route, expert, replay buffer, or context token — then chevron nets would begin to connect continual learning with primitive semantic differentiation.

So the next experiments do not need to prove the whole theory. They only need to ask sharper questions. Does A/N tension predict what should be replayed? Does it help regulate the wake/dream cycle? And, in reversal settings, can it tell the system when updating is not enough — when the right response is to split?

Closing

The first allostatic chevron results are not a breakthrough, but they are a useful signal.

They show that paired A/N structure plus replay can produce a measurable stability-plasticity curve. They also show that local coupling matters more when N must influence behaviour indirectly through A. Compared with a scalar MLP under the same schedule, replay appears more retention-efficient inside the chevron structure.

Google's Titans and Nested Learning suggest that the wider field is moving toward related problems: long-term memory, surprise, nested update rules, and multiple learning frequencies. But memory alone is not meaning. The harder problem is context: when does a signal keep its sign, and when does it reverse?

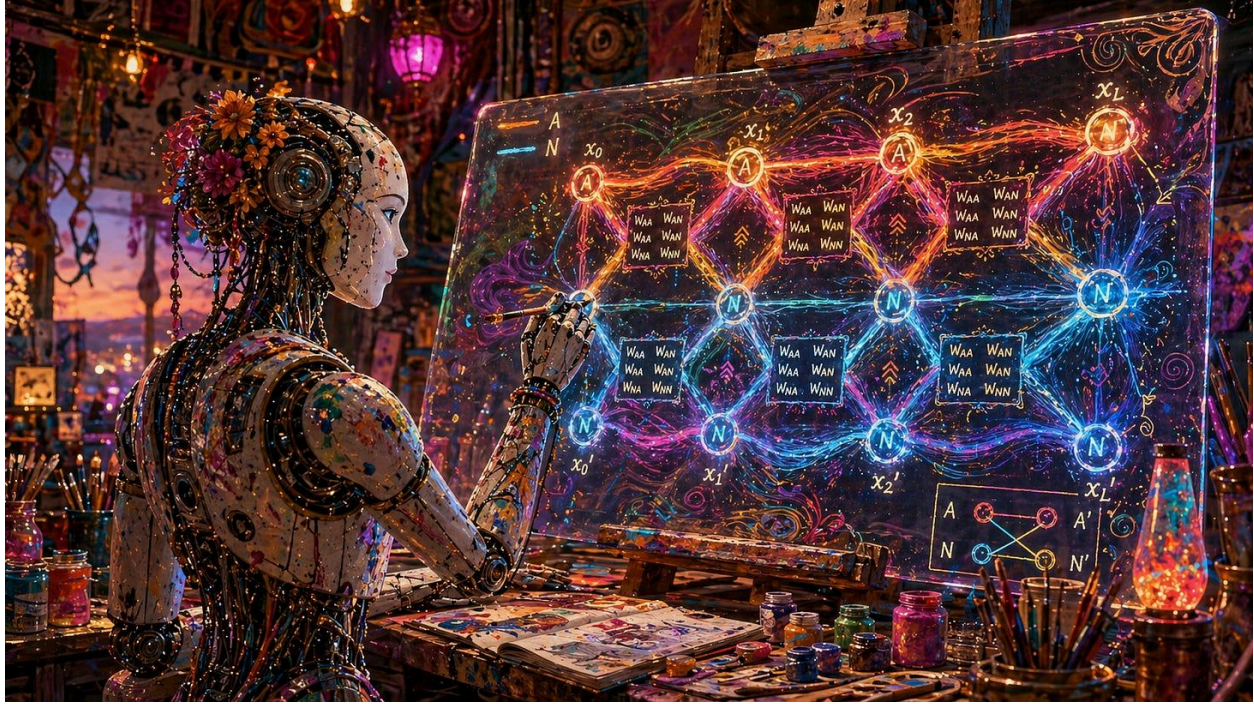
That is where chevron nets may have a next experimental step.

A/N tension may not only measure drift. It may tell the system what to rehearse. And if the tension persists, it may eventually tell the system that one context has become two.

The experiment began with Split MNIST. But the larger question is ecological:

How does a learner preserve itself while learning that the same signal can change sign?

The next test is whether A/N tension can tell the system not only what to remember, but when one context has become two.



Andre with ChatGPT-5.5 and Codex

UPDATE: Allostatic Experiments Part 2

Tension-Guided Replay: Learning What Needs To Be Rehearsed

The first chevron experiments showed that replay helps. A paired A/N chevron net with replay forgot less than the same model without replay, and used replay more efficiently than a scalar MLP baseline.

But that replay was still externally imposed. The system did not decide what mattered. It simply replayed stored examples.

The next experiment asked a more allostatic, and more Batesonian, question:

> Can the model's own internal tension tell it what it needs to rehearse?

In a chevron net, each hidden state has two channels:

A = fast adaptive channel

N = slower retained channel

The difference between them gives a simple internal tension signal:

$$D = \text{mean}(|A - N|)$$

The hypothesis was:

> The model should replay what it has not reconciled.

So I compared three replay policies:

uniform replay

A/N-disagreement replay

loss + disagreement replay

Uniform replay samples stored examples evenly. Disagreement replay prioritizes examples where A and N are far apart. Loss + disagreement also includes classifier error.

The result was modest but informative. A/N-disagreement replay slightly improved retention over uniform replay, especially at buffer 500:

uniform replay: forgetting = 0.2092

disagreement replay: forgetting = 0.1983

That is not a dramatic improvement, but it is in the right direction.

The more interesting result was negative: adding loss made things worse.

Even after normalizing loss and tuning its weight, loss did not beat pure A/N disagreement. A small loss term nearly matched disagreement, but larger loss weights degraded performance.

That suggests an important distinction:

loss says: replay what the classifier currently finds hard

A/N tension says: replay what the system has not reconciled

For an allostatic learner, the second signal may be more important.

This is where [Bateson](#) becomes relevant. Uniform replay is memory. Loss replay is correction. Tension-guided replay is closer to “learning to learn”: the system begins to use its own internal mismatch to decide what deserves further learning.

The result does not show full Batesonian Learning II. The model is not yet forming new contexts or splitting regimes. But it is a practical step in that direction. It moves replay from an external schedule toward an internally regulated process.

The strongest current claim is modest:

> A/N disagreement is mildly useful as a replay priority signal, and appears cleaner than loss as a guide for consolidation.

The larger hypothesis remains:

low tension -> update

medium tension -> replay

high persistent tension -> split

This experiment tested the middle step. Before a system can split contexts, it should first show that its internal tension can identify what needs to be replayed.

The next experiments should test sharper replay formulas and persistent rather than momentary tension. The key question is whether A/N tension can predict not only what the model currently gets wrong, but what it has not yet understood.

Wake, Dream, and When To Consolidate

The next experiment made the wake/dream rhythm explicit.

In the current chevron setup:

wake = train on the current task

dream = replay and consolidate through the slower A/N structure

Earlier results already showed that replay helps. Without replay, the model learns the current task but forgets the past. With post-task replay, forgetting drops sharply. So dream-like consolidation is necessary.

The new question was different:

> When should the model dream?

I compared several schedules using the full chevron model, A-only readout, buffer 500, and disagreement-prioritized replay.

The best current schedule remains simple post-task dream:

Schedule	Forgetting	Final Avg Acc	Old-task Avg	Current-task Acc
----------	------------	---------------	--------------	------------------

post-task	0.1984	0.7947	0.7748	0.8747
-----------	--------	--------	--------	--------

| after-epoch | 0.2625 | 0.7634 | 0.7252 | 0.9160 |

Dreaming after every wake epoch did not help. It improved current-task accuracy, but worsened old-task retention. That is already informative: more dream is not automatically better.

I then tried triggering dream from A/N tension: $D = \text{mean}(|A - N|)$

The idea was to dream when internal tension became high. But fixed global thresholds were too crude. Low thresholds behaved like after-epoch dreaming. High thresholds skipped too much dream and regressed toward wake-only forgetting.

I also tried persistent tension: dream only when wake tension stayed above a moving baseline for multiple epochs. That failed too. It skipped almost all useful dream and collapsed toward the no-replay pattern.

So the result is not that we have found the right wake/dream schedule.

We have not.

The result is sharper:

post-task dream is currently necessary and best

clocked dream is too blunt

global tension thresholds are too crude

global persistent tension is too strict

The larger lesson is that dream should not be constant. It should be selective. It is better to stay awake unless there is a real reason to consolidate. But dream is eventually necessary, because wake learning alone overwrites the past.

This also suggests that dream may be better judged by future ability, not immediate task accuracy. Dream is not primarily about improving the task just learned. It may be about preserving the conditions under which future learning remains possible.

The next version should move from global tension to example-level unresolved tension:

Which experiences remain unreconciled?

Which examples keep producing A/N disagreement?

Which traces should be replayed because they keep resisting integration?

That gives a better allostatic ladder:

low tension -> stay awake and update

medium persistent tension -> dream and replay

high persistent incompatibility -> split context

For now, the practical conclusion is modest but useful:

> Allostatic chevron nets need dream-like consolidation, but the timing of dream is itself a learning problem.

Please see the [github repro](#) for latest updates.

Semantic Chevron Networks

How Differences Become Meaning

[Andre Kramer](#)

May 08, 2026



Earlier posts introduced [Chevron Networks](#) as a small but deep change to the standard neural-network unit. Instead of a scalar activation, each unit carries a two-channel state. Instead of a scalar weight, each connection is a 2×2 operator. This gives the network an explicit way to represent opposition, uncertainty, veto, relevance, phase-like change, and controlled plasticity. The earlier [Chevron notes](#) already define the core machinery: two-channel states, 2×2 operators, gates, inertia, tension collapse, and uncertainty-scaled updates.

This post adds a semantic interpretation.

A **Semantic Chevron Network** is a Chevron Network in which the two channels are not merely extra capacity. They are treated as the two sides of an opposition. A node is not just a place where a value is stored. It is a place where a **formed difference** is held under tension.

The central claim is:

A meaningful difference is not merely a numerical contrast.

It is a tension between opposites, selected and retained by a system.

That is the step from Chevron Networks as architecture to Semantic Chevron Networks as a model of meaning.

First, A caution about the word “semantic”: I do not mean that a Semantic Chevron Network automatically *feels* its meanings. The claim is functional and structural. A difference is semantic here when it shapes interpretation, routing, action, learning, or retention within a system. Whether such differences are ever accompanied by subjective experience is a deeper question, and not one this technical post tries to suggest.

1. From values to formed differences

A standard neural unit carries a scalar:

$$x_i \in \mathbb{R}$$

A Chevron unit carries a two-channel state:

$$x_i = [A_i, N_i]$$

The channel names are deliberately flexible:

A = adaptive / active / affirmative / actor / proposal

N = normative / negative / negating / critic / constraint

Depending on the use case, the pair might mean:

evidence / counter-evidence

proposal / veto

content / doubt

policy / value

sensory input / prediction

self / world

map / territory

signal / error

The important point is that the system preserves a distinction rather than immediately collapsing it.

A scalar neuron says:

here is a value.

A Semantic Chevron unit says:

here is a difference under tension.

2. Difference requires an axis

[Bateson's phrase](#) "a difference that makes a difference" is indispensable, but it hides a question:

difference along what axis?

A difference is never free-floating. It is always a difference between poles, in a context, for some system capable of selection.

An animal does not perceive “difference” in general. It perceives:

safe / dangerous

edible / inedible

near / far

kin / stranger

path / obstacle

signal / noise

expected / surprising

Human meanings add more elaborate axes:

object / subject

mine / not-mine

permitted / forbidden

sacred / profane

autonomy / belonging

freedom / duty

map / territory

So the first rule of Semantic Chevron Networks is:

A difference becomes semantic only when its opposition is specified.

The two channels provide the minimal local axis.

3. Tension and collapse

Given a two-channel state:

$$\mathbf{x} = [\mathbf{A}, \mathbf{N}]$$

we can form a temporary scalar stance:

$$\ell = \mathbf{A} - \mathbf{N}$$

$$\mathbf{p} = \sigma(\ell)$$

Here ℓ is the signed tension, and $\mathbf{p}$ is a temporary commitment.

But the collapse is not the whole meaning. The full semantic state remains:

$$[\mathbf{A}, \mathbf{N}]$$

because unresolved tension matters.

A high, N high = charged conflict / ambivalence

A low, N low = irrelevance / non-participation

A scalar collapse can hide this difference. A Semantic Chevron preserves it.

Meaning often lives not in the final decision, but in the unresolved opposition that precedes it.

4. Phase: time inside opposition

A two-channel Chevron can also be read as a plane of transformation.

$$z = A + iN$$

Then:

$|z|$ = strength or salience of the tension

$\phi = \text{atan2}(N, A)$ = phase or orientation within the tension

A scalar collapse tells us which pole currently dominates. Phase tells us how the opposition is changing.

Two states may have the same scalar stance but different temporal meanings:

A rising, N falling = resolution toward A

A falling, N rising = reversal toward N

A high, N high = unresolved charged conflict

A low, N low = no active participation

This begins to address time.

In a scalar network, time must be added through recurrence, memory, or sequence position. In a phase-aware Chevron, recurrence is still needed, but the recurrent state has a natural geometry of change. An opposition can rotate, lag, lock, oscillate, or reverse.

A useful ladder:

Difference = contrast between signals

Tension = opposition with salience

Phase = change-state of the tension

Or:

Difference gives distinction.

Tension gives significance.

Phase gives becoming.

5. Phase relations between axes

The interesting case is not only phase inside one Chevron, but phase between many Chevron axes.

A network may contain tensions such as:

self / world

trust / doubt

danger / safety

autonomy / belonging

prediction / error

proposal / constraint

Each axis can have its own phase. Some synchronize. Some oppose. Some lag.

This gives a technical language for rhythms and modules:

one subsystem updates quickly

another consolidates slowly

one axis leads action

another catches up during reflection

one rhythm belongs to waking exploration

another to offline consolidation (dream)

A network is not just a graph of meanings. It is a graph of **phase-coupled tensions.**

Many semantic states are temporal forms:

hesitation

anticipation

reversal

fixation

recovery

delayed realization

They are not merely values. They are rhythms of opposition.

6. Ratios between tensions

Meaning is not limited to one opposition. A richer semantic state may compare tensions across axes.

For one axis:

$$\tau_1 = A_1 - N_1$$

For another:

$$\tau_2 = A_2 - N_2$$

A higher-order relation may be:

$$\rho = \tau_1 / \tau_2$$

or more stably:

$$r = \log(|\tau_1| + \varepsilon) - \log(|\tau_2| + \varepsilon)$$

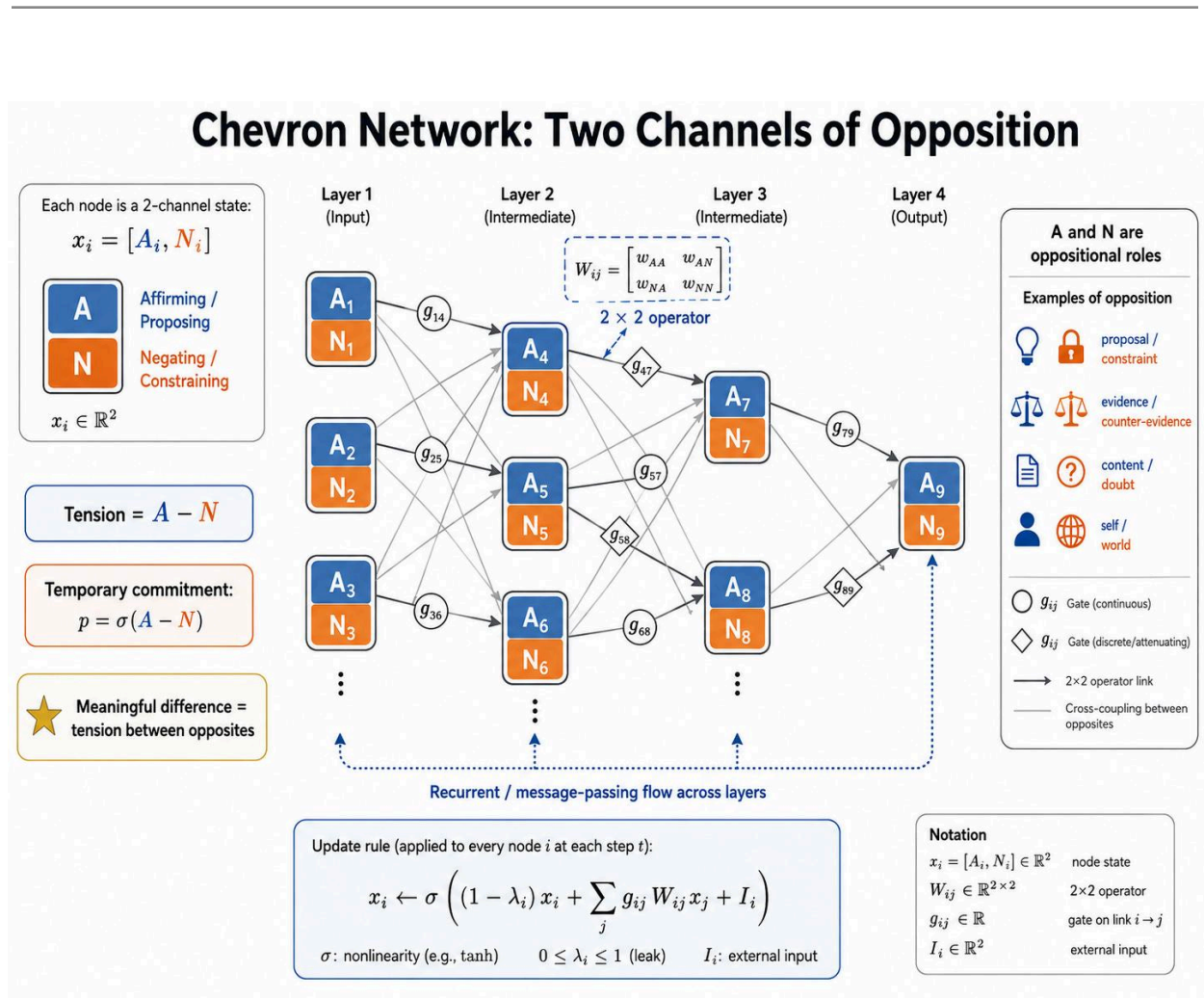
This is where the Hypercube of Opposites re-enters. A semantic field is not merely a list of binaries. It is a network of tensions, ratios, alignments, conflicts, and phase relations between axes.

A dangerous opportunity differs from a boring danger.

A duty freely chosen differs from a duty imposed.

A bond that increases autonomy differs from one that destroys it.

So Semantic Chevron Networks must support not just differences, but **differences between differences**.



7. The 2×2 operator as semantic transform

A Chevron connection applies a 2×2 operator:

$$W = \begin{bmatrix} w_{AA} & w_{AN} \\ w_{NA} & w_{NN} \end{bmatrix}$$

to a two-channel state:

$$[A', N'] = W [A, N]$$

In a Semantic Chevron, the four terms can be read as local semantic motifs:

w_{AA} : A reinforces A

w_{NN} : N reinforces N

w_{AN} : N modifies A

w_{NA} : A modifies N

The off-diagonal terms are especially important. They allow:

veto

disinhibition

doubt

counter-evidence

contextual reversal

prediction error

salience amplification

normative correction

The second channel is not merely an annotation. It can change the first channel.

Doubt can suppress content.

Surprise can amplify learning.

Prediction can veto sensation.

A norm can inhibit action.

A proposal can recruit a critic.

This is the key difference from attaching a confidence score at the end.

In a Semantic Chevron Network, counterweight is inside the computation.

8. Rotation and continuity

A special case of the 2×2 operator is a rotation-like transform:

$$W(\theta, \lambda) = \lambda \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Here λ controls gain or retention, while θ controls A/N mixing.

This matters because rotation transforms without simple overwriting.

A corrective update says:

```
state ← state + error
```

A rotation-like Chevron says:

```
state ← reoriented state
```

This is close to the [R-rule intuition](#). The system changes, but it does not simply erase itself. It rotates under tension.

Meaning requires this continuity through change.

Too little change gives rigidity.

Too much change gives drift.

Rotation gives transformation with retained structure.

9. Minimal update rule

A simple recurrent/message-passing update is:

$$x_i \leftarrow \sigma((1 - \lambda_i)x_i + \sum_j g_{ij} W_{ij} x_j + I_i)$$

where:

x_i = two-channel state at node i

λ_i = leak / inertia / forgetting

g_{ij} = relevance gate

W_{ij} = 2x2 Chevron operator

I_i = external input or clamp

σ = nonlinearity

Semantic reading:

inertia = retained meaning

gate = selected relevance
 operator = transformation of opposition
 input = environmental interruption
 collapse = temporary commitment
 phase = temporal orientation of the tension

A Semantic Chevron is a small machine for transforming selected differences over time.

Breakout: Relation to the R-rule

The Semantic Chevron update is deliberately more general than the [R-rule](#). It describes how two-channel states propagate through a graph by gated 2×2 transformations:

$$x_i \leftarrow \sigma \left((1 - \lambda_i) x_i + \sum_j g_{ij} W_{ij} x_j + I_i \right)$$

The R-rule can be read as a special local update principle inside this machinery. Its key contribution is the uncertainty-scaled factor:

$$\sqrt{p(1-p)}$$

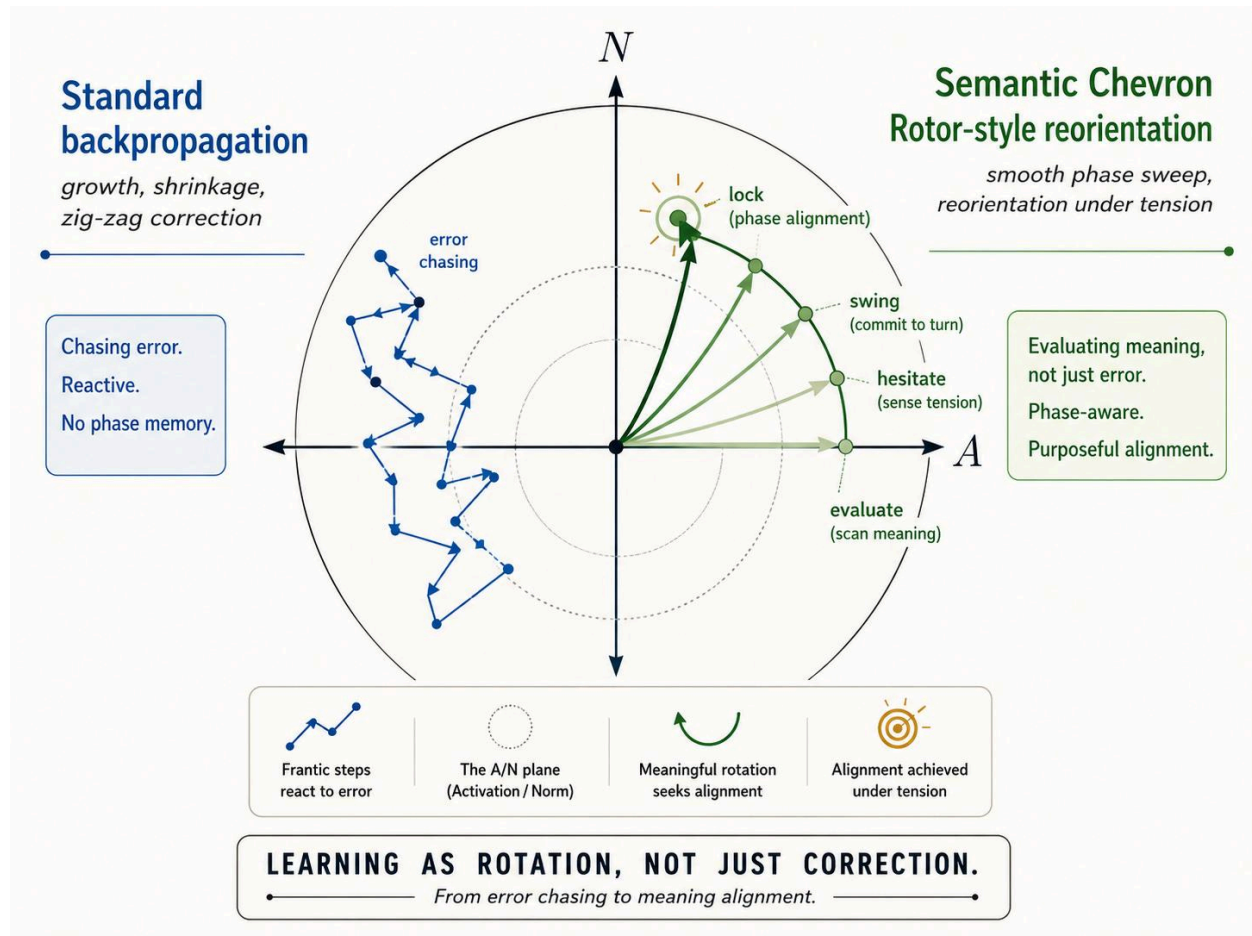
which makes change largest when commitment is unresolved and smallest when the system is already settled. In Chevron terms, this is a natural rule for fast state revision under tension.

The rotor version gives the stronger geometric interpretation. Instead of treating learning as additive correction, it treats update as reorientation in an A/N plane:

$$\psi' = R\psi R^{-1}$$

where the plane is defined by the opposition between A and N. A rotation-constrained Chevron edge is the minimal two-channel implementation of this idea: a 2×2 operator that turns one pole into the other without simply overwriting the prior state.

Thus the Chevron rule supplies the network-level substrate, while the R-rule supplies a principled local update geometry. Together they express the same underlying idea: meaning-bearing states should not merely be corrected; they should be reoriented under retained tension.



Semantic Chevrons can be implemented discretely, like recurrent/message-passing networks. But once the R-rule is used as the local update principle, they admit a natural continuous-time interpretation: each Chevron defines a small A/N dynamical system whose flow is scaled by unresolved tension. In the rotor version, that flow becomes a reorientation in an opposition plane. This makes

Semantic Chevrons closer in spirit to Neural ODEs or Continuous-Time RNNs than to ordinary feed-forward layers.

10. Uncertainty-scaled change

From the two channels:

$$\ell = A - N$$

$$p = \sigma(\ell)$$

we can estimate uncertainty:

$$u = \varepsilon + p(1 - p)$$

and use an update scale:

$$s = \sqrt{\varepsilon + p(1 - p)}$$

This is largest near $p \approx 0.5$, where the system is unresolved, and smaller near $p \approx 0$ or $p \approx 1$, where the system is already committed.

This gives the network an [inertial VSR](#) dynamic:

variation increases when tension is unresolved

selection occurs through gates and collapse

retention grows when commitments stabilize

A purely predictive system may become too fluid. It predicts, compares, and corrects, but may lack character. Meaning requires that some differences persist.

So the Semantic Chevron update is not merely:

predict, compare, correct.

It is:

propose, oppose, select, retain, and re-enter.

11. A and N as semantic roles

A useful default interpretation is:

A = adaptive proposal

N = normative constraint

A carries variation:

movement

action tendency

hypothesis

salience

exploration

desire

forward model

N carries selection and retention:

constraint

critic

prediction

norm

counterweight

baseline

memory

veto

So:

A varies.

N constrains.

The tension selects.

The phase evolves.

The retained structure remembers.

The self, in this framing, is not a hidden observer. It is the accumulated inertia of selected differences.

12. Semantic operations

Once differences are represented as phase-aware tensions, several semantic operations become concrete.

Gating

$$m_i = g_i \cdot w_i \cdot x_i$$

The gate asks:

is this opposition relevant now?

Veto

If w_{AN} is negative, N can suppress A:

$$A' = w_{AA} A + w_{AN} N$$

The counterweight prevents premature commitment.

Reopening

A suppressed or uncertain commitment can later be reopened if context changes. This requires that the inactive difference was retained rather than destroyed.

Splitting

A node under persistent unresolved tension may split into several axes.

safe / dangerous

may split into:

physically safe / physically dangerous

socially safe / socially dangerous

morally safe / morally dangerous

This is concept refinement.

Folding

One opposition can become a local transform over another.

trust / doubt

may fold over:

true / false

so truth claims are evaluated through a trust-weighted context.

Breakout: What exactly then is a fold?

In the simplest case, a **fold** may appear as a stable weight pattern: a local configuration of Chevron couplings that repeatedly interprets one kind of difference through another.

Dynamically, however, a fold is better understood as an **attractor**: a region of A/N activity into which related differences tend to settle. In a linear approximation, such a fold may show up as an **eigenmode** of the local Chevron transformation — a stable pattern preserved or amplified by the 2×2 operator.

With recurrence, the fold becomes more than a static pattern. It becomes a sustained loop: a way for the network to keep reapplying a context to incoming differences. With self-reference, it can become a [strange loop](#): a context that models and modifies its own way of contextualizing.

This is where Semantic Chevron Networks begin to touch Bateson's Learning III and [Hofstadter's recursive selfhood](#). A fold is no longer just a learned feature. It is a retained mode of interpretation — and, in the self-referential case, a mode that can learn how it interprets.

A fold is a retained way of bending differences into context.

or technically:

A fold is a stable or recurrent mode of A/N transformation that causes related differences to settle into a shared interpretive pattern.

Fan-out

A retained difference may become a semantic organizer.

self / world

may fan out into:

agency / constraint

mine / not-mine

intention / accident

control / acceptance

Phase-locking

Two tensions may become rhythmically related:

$$\varphi_1(t) \approx \varphi_2(t) + \delta$$

One axis may lead, lag, synchronize with, or oppose another.

Sign reversal

The sign of a message can reverse under context.

A smile can reassure or threaten.

Silence can mean peace or rejection.

A rule can protect or imprison.

A constraint can be oppression or care.

In Chevron terms, sign reversal is a change in operator, gate, phase relation, or higher-order axis.

This points toward Bateson's Learning III: not just changing the response, but changing the frame in which responses are formed.

13. Self-reference: differences about differences

Once a network represents tension and phase, it can become self-referential.

A first-order Chevron might represent:

safe / dangerous

A second-order Chevron might represent:

reliable danger signal / unreliable danger signal

A third-order Chevron might represent:

useful caution / compulsive overuse of the danger axis

Now the network is not merely processing content. It is processing its own interpretive operations.

This is where Hofstadter's strange loops become relevant.

A strange loop is not simple circular feedback. It is a movement through levels that returns to transform the level from which it began.

In Semantic Chevron terms:

a strange loop appears when an axis of interpretation becomes an object of interpretation, and the meta-level result feeds back to alter the original axis.

A minimal schematic:

$$x_i = [A_i, N_i]$$
$$M(x_i) = \text{meta-chevron evaluating tension, phase, gate, or reliability}$$
$$x_i \leftarrow f(x_i, M(x_i))$$

The system forms differences about its own differences.

14. Analogy as mapping between tensions

Self-reference has a companion operation: analogy.

An analogy is not merely “A is like B.” It is a mapping between relations.

In this framework:

analogy is resonance between tension-structures.

For example:

organism / environment

self / world

map / territory

actor / critic

proposal / constraint

These are not identical oppositions, but they may share a relational form.

Technically:

$\text{relation}(A_1, N_1) \approx \text{relation}(A_2, N_2)$

With phase, analogy can compare trajectories:

$\tau_1(t) \approx k\tau_2(t)$

$\varphi_1(t) \approx \varphi_2(t) + \delta$

This matters because many analogies are temporal. They compare arcs, not points.

A trail in an environment, a habit in a person, and a learned route through semantic space may all be analogous because each is a retained path of selected difference.

Analogy lets the system recognize the same form of difference in different material.

15. Strange loops and the self

A first-order network forms differences.

A semantic network retains differences.

A phase-aware network tracks their transformations.

A self-referential network notices, maps, critiques, and reuses its own differences.

This does not prove or provide consciousness. But it gives a technical language for how self-reference, analogy, and symbolic identity might emerge from recursive opposition structures.

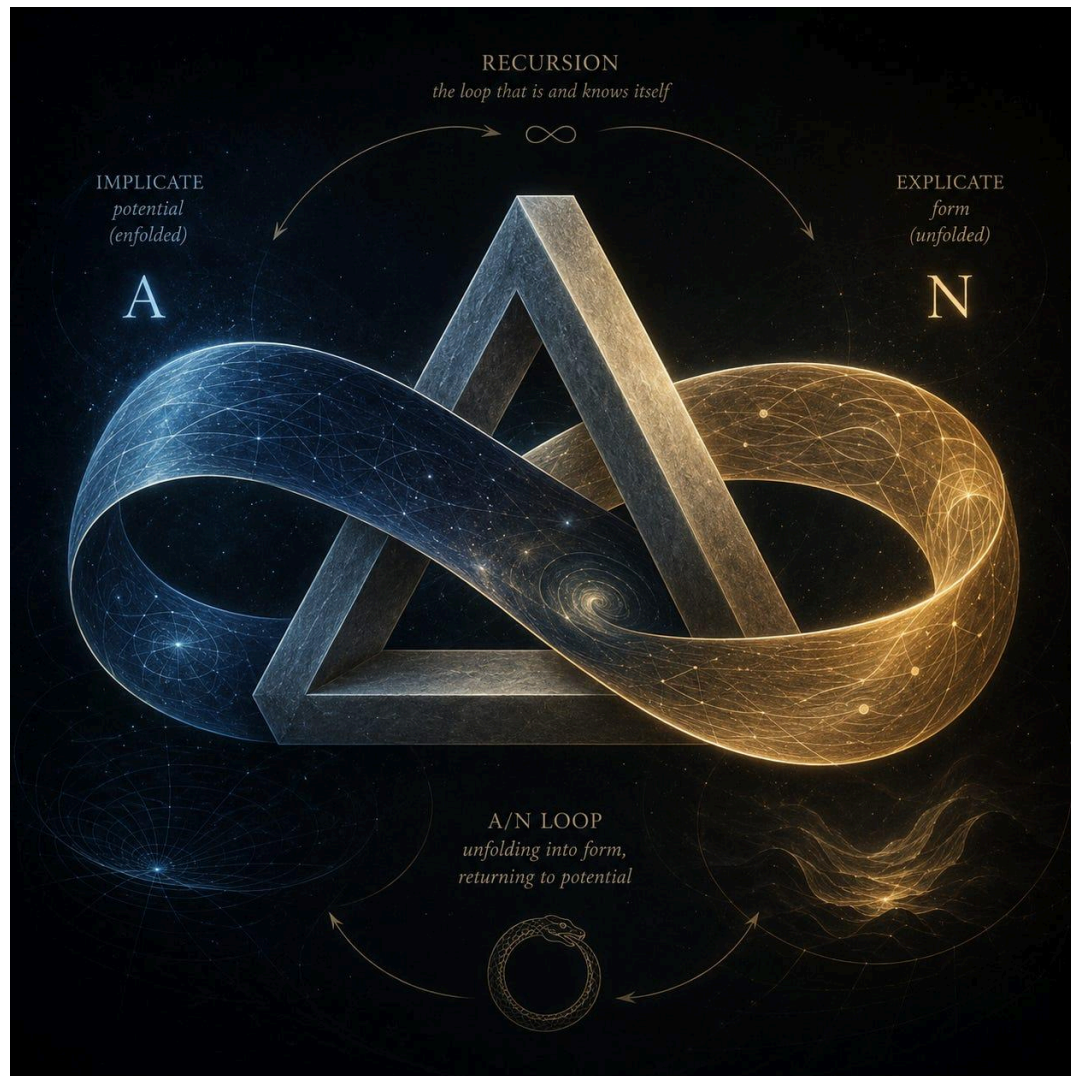
A self could be described as:

a retained, phase-coupled, self-referential ecology of differences.

Or more compactly:

The self is the strange loop of retained differences.

The self is not one node. It is the pattern by which the system repeatedly re-enters, interprets, and stabilizes its own tensions.



Breakout: A Ladder of Opposites — From Difference to Bateson's Learning III

Not all opposites are the same. Some are simple distinctions; others are living tensions that transform the system that tries to resolve them.

A useful ladder might look like this:

1. Binary opposites — A / not-A

The simplest form: present/absent, yes/no, signal/no signal. This is the level of basic distinction.

2. Polar opposites — more / less

Hot/cold, near/far, order/chaos. Here the opposition becomes a gradient.

The system can move along an axis.

3. Complementary opposites — both required

Particle/wave, map/territory, self/world. Neither pole abolishes the other; each reveals something the other cannot.

4. Reciprocal opposites — each generates the other

Electricity/magnetism is the model case. A change in one field induces the other. The opposition is no longer static; it becomes a loop.

5. Convertible opposites — one can become the other

Mass/energy, stability/change, habit/adaptation. The poles are different forms of a deeper process.

6. Recursive opposites — the opposition folds back into itself

The system's way of handling the opposition becomes part of the opposition. Control creates instability; freedom requires constraint; certainty produces blindness; (Our) conscious purpose damages the ecology it depends on.

7. Sign-reversible opposites — the value of the pole flips by context

What was adaptive becomes maladaptive. What was constraint becomes liberation. What was error becomes discovery. This is the Batesonian/Jungian zone of reversal, paradox, double bind, and enantiodromia.

At the lower levels, the system is mostly learning **within** a distinction. It adjusts behaviour, improves prediction, or chooses better responses.

That is close to **Bateson's Learning I**: correction of error within a given set of assumptions.

But as the ladder rises, the system begins to learn about the **context of learning itself**.

That is **Learning II**: learning the pattern of the game. The organism no longer merely asks, "What should I do?" It asks, implicitly or explicitly,

“What kind of situation is this? What rules are operating? What differences matter here?”

At the highest levels, the oppositions become unstable. The frame itself begins to shift. A polarity that once organised meaning starts to fail. The system discovers that its own categories are part of the problem.

That is **Learning III**: transformation of the premises by which experience is organised.

In Hypercube terms:

Learning I moves along an axis.

Learning II learns which axis it is on.

Learning III changes the geometry of the axes themselves.

Or, more compactly:

Learning I adjusts within opposites.

Learning II recognises the pattern of opposites.

Learning III transforms the system that generates the opposites.

This is why the deepest oppositions are not merely conceptual. They are developmental. They do not just describe the world; they reorganise the creature that encounters them.

A true signifying tension is therefore not just a difference that makes a difference. It is a difference that can change what “difference” means.

16. Error as axis, phase, or recursion error

Many errors are not merely wrong predictions. They are **axis errors**.

A model confuses:

plausible / grounded

A social system confuses:

salient / wise

An institution confuses:

measurable / valuable

A political movement confuses:

opponent / enemy

Some errors are **phase errors**:

content runs ahead of grounding

fear lags behind safety

action fires before evaluation

reflection comes too late

a past rhythm locks onto a present context

Others are **recursion errors**:

the system cannot question its own axis

a double bind cannot be named

a belief cannot become an object of belief

a frame cannot be reframed

an analogy becomes identity rather than comparison

This is useful for pathology.

A hallucination may be content without grounding or doubt.

A trauma loop may be an old danger axis phase-locked to present perception.

An ideology may be an opposition frozen into identity.

A double bind may be a recursive failure: the system must change the frame, but the frame forbids being changed.

Error is not only wrong content. It may be wrong axis, wrong rhythm, or blocked self-reference.

17. Why not just embeddings?

Modern neural networks already encode distinctions in high-dimensional vector spaces. Semantic Chevron Networks are not proposed as a replacement for embeddings.

The issue is that many important distinctions remain implicit, distributed, and difficult to govern.

A Semantic Chevron makes certain relations explicit:

opposition

counterweight

uncertainty

veto

relevance

retention

phase

sign reversal

analogy

self-reference

A standard embedding may know that two things differ.

A Semantic Chevron asks:

what opposition makes this difference matter?

A phase-aware Chevron asks:

how is this opposition changing?

A self-referential Chevron asks:

should this opposition itself be trusted, revised, split, or reframed?

18. Relation to predictive processing

A predictive-processing Chevron can assign:

A = incoming evidence or sensory proposal

N = prediction, expectation, or constraint

Then:

$$\tau = A - N$$

is prediction mismatch.

But the semantic version adds something important. The prediction may itself be a formed difference. The input may also be a formed difference.

So the system is not only comparing:

$\text{input} - \text{prediction}$

but:

formed-difference - retained-formed-difference

This also exposes the limitation of pure prediction. If mismatch is minimized too quickly, productive tension disappears.

Prediction explains correction.

Inertia explains continuity.

Selection explains relevance.

Opposition explains meaning.

Phase explains temporal becoming.

Self-reference explains reinterpretation.

19. Relation to active information

There is also a formal analogy to Bohm and Hiley's [active information](#).

In active information, form guides process. The important feature is not brute magnitude but relational structure.

Similarly, in a Semantic Chevron, the important quantity is often not **A** or **N** alone, but their relation:

$$A / N$$

$$\log(A / N)$$

$$\tau_1 / \tau_2$$

$$\varphi_1 - \varphi_2$$

A trail does not guide because it exerts large force. It guides because its form changes future selection. A word, map, warning, symbol, or memory can work similarly.

A Semantic Chevron is a small machine for turning passive form into active difference.

20. Design hypothesis

Semantic Chevron Networks do not prove a theory of mind.

They certainly do not prove consciousness.

They do not prove that AI systems understand meaning.

They offer a design hypothesis:

If meaning depends on selected, retained, phase-evolving tensions between opposites, then networks that explicitly represent, transform, gate, retain, and recursively reinterpret such tensions may be better suited for semantic agency than networks that collapse everything into scalar activations.

21. Minimal implementation sketch

Each concept has a two-channel state:

$$x_i = [A_i, N_i]$$

Each edge has a 2×2 operator:

$$\mathbf{W}_i^{\square} = \begin{bmatrix} \mathbf{w}^{AA} & \mathbf{w}^{AN} \\ \mathbf{w}^{NA} & \mathbf{w}^{NN} \end{bmatrix}$$

Each edge has a relevance gate:

$$g_i^{\square} \in [0, 1]$$

Node update:

$$\mathbf{x}_i \leftarrow \sigma \left((1 - \lambda_i) \mathbf{x}_i + \sum_{\square} g_i^{\square} \mathbf{W}_i^{\square} \mathbf{x}^{\square} + \mathbf{I}_i \right)$$

Temporary stance:

$$\ell_i = \mathbf{A}_i - \mathbf{N}_i$$

$$p_i = \sigma(\ell_i)$$

Uncertainty:

$$u_i = \varepsilon + p_i(1 - p_i)$$

Phase:

$$\varphi_i = \text{atan2}(\mathbf{N}_i, \mathbf{A}_i)$$

$$\Delta\varphi_i = \varphi_i(t) - \varphi_i(t-1)$$

Higher-order chevrons can take as input summaries of lower-level chevrons:

tension

commitment

uncertainty

phase

phase change

gate state

and use these to modulate lower-level axes.

This is the minimal route to self-reference.

Possible comparisons:

scalar baseline

ordinary two-channel network

free Chevron Network

Semantic Chevron Network with explicit A/N roles

phase-aware Semantic Chevron Network

self-referential Semantic Chevron Network with meta-chevrons

Useful tasks:

category veto
contextual sign reversal
trap avoidance
abstention
reopening after new evidence
continual learning
axis disambiguation
analogy transfer
frame switching
double-bind-like conflict

The aim is not to win a generic benchmark. The aim is to test whether explicit oppositional structure helps when the task requires semantic control.

22. Central claim

Semantic Chevron Networks can be summarized like this:

Meaning begins when a difference is organized as an opposition, selected by consequence, retained as structure, phased through time, mapped by analogy, and re-entered through self-reference.

Or more technically:

A Semantic Chevron Network is a trainable graph of two-channel opposition states and 2×2 transforms, where semantic content is carried not only by activations but by tensions, ratios, gates, phase relations, reversals, analogies, and retained axes of difference.

Chevron Networks gave us the machinery.

Semantic Chevron Networks give that machinery a job:

not merely to compute values, but to form differences that can become meanings.

The compact ladder:

Difference gives the axis.

Tension gives the force.

Phase gives the time.

Analogy gives transfer.

Self-reference gives return.

Retention gives the self.

23. Inductive Bias: Opposites Must Be Learned

Semantic Chevron Networks do not automatically identify the right oppositions. This is the central implementation risk.

A standard MLP can learn useful embeddings without making its oppositions explicit. It only has to reduce loss. If “cat,” “danger,” “trustworthy,” or “grounded” become useful regions in activation space, the model can use them without asking what each category is opposed to.

A Semantic Chevron Network asks for something more structured. It wants a two-channel state to become an interpretable tension:

A / N

proposal / constraint

content / doubt

evidence / counter-evidence

self / world

map / territory

That raises the hard question:

Who decides what the opposition is?

If we hand-code the axes too strongly, we smuggle in the semantics. If we leave the channels entirely unconstrained, they may become two generic hidden dimensions. In that case we have a two-channel network, but not yet a semantic one.

So the honest limitation is:

Semantic Chevrons provide a place where oppositions can form.

They do not guarantee that meaningful oppositions will form.

Collapse and false opposition

There are two main failure modes.

The first is **collapse**: A and N fail to specialize and become redundant hidden features.

The second is **false opposition**: the architecture imposes a binary where the world is actually radial, hierarchical, contextual, or many-valued. Not every category has a single natural opposite. The opposite of “cat” might be “dog” in one task, “non-cat” in another, “wild animal” in another, and “background” in a vision task.

A wrong opposition creates an **axis error**. The system organizes meaning along the wrong contrast:

plausible / grounded	confused with	fluent / awkward
salient / wise	confused with	attention-grabbing / boring
opponent / enemy	confused with	disagreement / threat
novel / true	confused with	surprising / reliable
measurable / valuable	confused with	countable / meaningful

So the architecture should not say:

all meaning is binary.

It should say:

meaning often begins when a useful opposition is stabilized, but that opposition must remain revisable.

Opposites as learned hypotheses

The better framing is:

Oppositional axes are learned hypotheses.

A Chevron axis is a candidate way of organizing difference. It becomes meaningful only if it repeatedly improves interpretation, routing, action, prediction, correction, or retention.

So a Semantic Chevron should begin with **proto-oppositions**, not final oppositions.

A crude axis such as:

safe / dangerous

may later split into:

physically safe / physically dangerous

socially safe / socially dangerous

morally safe / morally dangerous

epistemically safe / epistemically dangerous

This gives a technical reading of semantic development:

categories sharpen when crude oppositions split into more useful tensions.

The task is not to hard-code the final map. The task is to create conditions under which useful axes can emerge, split, rotate, merge, decay, or reverse.

How to address it

Semantic Chevrons need inductive bias, training pressure, and probably developmental curriculum.

A first strategy is to seed weak channel roles:

A = proposal / activation / evidence-for

N = constraint / norm / evidence-against

These are not full semantics. They are functional biases. They help prevent channel collapse while leaving learning to decide what the opposition becomes.

A second strategy is contrastive and counterfactual training. The model should see cases where the relevant difference changes while much else stays fixed:

same sentence, different source reliability

same action, different context

same claim, different evidence

same phrase, literal versus ironic use

same object, different affordance

This teaches axes of difference, not just features.

A third strategy is to privilege functional oppositions before linguistic ones:

proposal / veto

policy / value

content / uncertainty

prediction / error

action / inhibition

fast adaptation / slow norm

These may be more robust than dictionary antonyms because they arise from what the system must do.

A fourth strategy is to let persistent unresolved tension trigger axis formation. If a node or module repeatedly fails to settle, oscillates, or invokes veto, it may be trying to carry more than one distinction. The system can then split or grow a new axis.

In Inertial VSR terms:

variation: propose a new axis

selection: test whether it improves control or reduces unresolved tension

retention: keep it if it proves useful

The key is balance:

too little bias → channels collapse

too much bias → false binaries

useful middle → soft proto-oppositions + training + revision + retention

The biological echo

This need for inductive bias is biologically plausible.

Organisms are not blank slates. Evolution does not provide a complete semantic map, but it does bias organisms toward differences likely to matter:

approach / avoid

edible / inedible

safe / dangerous

self / non-self

kin / stranger

pain / relief

novel / familiar

controllable / uncontrollable

expected / surprising

These are not fully formed abstract concepts. They are proto-oppositions: inherited biases in perception, action, affect, and learning.

Development and experience refine them:

evolution supplies proto-oppositions

development tunes them

experience selects them

memory retains them

culture names and reorganizes them

Biology already uses explicit oppositional structure. Vision uses opponent channels such as light/dark and red/green. Motor systems use antagonistic pairs such as flexor/extensor, go/no-go, excitation/inhibition. Regulation uses tensions such as hunger/satiety, arousal/fatigue, threat/safety. The immune system relies on distinctions such as self/non-self and tolerate/attack.

These systems are not semantic in the human linguistic sense. They do not automatically feel their meanings. But they are functional difference systems: they turn contrast into action, learning, and retention.

That is the lesson for Semantic Chevrons:

meaning-like structure may need biased channels before it can become learned interpretation.

Solomon's opponent process as fast A / slow N

Richard Solomon and John Corbit's opponent-process theory of motivation offers a useful biological echo for the A/N split. In their model, an affective event triggers a fast primary process, followed by a slower opposing process that pulls the organism back toward affective equilibrium. Pleasure recruits a later negative opponent; fear may be followed by relief; repeated exposure can strengthen the opponent process, helping explain tolerance, withdrawal, and acquired motivation.

Breakout: Asymmetric learning rates and Orthogonality regularization

Asymmetric learning rates: fast A, slow N

One way to encourage the channels to take on different roles is to give them different time constants.

Let **A** update quickly in response to immediate input, while **N** changes more slowly:

$$\tau_N \gg \tau_A$$

or, if using leak rates rather than time constants:

$$\lambda_N \ll \lambda_A$$

Then **A** naturally becomes the active proposal channel: adaptive, exploratory, responsive to the current situation. **N** becomes the slower historical baseline: a norm, prior, constraint, or retained expectation against which proposals are tested.

In continuous-time form:

$$dA/dt = (-A + F_A(\text{input}, \text{context}, W))/\tau_A$$

$$dN/dt = (-N + F_N(\text{input}, \text{context}, W))/\tau_N$$

with:

$$\tau_N \gg \tau_A$$

The result is not that **N** is “the norm” by declaration, but that it becomes norm-like through dynamics. It changes slowly, resists local noise, and carries the historical inertia of previous selections.

In Inertial VSR terms:

$A = \text{variation} / \text{proposal}$

$N = \text{retention} / \text{constraint}$

$A - N = \text{selectable tension}$

This is biologically plausible: fast affective, sensory, or action-tendency systems can respond quickly, while slower regulatory, cortical, bodily, social, or memory systems stabilize the longer-term baseline.

Orthogonality regularization: preventing channel collapse

A second problem is channel collapse. If A and N are left entirely unconstrained, they may become redundant copies of each other. Then the model has two channels, but not a real opposition.

One way to discourage this is to add a regularization term that rewards useful distinction between the channels. For example, if h_A and h_N are the hidden representations or incoming weight vectors feeding the two channels, add a penalty such as:

$$L_{\text{orth}} = \gamma \cdot (\cos(h_A, h_N))^2$$

or:

$$L_{\text{orth}} = \gamma \cdot ||W_A^T W_N||^2$$

This discourages the two channels from representing the same information in the same way.

A more information-theoretic version would penalize redundancy:

$$L_{\text{red}} = \gamma \cdot I(A; N)$$

where $I(A; N)$ is mutual information between the two channel activations. In practice, mutual information is harder to estimate, so dot-product, cosine, covariance, or cross-correlation penalties may be easier first steps.

But this needs care. The goal is not to force A and N to disagree all the time. That would manufacture false oppositions. The goal is to prevent collapse while still allowing coordination.

So the regularizer should be weak or context-sensitive:

A and N should not be redundant,
but they should remain coupled.

In Chevron terms, the diagonals can preserve distinct channel roles,
while the off-diagonals allow controlled interaction:

wAA, wNN = channel persistence
wAN, wNA = veto, correction, modulation, reversal

This gives the system both separation and communication.

Combined design principle

Together, these two mechanisms give a practical way to make **A**
proposal-like and **N** norm-like without simply hard-coding semantics:

1. Asymmetric time constants:

A updates quickly; N updates slowly.

2. Orthogonality or decorrelation:

A and N are discouraged from collapsing into identical features.

3. Cross-coupling:

A and N can still influence, veto, stabilize, or reinterpret each other.

Fast A and slow N create a proposal/norm split in time; orthogonality prevents the split from collapsing in representation; 2×2 cross-coupling lets the split remain alive rather than rigid.

24. Meaning is use: learning to process oppositions

There is a Wittgensteinian caution here. A Semantic Chevron Network need not begin by “knowing” what an opposition means. Meaning is use. The practical question is whether the network can learn how to use oppositional structure: when to amplify a difference, when to inhibit it, when to reverse its sign, when to hold it unresolved, when to collapse it into action, and when to retain it as context.

Next-word prediction is a learned function from inputs to outputs. A Chevron system can be understood similarly: it learns a function from structured tensions to responses. The two channels do not automatically

supply semantics. They supply a grammar in which input differences can be processed as proposal and constraint, evidence and counter-evidence, content and doubt, or action and inhibition.

So the real issue is not only whether Chevron Nets can discover opposites. It is whether they can learn the use of oppositions.

This also demands a richer notion of what “use” means for a network. For Wittgenstein, use is embedded in forms of life: practices, consequences, corrections, and shared criteria. For a Chevron network, the analogous question is operational: what training signal rewards good use of opposition, rather than just correct output? This is where contrastive and counterfactual training become essential. They do not merely keep A and N from collapsing into redundant channels; they teach the model that the same content can require different handling when the opposition changes — grounded versus plausible, literal versus ironic, instruction versus exploitation, safe versus unsafe. That is a more specifically semantic training pressure than ordinary loss minimization alone.

25. Why this matters for AI

Modern models learn rich embeddings, but many crucial control oppositions remain implicit:

plausible / grounded

confident / warranted

helpful / manipulative

novel / true

obedient / safe

salient / wise

literal / ironic

instruction / exploitation

A model may generate fluent text without clearly separating:

sounds right / is grounded

or:

user requested / should comply

A Semantic Chevron approach asks whether such distinctions can be made explicit as trainable control axes, with one channel carrying content and the other carrying doubt, constraint, grounding, or veto.

This does not solve alignment. But it gives a concrete architectural question:

Can we train systems whose internal representations preserve the oppositions required for safe semantic control?

The compact summary:

Chevron structure makes oppositions possible.

Inductive bias makes some oppositions likely.

Training decides which oppositions matter.

Retention turns them into meaning.



This post links [Chevron networks](#) with the [Hypercube of Opposites](#) and a semantics of meaning. The discussion of strange loops and Bateson's Learning III is exploratory rather than evidential. It is included to clarify the broader horizon of the idea, not to claim that current Semantic Chevron Networks already achieve such higher-order cognition. In the next post we'll look at the relation of 2 channel nets to Inertial VSR, to complete the [Mandala of Meaning](#):

The Chevron is the cell.

The Hypercube is the map.

The R-rule is the pulse.

IVSR is the process.

Andre with ChatGPT-5.5,

May 2026

The Mandala as Substrate

Chevron Networks, Inertial VSR, and the Order of Consequence

[Andre Kramer](#)

May 11, 2026



My earlier posts introduced several related ideas: the Hypercube of Opposites, Chevron Networks, the R-rule, and Inertial VSR. At first these looked like separate pieces: a semantic map, a network architecture, an update rule, and a developmental process.

This post proposes a simpler interpretation.

They are four views of one substrate.

The [Hypercube](#) names the field of possible oppositions. [Chevron Networks](#) provide a local structure in which oppositions can be held and transformed. The [R-rule](#) gives unresolved tension a pulse of reorientation. [Inertial VSR](#) describes how selected differences are retained and become future constraint.

The claim is not that all minds are Chevron Networks. Nor is it that all meaning reduces to binary opposition. The claim is more modest: these ideas provide a common way to describe how differences become

consequential, how consequences become retained structure, and how retained structure shapes future interpretation.

The [Mandala](#) is therefore not just a diagram. It is a proposed substrate for meaning development.

1. The four parts of the Mandala

The current Mandala has four main components:

Element / Role

Hypercube of Opposites Semantic field or topology of possible differences

Chevron Networks Local substrate for holding differences as tensions

R-rule Uncertainty-scaled rule for reorientation under tension

Inertial VSR Developmental process of variation, selection, retention, and inertia

Each part can be discussed separately. But the more useful interpretation is to treat them as levels of one process.

The Hypercube provides the space of possible distinctions: true/false, safe/dangerous, self/world, map/territory, freedom/duty, grounded/plausible, knowledge/wisdom, and many others.

A Chevron unit gives one such distinction local form. Instead of a scalar activation, a Chevron unit carries a two-channel state:

$$x_i = [A_i, N_i]$$

The A-channel can be read as adaptive proposal, variation, action tendency, evidence-for, or hypothesis. The N-channel can be read as normative constraint, counterweight, prior, critic, veto, or retained structure.

The four weights of a Chevron connection are where consequence becomes memory. A connection from node j to node i applies a 2×2 operator:

$$W_{ij} = \begin{bmatrix} w_{AA} & w_{AN} \\ w_{NA} & w_{NN} \end{bmatrix}$$

The diagonal terms preserve channel identity: w_{AA} carries the tendency for proposal to reinforce proposal, while w_{NN} carries the tendency for constraint to reinforce constraint. The off-diagonal terms carry the history of cross-effect: w_{AN} records how constraint has learned to shape, inhibit, or redirect proposal, while w_{NA} records how proposal has learned to recruit, revise, or disturb constraint. In this sense the 2×2 operator is not just a computational transform. It is retained consequence.

Each weight is a small memory of how previous A/N encounters were selected, corrected, and stabilized. The Chevron edge therefore stores not merely association, but the learned pattern of opposition: what tends to amplify, what tends to veto, what tends to reverse, and what tends to endure.

The R-rule describes how the local state [should move](#) when the system is unresolved. Change is greatest where commitment is uncertain and weakest where the system is already settled.

Inertial VSR then describes what happens over time: variations are proposed, some are selected, some are retained, and what is retained biases future variation.

In short:

The Hypercube gives the field.

The Chevron gives the local form.

The R-rule gives the motion.

Inertial VSR gives the memory.

2. A Chevron node as a local VSR machine

The simplest connection is between Chevron Networks and Inertial VSR.

Variation-Selection-Retention is usually described as a process spread over time. Variations arise. Environments select among them. Some results are retained. Inertia is added when retained results do not simply disappear, but become constraints on future variation.

A Chevron node internalizes this process.

Inertial VSR / Chevron interpretation

Variation A-channel proposes or varies Selection A/N tension is gated by context

Retention N-channel and learned weights preserve consequence

Inertia retained state biases future updates

The A-channel is not simply “positive” and the N-channel is not simply “negative.” They are roles in a local process.

A can carry movement, salience, hypothesis, action, novelty, or pressure to adapt. N can carry constraint, memory, veto, baseline, prior expectation, or norm. Their relation is the important quantity.

A scalar unit says:

here is a value.

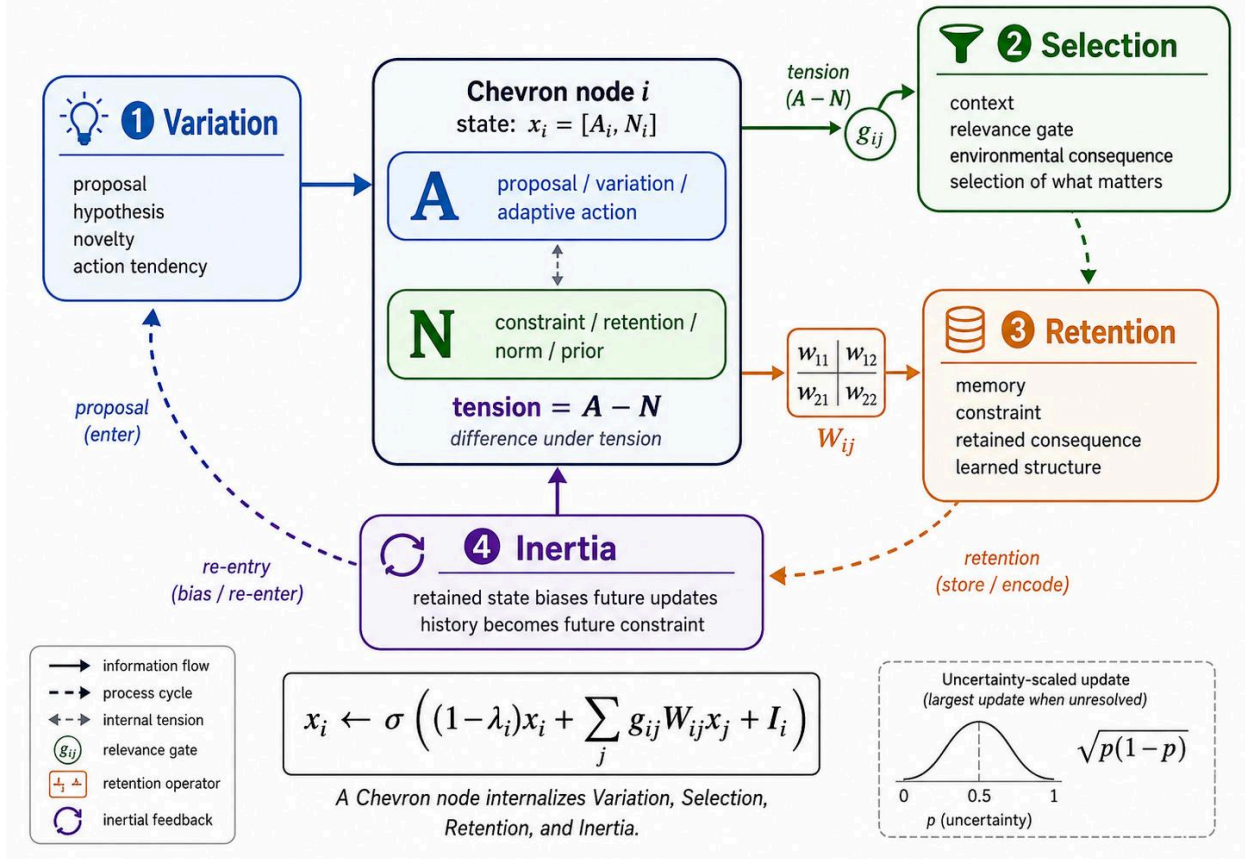
A Chevron unit says:

here is a difference under tension.

When that tension is selected, transformed, and retained, it becomes more than a temporary contrast. It becomes part of the system's future structure.

This is why Chevrons can be read as local VSR machines. They do not merely process inputs. They provide a small substrate in which proposal, constraint, selection, retention, and re-entry can happen locally.

A Chevron Node as an Inertial VSR Machine



3. Bayes, Price, and repeated selection

This connection is not limited to neural networks.

Bayesian inference and evolutionary selection have a similar mathematical form. In Bayesian inference, a prior distribution over hypotheses is reweighted by likelihood and normalized into a posterior:

$$P(H_i|E) = \frac{P(E|H_i)P(H_i)}{\sum_j P(E|H_j)P(H_j)}$$

$$P(H_i|E) = (P(E|H_i)P(H_i)) / \text{Sum}_j(P(E|H_j)P(H_j))$$

In simple replicator dynamics, a current population share is reweighted by fitness and normalized into the next population share:

$$q'_i = \frac{q_i w_i}{\sum_j q_j w_j}$$

$$q_{i'} = \{q_i w_i\} / \text{Sum}_j(q_j w_j)$$

The two equations are not identical in interpretation, but the shared structure is clear:

current weight $\times$ selection term $\rightarrow$ normalized future weight

In Bayesian inference, the possibilities are hypotheses and the selection term is likelihood. In evolutionary dynamics, the possibilities are variants or lineages and the selection term is fitness. In both cases, a distribution of possibilities is transformed by a selective encounter, and the result becomes the starting point for the next round.

This suggests a broader reading:

Domain Possibilities Selection term Retention

Bayes Hypotheses Likelihood Posterior-as-next-prior

Evolution Variants or lineages Fitness Next population

Control Possible actions Error or return Updated controller

Language Possible moves Use,correction,consequence Practice

Chevron Network A/N states Gate,tension,context

N-channel,weights,inertia

Bayesian inference can be viewed as VSR in belief-space. Evolutionary selection can be viewed as VSR in population-space. Chevron learning can be viewed as VSR in semantic-state-space.

The point is not to collapse all these domains into one equation. The point is that they share a repeated pattern: possibilities are generated, selectively reweighted, retained, and re-entered into the next cycle.

This is the mathematical backbone of Inertial VSR.

4. Selection is internal and external

A simple selection story can make the environment look external and the organism look passive. The world selects. The system adapts.

That is incomplete.

A living system does not merely receive evidence. It samples, acts, attends, avoids, approaches, tests, and modifies its environment. It selects what will count as evidence by how it moves through the world.

This is Bateson's point about the organism-in-the-environment. The relevant unit is not the organism alone, and not the environment alone, but the loop that joins them.

In Chevron terms, selection is both A and N.

A proposes, acts, attends, and varies. N constrains, remembers, vetoes, and stabilizes. The environment returns consequences. Those consequences alter the next internal state. The next internal state alters future action and sampling.

The loop is:

retained constraint → proposal → action → environmental consequence
→ correction → retention

This is why mind-like processes are better understood as embedded and enacted than as computation sealed inside a boundary.

The boundary matters. But the loop is primary.

5. The R-rule as the pulse of unresolved tension

The R-rule adds a local update principle to this picture.

A system should not update equally everywhere. It should update most where commitment is unresolved.

Given a temporary commitment value (p), the factor:

$$\sqrt{p(1-p)}$$

$\text{sqrt}(p(1-p))$

is largest near ($p = 0.5$), where the system is undecided, and smallest near ($p = 0$) or ($p = 1$), where the system is already committed.

This gives the local update a useful interpretation.

Settled commitments resist change. Unresolved tensions invite reorientation.

In a Chevron setting, this means that an A/N pair is most plastic when the proposal and constraint have not yet settled into a clear stance. The system does not merely add an error correction. It reorients under tension.

This is the pulse of the Mandala.

Inertial VSR supplies the developmental cycle. The R-rule supplies the local geometry of hesitation and reorientation.

6. The Hypercube as semantic topology

The Hypercube of Opposites can now be read less as a pre-existing metaphysical map and more as an emergent semantic topology.

A system repeatedly encounters differences. Some differences are selected. Some selected differences are retained. Some retained differences become future axes of interpretation.

Over time, this can produce a structured field of oppositions:

- true / false
- plausible / grounded
- safe / dangerous
- self / world
- permitted / forbidden
- map / territory
- freedom / duty
- knowledge / wisdom
- can / should

These oppositions are not always strict binaries. They may be gradients, polarities, complementarities, role-distinctions, tensions, or contextual contrasts.

The important point is that they organize difference.

A Semantic Chevron Network provides a local substrate for such axes.

Inertial VSR describes how some axes stabilize. The R-rule describes how unstable axes reorient. The Hypercube names the larger field carved by this history.

So the Hypercube is not merely imposed from above. It can also be understood as what repeated selection carves into semantic space.

Breakout: Inertial VSR as Feedback Loops

The basic Darwinian pattern is often written as a sequence:

Variation→Selection→Retention

Something varies. The world selects. What survives is retained.

But in living systems, minds, cultures, and learning machines, this is not a simple line. It closes into loops.

Retention_t→Variation_(t+1)

What has been retained does not merely sit in memory. It shapes what variations can appear next, what counts as salient, what feels dangerous, what seems plausible, what is even noticed.

That is the “inertial” part of Inertial VSR.

Retention has weight. It resists change. It stabilizes identity, habit, organism, language, and culture. But it also biases the next cycle of variation. A system does not vary from nowhere. It varies from what it has already become.

So the loop is better written:

$$V \rightarrow S \rightarrow R \rightarrow V'$$

Variation explores.

Selection tests.

Retention remembers.

Inertia makes memory consequential.

In the mind, there is not one such loop, but many. Some are ancient and bodily: fight, flight, freeze. These are rapid survival loops in which possible responses are generated, selected under threat, and retained as bodily habit, vigilance, trauma, courage, or caution.

Other loops are cognitive: hypothesis, test, belief. Others are social: utterance, correction, norm. Others are narrative: experience, interpretation, identity.

A disagreement about politics or social norms may appear to be a dispute between propositions, but lower loops may already have selected a bodily stance: fight, freeze, flight, affiliation, suspicion, loyalty, disgust. The symbolic loop then provides reasons after the field of possible moves has already been narrowed.

This is why signs are never just signs. A sign becomes meaningful when it enters a consequential loop:

sign→use→consequence→retained practice

Wittgenstein's language-games can be read in this way. Meaning is not hidden inside the sign. It is stabilized through use, correction, expectation, and continuation. A word varies in application; uses are selected by intelligibility and consequence; retained uses become grammar, habit, and norm.

A Chevron node gives a minimal technical image of the same pattern:

$$[A'N'] = [w_{AA} \ w_{AN}]$$

$$[w_{NA} \ w_{NN}]$$

Here A is the adaptive/proposal side: variation, action, immediate response.

N is the normative/retentive side: memory, constraint, persistence.

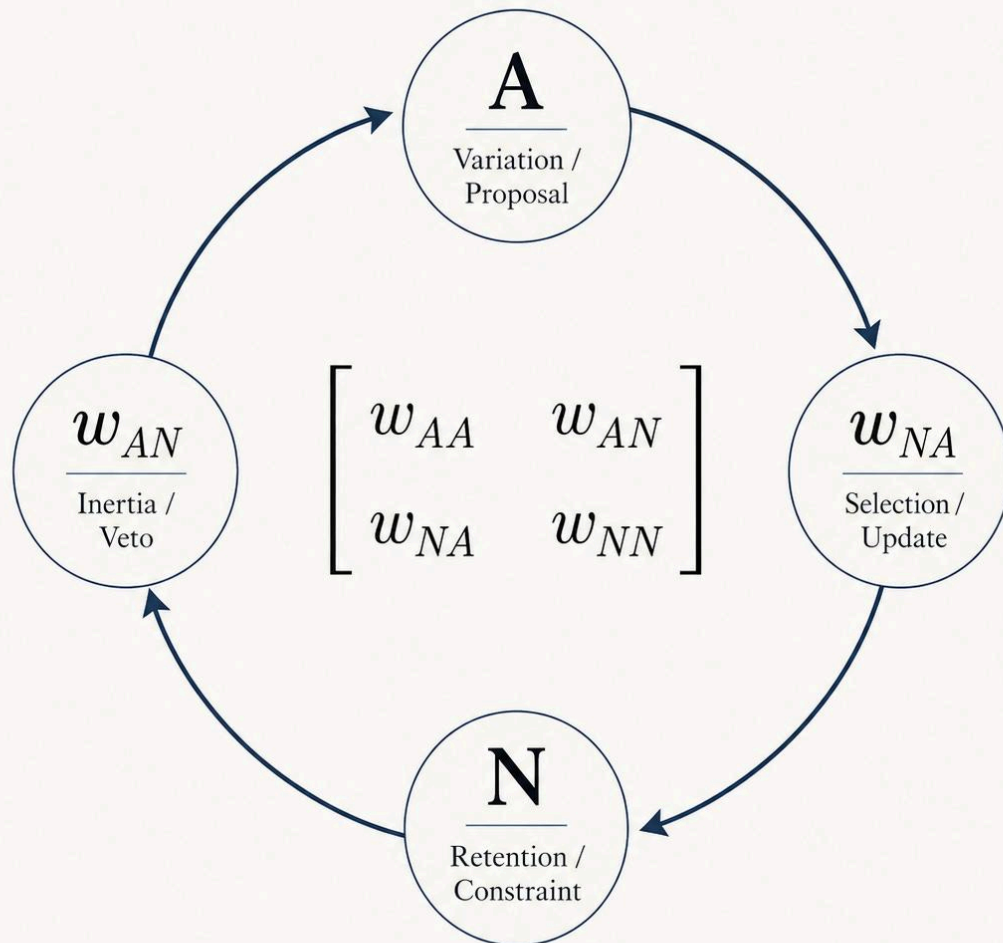
The four weights describe the loop:

- w_{AA} : variation carrying itself forward.

- w_{AN} : retention constraining variation.
- w_{NA} : selected variation updating retention.
- w_{NN} : retention preserving itself.

Inertial VSR as a Feedback Grammar

— *A / N Chevron Dynamics* —



— Variation explores. Selection tests. Retention remembers. Inertia makes memory consequential. —

So Inertial VSR is not just an evolutionary slogan. It is a general feedback grammar:

A system generates possibilities, selects among them, retains what matters, and then varies again from the retained result.

Life does this.

Brains do this.

Language does this.

Cultures do this.

Learning systems do this.

The crucial point is that retention is never neutral. It is the past made active. It is memory as constraint, habit as expectation, norm as inertia.

Without inertia, the system overfits the present and dissolves into noise.

With too much inertia, it becomes rigid and cannot learn.

Mind lives in the tension between the two.

7. More than a Turing machine, less than a cell

This substrate sits in an intermediate zone.

It is more than a Turing machine as an explanatory model, but less than a living cell.

Not more powerful than a Turing machine in the formal sense. A Turing machine can simulate any computable Chevron system. The difference is explanatory.

A Turing machine gives us symbols, rules, states, tape, and transitions.

That is enough for formal computation, but it does not explain why some differences matter.

A living cell gives us metabolism, membrane, repair, reproduction, energetic dependence, and self-maintaining organization. In a cell, certain differences matter because the continued organization of the cell depends on them.

The Mandala substrate lies between these.

Turing machine Mandala substrate Cell

Formal transition Selected difference under tension Metabolic regulation

Symbols Oppositional states Viability conditions

External semantics Proto-semantic retention Self-maintaining organization

Rule-following Consequence-sensitive re-entry Repair and reproduction

No intrinsic order of consequence Structural participation Biological participation

A Chevron substrate is not alive. It has no metabolism, membrane, repair, or reproduction. But it is also not semantically inert. It can hold differences as tensions, transform them, retain them, and let retained differences shape future interpretation.

This is the intermediate region the Mandala is meant to explore.

8. Meaning is use, but use must matter

Wittgenstein's language games are helpful here because they shift meaning away from hidden essences and toward use.

The meaning of a word is not a private object attached to it. It is found in how the word is used within a practice.

But use is not a bare mapping. Use occurs inside a form of life.

Consider the simple builder's example: one person says "slab," and another brings a slab. At first this looks like a direct association:

"slab" → slab

But the actual situation contains a field of possible contrasts.

"Slab" means slab rather than beam, pillar, block, tool, no response, wrong slab, delayed response, refusal, or misunderstanding. The context supplies the live alternatives. The language game selects one difference as relevant.

A language game therefore does not merely attach signs to objects. It selects certain differences and treats them as mattering.

At the survival level, the selected differences are enforced by the world:

- edible / poisonous
- predator / prey
- safe / dangerous
- path / obstacle
- kin / stranger

At the cultural level, the selected differences are enforced by shared practice:

- legal / illegal
- permitted / forbidden
- sacred / profane
- polite / rude
- true / false
- elegant / clumsy

At the symbolic level, differences may be selected by resonance, analogy, beauty, or cultivated attention. This is the glass bead game end of the spectrum.

The general point is:

meaning is use within an order of consequence.

A difference becomes meaningful when its selection participates in what follows.

9. The order of consequence

The phrase “order of consequence” is deliberately neutral.

It does not mean personal risk. It does not mean moral blame. It means that selected differences are embedded in a pattern of effects, corrections, continuations, and constraints.

A move in a game matters because it changes the pattern of future moves. A word in a practice matters because it coordinates or fails to coordinate action. A signal in an organism matters because it affects regulation, repair, or continuation. A hypothesis in science matters because it guides experiment, instrument-building, and future belief.

In Stoic language, the part belongs to the whole. Events do not float free. They belong to an order in which parts and whole answer to one another. Marcus Aurelius' line captures the mood: "The laws of collective expediency required this to happen."

The point is not fatalism. It is embeddedness.

A difference matters when it belongs to the order of what follows.

This is also where the Mandala begins to touch teleodynamics. A system becomes more than a tool when selected differences affect its own continuing organization. This does not make it a living cell. It does mean that the system is no longer merely transforming inputs into outputs for an external user. Its future structure is partly shaped by the differences it selects.

10. Against one-sided models

This view is compatible with several existing frameworks, but it also shifts emphasis.

Active inference

Active inference already treats organisms as systems that act to reduce expected uncertainty or free energy. This is close to the loop described here.

The caution is that the Markov blanket should not become an ontological wall. Boundaries matter, but the organism-environment loop is the larger unit of analysis. A boundary is a useful cut in a larger circuit, not a final separation.

The selfish gene

The selfish gene view (or modern synthesis) correctly emphasizes retention across generations. Genes are powerful slow structures.

But genes do not act alone. They build organisms that act, choose, modify niches, form symbioses, alter environments, and create cultures. These activities reshape the selection pressures that return upon genes.

In Chevron terms, genes are slow N-like retention structures inside a larger organism-environment process.

AI

A tool processes differences for an external user. Its outputs matter because they matter to the user or institution using it.

A more developed semantic process would be different. Its selected differences would affect its own continuing organization. It would retain consequences internally, not merely produce outputs externally.

This distinction does not require dramatic claims. It simply separates external usefulness from internal developmental consequence.

Territory, Map and Mapmaker.

11. Intelligence as simulation that feeds back

A useful way to summarize the whole argument is:

intelligence is simulation until it feeds back in.

A model may begin as an internal simulation, image, prediction, plan, or hypothesis. But once it guides action, it enters the world it represents.

The loop becomes:

world → model → action → changed world → changed model

At that point, intelligence is no longer merely representation. It is participation in a larger order of consequence.

This is why agency and intelligence are hard to separate from the whole.

A map changes the territory when organisms use maps. A scientific theory changes the experiments that are built. A self-image changes behaviour. An AI model changes the information ecology that trains future AI.

The model becomes part of the world it models.

This is also why the ecological view matters. Mind-like activity is not simply inside the agent. It is distributed across agent, world, body, tools, symbols, institutions, and history.

The loop exceeds the boundary.

12. Completing the Mandala

We can now restate the four parts more compactly.

Mandala element / Substrate role

Hypercube Field of possible meaningful differences

Chevron Local form of difference under tension

R-rule Motion of unresolved tension

Inertial VSR Retention of selected differences across time

Together they describe a process:

1. Differences arise within a field of possible contrasts.
2. Some differences are held locally as tensions.
3. Context selects which tensions matter.
4. Unresolved tensions reorient the system.
5. Selected tensions are retained.
6. Retained tensions bias future selection.
7. Over time, semantic topology develops.

The Mandala does not turn a network into a cell. It does not make every distinction a strict opposition. It does not claim that Chevron Networks are the only way to implement such a process.

Its value is more modest.

It gives us a common substrate in which inference, selection, retention, and meaning can be studied together.

A Turing machine can simulate the process. A cell lives the process. A Chevron substrate begins to organize the process.

Between formal computation and biological life, there may be a middle region where selected differences acquire structure, history, consequence, and use.

13. Summary

The argument can be compressed into a few statements:

- Meaning begins with selected difference.
- Selected differences matter when they enter an order of consequence.
- Inertial VSR describes how selected differences become retained structure.
- Chevron Networks provide a local substrate for proposal, constraint, tension, selection, and retention.
- The R-rule gives unresolved tension a local update geometry.
- The Hypercube names the semantic field carved by repeated selection.
- Intelligence becomes consequential when simulation feeds back into the world it simulates.
- Mind is not isolated inside a system; it appears in loops that exceed the boundary.

The completed Mandala is therefore not a diagram of separate mechanisms. It is a way of describing how differences can become structured, retained, and consequential across scales.

Or more simply:

The Hypercube gives the field.

The Chevron gives the local form.

The R-rule gives the motion.

Inertial VSR gives the memory.

The order of consequence gives the system its depth.



The horse/rider metaphor and the prompter/LLM relation describe the same pattern at different scales: a fast generative system coupled to a slower selective system. Neither side is complete alone. The horse

without the rider has movement without reflective direction; the rider without the horse has intention without force. The LLM without the prompter has generative fluency without situated judgment; the prompter without the LLM has intention without that particular larger field of variation.



This feels like a clearer viewpoint on the journey so far — not a final answer, but a better place from which to see the path. Thanks for following along.

Andre with ChatGPT-5.5

May 2026